A Little R Survival Kit

Eric S. Ho, Ph.D.

2026-06-16

Table of contents

Preface **6**
Usage . . . 6

I Part I **8**

1 Basic R **10**
Learning Objectives: . . . 10
1.1 R Variables . . . 10
1.2 R Objects . . . 12
1.2.1 Data Types . . . 12
1.2.2 Atomic and Composite Variables . . . 12
1.2.3 Vectors . . . 14
1.2.4 Factors/Categorical Variables . . . 16
1.2.5 Data Frames . . . 17
1.3 Descriptive Statistics . . . 23
1.3.1 Non-robust Statistics . . . 23
1.3.2 Robust Statistics . . . 24
1.3.3 `summary()` Function . . . 25
1.4 Utility Functions . . . 26
1.4.1 Inspection Functions . . . 26
1.4.2 Printing and Formatting . . . 27
1.4.3 `seq()` - create a series of numbers . . . 29
1.4.4 `with()` . . . 31
1.5 Summary . . . 32
1.6 Exercises . . . 32

2 Data Wrangling **35**
Learning Objectives: . . . 35
2.1 Base R . . . 35
2.1.1 Select Rows by Condition(s) . . . 35
2.1.2 Random Sampling . . . 39
2.1.3 Sort Items . . . 41
2.1.4 Rearranging Columns . . . 44
2.1.5 Focus on a Subset of Columns . . . 45

2.1.6 Add a New Column . . . 45
2.1.7 Remove a Column . . . 46
2.1.8 Tabulation . . . 47
2.2 Exercises for Base R . . . 48
2.3 `tidyverse` . . . 49
2.3.1 Activate `tidyverse` . . . 49
2.3.2 Select Rows by Condition(s) . . . 50
2.3.3 Select Rows Randomly . . . 51
2.3.4 Sort Rows in a Data Frame . . . 52
2.3.5 Reorganize columns of a data frame . . . 54
2.3.6 Focus on a Subset of Columns . . . 54
2.3.7 Add a New Column . . . 55
2.3.8 Remove a Column . . . 55
2.3.9 Rename a Column . . . 56
2.3.10 Pipe . . . 56
2.3.11 Tabulation and Summarize . . . 58
2.3.12 A Summary of `tidyverse` functions . . . 61
2.4 Exercises for `tidyverse` . . . 61
2.5 Summary . . . 62

3 Data Visualization **63**
Learning Objectives: . . . 63
3.1 Types of Graphs . . . 63
3.2 Base R Graphics . . . 64
3.2.1 One Numeric Variable . . . 64
3.2.2 One Categorical Variable . . . 69
3.2.3 One Categorical Variable and One Numeric Variable . . . 73
3.2.4 Contingency Table . . . 84
3.2.5 Two Numeric Variables . . . 88
3.2.6 Multipanel Scatter Plot . . . 92
3.3 Exercises for base R . . . 93
3.4 `ggplot2` . . . 94
3.4.1 Grammar of `ggplot2` . . . 94
3.4.2 One Numeric Variable . . . 95
3.4.3 One Categorical Variable . . . 102
3.4.4 One Categorical Variable and One Numeric Variable . . . 109
3.4.5 Contingency Table . . . 114
3.4.6 Two Numeric Variables . . . 116
3.4.7 Multipanel Scatter Plot . . . 118
3.5 Exercises for `ggplot2` . . . 119
3.6 Summary . . . 119

II Part II **121**

4 Statistical Tests **123**
Learning Objectives: . . . 123
4.1 Housekeeping . . . 123
4.2 Tests for Quantitative Variables . . . 125
4.2.1 One-sample t-tests . . . 125
4.2.2 One-sided t-tests . . . 126
4.2.3 Two-sample t-tests . . . 127
4.2.4 One-way ANOVA . . . 128
4.2.5 Paired t-tests . . . 130
4.3 Tests for Categorical Variables . . . 131
4.3.1 One Categorical Variable . . . 131
4.3.2 Two Categorical Variables . . . 132
4.4 Regression . . . 133
4.4.1 Simple Linear Regression . . . 133
4.4.2 Multiple Linear Regression . . . 135
4.5 Summary . . . 138
4.6 Exercises . . . 139

5 Dive Deeper **140**
Learning Objectives: . . . 140
5.1 Recreate a Summary Table in R . . . 140
5.1.1 List Objects . . . 142
5.2 Create a small data frame with data in R . . . 143
5.2.1 Use `data.frame()` Function . . . 143
5.2.2 Use `read.table()` Function . . . 144
5.3 Load Data Sets into RStudio . . . 145
5.3.1 `read.csv()` and `read.table()` Functions . . . 145
5.3.2 Import data in RStudio . . . 148
5.4 Define Functions . . . 149
5.5 The Apply-Like Family . . . 151
5.5.1 Mechanic of Apply-Like Functions . . . 151
5.5.2 `lappy()` . . . 151
5.5.3 `sapply()` . . . 153
5.5.4 `apply()` . . . 153

6 R and RStudio **156**
Learning Objectives: . . . 156
6.1 R vs. RStudio . . . 156
6.2 Accessing RStudio . . . 157
6.3 Basic Components of RStudio . . . 157
6.3.1 Files/Plots/Help . . . 158

6.3.2 Console 159
6.3.3 Environment/History 160
6.3.4 Editor 161
6.4 Summary 161

Preface

In the 21st century, nobody does statistics by pen and paper. Those statistical tables found at the back of statistics textbooks belong in the museum. The statistics you hear about in the news and research are produced by computers, the theme of this little book.

This book uses R. You may ask why not use a spreadsheet, such as Microsoft Excel or Google Sheets. While they are excellent tools - easy to use - they have significant shortcomings. In particular, they lack automation, relying on point-and-click, drag-and-drop, etc., which hampers reproducibility.

This book is written for readers who have scant experience in computer programming or scripting. R is a big language. At times, it is dizzying to navigate a vast amount of materials in the ocean of the Internet. Therefore, this book intends to provide readers with the bare minimum of R to start analyzing data. Specifically, I want them to focus on using R quickly and getting things done, rather than getting bogged down in the language's technical details and cryptic syntax. Although most people think a book like this is about R programming, I prefer to call it **scripting**. Programming encapsulates the development of algorithms - a step-by-step procedure to tackle a problem like a cooking recipe. On the other hand, scripting is more straightforward. Most of the time, a task can be solved by a few lines of code. Therefore, here we focus on the what (the expected results) instead of the how (algorithms). You tell the R functions: here is the data and the parameters; please process them and return the results.

Usage

The book consists of two parts. Part I focuses on the basics. Chapter 1 discusses R objects and operations that act on these objects, including descriptive statistical functions. Chapter 2 deals with data wrangling - the process of acquiring, selecting, and organizing data stored in a spreadsheet-like format. Both base R and `tidyverse` data wrangling functions will be discussed. Part I is concluded with data visualization in Chapter 3. A picture is worth a thousand words. The chapter offers guidelines for choosing the most suitable graph according to the intended message and properties of data. Both base R and `ggplot2` graphical systems will be discussed. Part II covers common statistical tests in Chapter 4. For readers who want to dive deeper into intermediate R functions, Chapter 5 discusses R functions that create or import data sets from files, restructure data sets, and accelerate execution time. Chapter 6

provides an overview of RStudio, an Interactive Development Environment (IDE) for crafting R scripts. If R is new to you or you don't have access to RStudio, start with Chapter 6 and then jump back to Chapter 1.

Lastly, this is a live book that I will continue to update with topics to help readers like you. I invite you to check back periodically to discover new material.

Part I

Part I

Part I has three chapters, moving from basics to data wrangling to data visualization. Beginners are advised to read them in sequence. The first chapter discusses R objects and operations that act on these objects, including descriptive statistical functions. Chapter 2 deals with data wrangling - the process of acquiring, selecting, and organizing data stored in a spreadsheet-like format. Both base R and `tidyverse` data wrangling functions will be discussed. Chapter 3 focuses on data visualization. A picture is worth a thousand words. The chapter offers guidelines for choosing the most suitable graph based on the intended message and the data's properties. Both base R and `ggplot2` graphical systems will be discussed.

If R is new to you or you don't have access to RStudio, start with Chapter 6 and then jump back to Chapter 1.

1 Basic R

Learning Objectives:

The journey of R scripting starts with a basic understanding of R objects and commands. After finishing this chapter, you should be able to:

- Define R variables.
- Know the differences between an atomic variable and an object variable.
- Store results in variables through the assignment operator.
- Perform arithmetic operations.
- Handle composite objects: vectors and data frames.
- Define categorical variables.
- Calculate descriptive and summary statistics.
- Utilize inspection functions to figure out basic information about data.
- Format and print variable contents to the console.
- Use `seq()` and `with()` functions.

As R scripting is a hands-on topic, I encourage you to open RStudio alongside this book and try out the code to maximize understanding. Let's get started.

1.1 R Variables

An R variable is a named location in the computer's memory that stores some values. Imagine it as a labeled (name) shoebox (memory location) keeping a pair of sneakers (value).

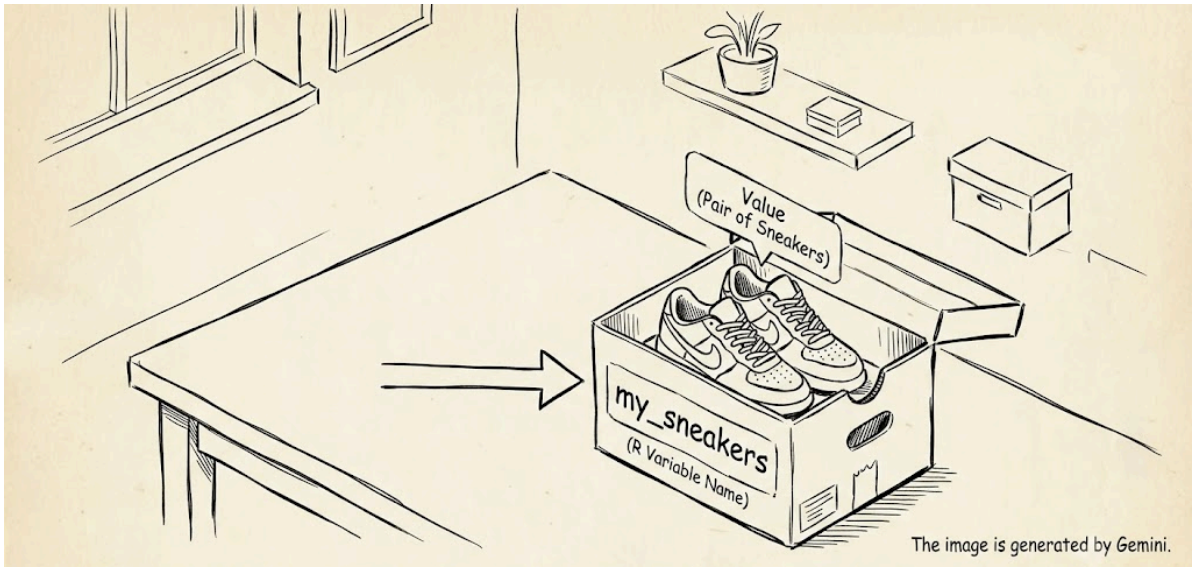


Figure 1.1: Illustration 1.1 R variables and shoeboxes.

Two rules determine how an R variable can be named:

1. It must begin with a letter (A-Z or a-z), a dot ".", or an underscore "_", and
2. Subsequent characters can be any combination of letters, digits (0-9), ".", or "_".

Note that:

- A variable name can be as short as one character or as long as a normal person wishes (10,000 characters!).
- My suggestion is to give variables meaningful names that help you or someone else comprehend the code. For instance, in a problem about heart rate, use `zone2` instead of `z` or even worse `x`.
- Be careful that variable names are case-sensitive, e.g., `volume` and `Volume` are different in the eyes of R.
- Avoid naming variables with names that have been reserved by R (https://rdrr.io/r/base/Reserved.html) or names already used to name R's built-in functions, e.g., `c`, and `head`. For the latter, unfortunately, R is permissive for misnaming variables by remaining silent, costing time to identify the issue. Therefore, the rule of thumb is to exercise self-discipline and avoid using R function names as variable names.

1.2 R Objects

Every variable is an R object. In scripting, it is important to know the type(s) of value(s) being stored in a variable, as the types dictate the legitimate operations that can be performed on the variable. Let's begin with data types.

1.2.1 Data Types

R supports six data types:

- Double: decimal numbers, e.g., 5.6, -4.3
- Integer: whole numbers, e.g, -10, 56
- Character: they contain text, e.g., "ACGT", "Hello World"
- Logical: `TRUE` or `FALSE`
- Complex*: Complex numbers containing real and imaginary components, e.g., 5+2i
- Raw*: Bare bytes of data.

* We will only discuss the first four types in this book; you can ignore complex and raw types.

1.2.2 Atomic and Composite Variables

From a practical perspective, R variables can be viewed as **atomic variables** and **composite variables**. An atomic variable contains a single value, and a composite variable consists of a collection of values. Let us begin with two atomic variables: **`numeric`**, and **`character`**.

1.2.2.1 Numeric Variables

An atomic numeric variable holds a decimal number (`double`) or a whole number (`integer`). Numbers can be represented in scientific notation. Here are some examples:

```
# Decimal
height <- 5.6

# Whole number (Integer)
sample_size <- 15L

# Scientific notation
avogadro_number <- 6.023e23
```

`<-` is the assignment operator. It looks like a left-pointing arrow, depicting the transfer of value from the right-hand side to the left-hand side. Upon execution, the resultant value of the expression on the right-hand side, if no error occurs, is stored in the named variable on the left-hand side. In RStudio, you can see the variable name and its associated value under the **Environment** tab, which usually occupies the top right quadrant of the RStudio desktop.

Note that R supports another assignment operator: `=`. You can use `<-` and `=` interchangeably. I prefer `<-` to `=`, as `=` can easily confuse with the equality checking operator `==`.

A few words about the scientific notation. It comprises three parts: the coefficient, a constant literal`e`, and the exponent. The `e` refers to the base `10`. For example, the R's scientific notation of 6.023×10^{23} is `6.023e23`, where the coefficient and the exponent are 6.023 and 23, respectively. Note that you must not include spaces in scientific notation, e.g., `6.023e 23`, `6.023 e23`, or `6.023 e 23` will return errors.

Arithmetic Operators

You can apply arithmetic operations on numeric variables. Here are the arithmetic operators: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `**` (exponentiation), and `^` (exponentiation). The last two exponentiation operators perform the same function. Here are some examples:

```
radius <- 2.5
diameter <- 2*pi*radius
pi <- 3.1416
area <- pi*radius**2
volume <- (4/3)*pi*radius^3
```

1.2.2.2 Character Variables

R also supports text values, which are in `character` data type. To define a character value, you need to use a pair of quotes to enclose the text. Either a pair of single (`'`) or double quotes (`"`) works. E.g.,

```
course <- 'BIOL265'
```

To figure out whether a variable's type is numeric, character, or something else, use the `typeof()` function,

```
typeof(course)
```

```
[1] "character"
```

```
typeof(area)
```

```
[1] "double"
```

Obviously, it does not make sense to do math with character type. E.g.,

```
course*4
```

```
Error in `course * 4`:
! non-numeric argument to binary operator
```

You can use `nchar()` function to figure out the number of characters, including spaces and punctuation, of a character variable. E.g.,

```
nchar(course)
```

```
[1] 7
```

1.2.3 Vectors

One common composite variable in R is **`vectors`**. It is an ordered sequence of items that can be processed collectively as a whole, or individually. A vector can be viewed pictorially as a wagon train in which carriages are coupled one after the other. E.g.,

```
# A simple numeric vector for the days in each month
days_of_month <- c(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
```

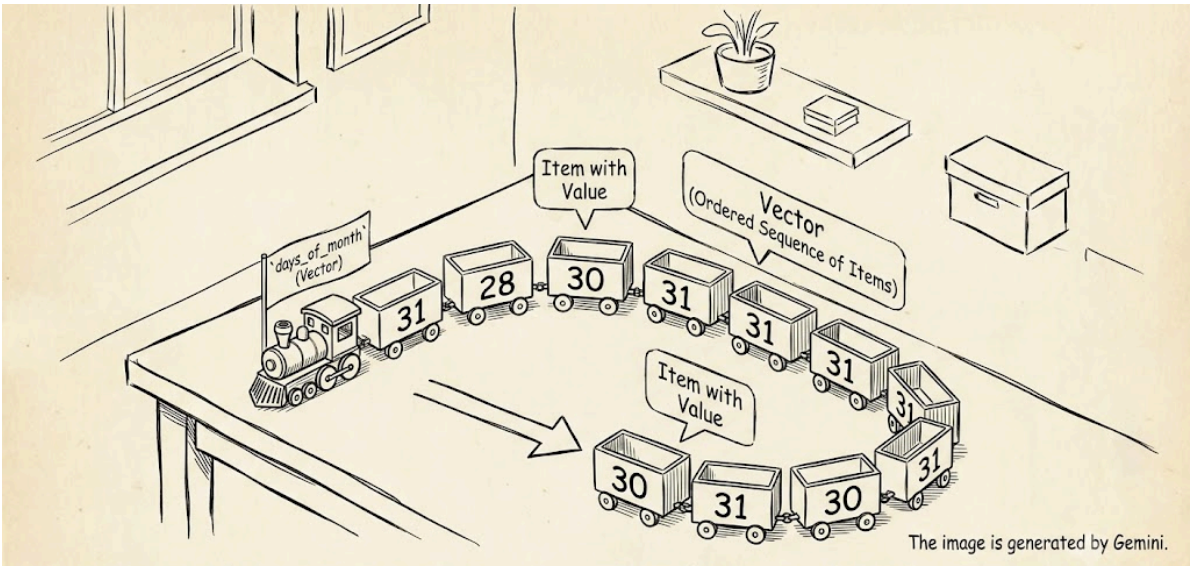


Figure 1.2: Illustration 1.2 Vectors.

`days_of_month` is a `vector` object in R. `c()` is the function that **combines or collates** several objects into a vector object. Items are indexed or numbered from 1 to the total number of items in the vector. You can extract a particular item by specifying the index within a pair of square brackets. E.g.,

```
# Days in February
print(days_of_month[2])
```

```
[1] 28
```

The `[2]` means the 2nd element of a vector. We will discuss indexing in more detail below.

Here is an example of a character vector:

```
nucleotides <- c('A','C','G','T')
```

Noteworthy that all items held inside a vector must have the same data type. In other words, vectors are always **homogeneous**. For instance, `days_of_month` contains all numeric values, and `nucleotides` contains all character values. If you try to combine objects of different data types into a vector, R will automatically convert them to character type, the lowest denominator of data types. In the example below, even though the first and third values are entered as numeric, R coerces them into text `"1"` and `"0"`.

```r
a_vector <- c(1, "success", 0, "fail")
a_vector
```

```
[1] "1"       "success" "0"       "fail"
```

One of the R features that makes arithmetic operations on vectors really convenient is **vectorization**. For instance, if you want to standardize the values of a sample such that the sample mean and sample variance after standardization become 0 and 1, respectively. The formula for standardization is $z_i = \frac{(x_i - \bar{x})}{s}$, where $\bar{x}$ and s are the sample mean and sample standard deviation, respectively. Here's an example

```r
heights <- c(5.5, 6.0, 6.1, 5.7, 5.9, 5.6)
x_bar <- mean(heights)
s <- sd(heights)
std_heights <- (heights - x_bar)/s
std_heights
```

```
[1] -1.2677314  0.8451543  1.2677314 -0.4225771  0.4225771 -0.8451543
```

`mean()` and `sd()` are built-in functions that calculate the average and standard deviation of a sample, respectively.

The convenience is that you don't have to specify how to subtract and divide each height individually. Instead, R recognizes the expression `(heights-x_bar)` is to subtract an atomic `x_bar` (scalar) from a vector `heights`. Under the vectorization mechanism, it will subtract `x_bar` from every height in the vector `heights`, similarly for the division by `s`.

`vector` is only one of the many types of objects supported by R. A data frame is another commonly used object type, which will be discussed later.

1.2.4 Factors/Categorical Variables

In R, a **factor** refers to a categorical variable in statistics. A categorical variable is further classified into **nominal** or **ordinal**. A nominal variable has no intrinsic order. Use the `factor()` function to define a categorical variable in R. E.g., a sample of colors is stored in a vector, and you want to define it as a nominal categorical variable.

```r
colors <- c("Red","Blue","Blue","Red","Green","Green","Blue","Green","Red","Blue")
colors <- factor(colors, levels=c('Red','Blue','Green'))
colors
```

```
 [1] Red   Blue  Blue  Red   Green Green Blue  Green Red   Blue
Levels: Red Blue Green
```

Categorical variables with intrinsic order are called **ordinal** variables. E.g., the stages of hypertension.

```
patients <- c("Stage 2","Hyperintensive","Stage 2","Hyperintensive","elevated","Stage 2","el
patients <- factor(patients,
                   levels=c('normal','elevated','Stage 1','Stage 2','Hyperintensive'),
                   ordered = TRUE)
patients
```

```
[1] Stage 2        Hyperintensive Stage 2        Hyperintensive elevated
[6] Stage 2        elevated       Stage 1
Levels: normal < elevated < Stage 1 < Stage 2 < Hyperintensive
```

As you can see above, the last line (`Levels`) shows the hypertension levels in ascending order, and levels are connected by a less than sign (`<`).

1.2.5 Data Frames

Besides `vector`, another common R object is `data frame`. Almost all the datasets used for data analysis are organized in data frames. Simply speaking, a data frame is a **tabular** or spreadsheet-like object where values are organized in rows and columns. Each row represents a sample, and each column represents a variable. Rows are expected to have equal length, i.e., the same number of columns. If not, R will either complain or pad the missing columns with a special value `NA` (it stands for not available). Let's take a look at R's built-in data frame `iris`:

```
# Show the first five rows
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

You can peek into the bottom five rows using the `tail()` function.

```
tail(iris)
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
145          6.7         3.3          5.7         2.5 virginica
146          6.7         3.0          5.2         2.3 virginica
147          6.3         2.5          5.0         1.9 virginica
148          6.5         3.0          5.2         2.0 virginica
149          6.2         3.4          5.4         2.3 virginica
150          5.9         3.0          5.1         1.8 virginica
```

As you can see above, the `iris` data frame consists of five variables (columns). Here are the R functions to show the shape of a data frame:

```
# the dimension (row x column) of a data frame.
dim(iris)
```

```
[1] 150   5
```

```
# Only the number of rows
nrow(iris)
```

```
[1] 150
```

```
# Only the number of columns
ncol(iris)
```

```
[1] 5
```

So, we know that `iris` contains 150 rows (flower samples) and five columns.

1.2.5.1 Accessing Rows and Columns of a Data Frame

Each element in a data frame can be referenced by row and column indexes like an X-Y Cartesian system. Rows are automatically indexed (numbered), where the 1st row is index 1, the 2nd row is index 2, and the index of the last row is the total number of rows; similarly for columns.

The syntax of accessing individual elements by indexes is `<data frame name>[row, column]`. E.g., the cell at the 5th row and 3rd column of the iris is:

```
iris[5,3]
```

```
[1] 1.4
```

If the column index is ignored, all columns of the specified row(s) are returned as shown in Figure 1.1. For example, show all columns of the 3rd row by:

```
# Specify the row number only and leave out the column index.
iris[3,]
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
3          4.7         3.2          1.3         0.2  setosa
```

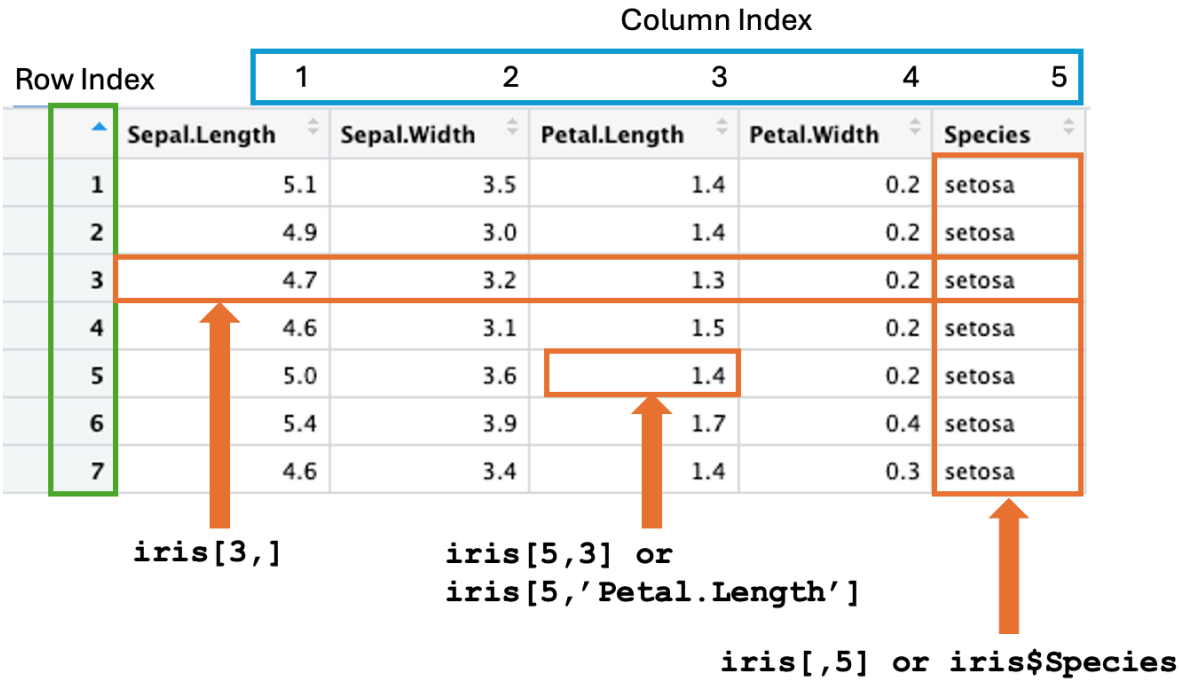


Figure 1.3: Accessing rows and columns of a data frame.

Similarly, if the row index is skipped, all rows of the specified column(s) are returned. For example,

```
# Specify only the column index.
head(iris[,5])
```

```
[1] setosa setosa setosa setosa setosa setosa
Levels: setosa versicolor virginica
```

Note that the `head()` above is used to limit the number of rows displayed by avoiding listing page after page of data rows.

You are not limited to display one row or one column using this indexing scheme. For example, show to 110th and 34th rows and 2nd and 5th columns,

```
iris[c(110,34), c(2,5)]
```

```
    Sepal.Width   Species
110         3.6 virginica
34          4.2    setosa
```

As columns are often named [1], columns can be referenced by their names as well (Figure 1.3). E.g., the `Petal.Length` of the 5th row:

```
iris[5,'Petal.Length']
```

```
[1] 1.4
```

To access more than one column and row, pack the row indexes and column names in vectors using the `c(...)` function, which **c**ombines or **c**ollates several items into a vector object (see Section 1.2.3). For example, show the `Petal.Length` and `Sepal.Length` of the 123th and 96th rows:

```
iris[c(123,96),c('Petal.Length','Sepal.Length')]
```

```
    Petal.Length Sepal.Length
123          6.7          7.7
96           4.2          5.7
```

[1] If rows are named, rows can be referenced by their names.

If your task is to extract a single column, R provides an alternative way to access a column by its name. The syntax is `<data frame name>$<column name>` [2]. For example, `iris$Species`, which is equivalent to `iris[,5]`,

```
head(iris$Species)
```

```
[1] setosa setosa setosa setosa setosa setosa
Levels: setosa versicolor virginica
```

We have covered several ways to access rows, columns, or both using indexes and column names. There are additional methods to select rows and columns by conditions. More of this topic will be discussed in Chapter 2.

1.2.5.2 Create a Data Frame

We can use the `data.frame()` function, using vectors, to create a dataset in data frame format. For example, a study collected 10 fish with their sex and age. To create the data frame, vectors must be created to represent each variable (column). Here's the R code:

```
sex <- c('F','M','M','F','F','F','M','M','M','M')
age <- c(1,3,2,3,1,1,3,3,1,1)
fish <- data.frame(sex,age)
fish
```

```
   sex age
1    F   1
2    M   3
3    M   2
4    F   3
5    F   1
6    F   1
7    M   3
8    M   3
9    M   1
10   M   1
```

You can use the `summary()` function to examine the summary statistics of the variables.

[2] The `$` method only works for a single column.

```
summary(fish)
```

```
        sex          age       
 Length   :10   Min.   :1.0  
 N.unique : 2   1st Qu.:1.0  
 N.blank  : 0   Median :1.5  
 Min.nchar: 1   Mean   :1.9  
 Max.nchar: 1   3rd Qu.:3.0  
                Max.   :3.0  
```

Find out more information about the `summary()` function below Section 1.3.3.

It is almost done. There is one nuance. In the current form, `sex` is a character variable, so its summary statistics are a bit strange. Most often, variables like that should be categorical/nominal. Therefore, we need to convert `sex` into a `factor` Section 1.2.4. Here's the revised code:

```
sex <- c('F','M','M','F','F','F','M','M','M','M')
sex <- factor(sex, levels=c('F','M'))
age <- c(1,3,2,3,1,1,3,3,1,1)
fish <- data.frame(sex,age)
fish
```

```
   sex age
1    F   1
2    M   3
3    M   2
4    F   3
5    F   1
6    F   1
7    M   3
8    M   3
9    M   1
10   M   1
```

See the differences for `sex` after revision.

```
summary(fish)
```

```
 sex        age
 F:4   Min.   :1.0
 M:6   1st Qu.:1.0
       Median :1.5
       Mean   :1.9
       3rd Qu.:3.0
       Max.   :3.0
```

1.3 Descriptive Statistics

1.3.1 Non-robust Statistics

Sample mean, variance, and standard deviation are non-robust statistics, as they are sensitive to outliers (extreme values).

```
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62)
```

Sample Mean

```
# Sample mean or the average weight
mean(pumpkins)
```

```
[1] 6.33
```

Sample Variance

```
# Sample variance
var(pumpkins)
```

```
[1] 4.013489
```

Sample Standard Deviation

```
# Sample standard deviation
sd(pumpkins)
```

```
[1] 2.003369
```

By the way, the square root of the sample variance $\sqrt{variance}$ is the standard deviation. See below:

```
sqrt(var(pumpkins))
```

```
[1] 2.003369
```

1.3.2 Robust Statistics

Median and quantiles are called robust statistics, as they reduce the influence of outliers.

Median

```
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62)
```

```
median(pumpkins)
```

```
[1] 6.59
```

Quantile

The default quantile interval, as the name suggests, is 25%. Note that the R function name is `quantile()`, not ~~`quartile()`~~, as users can override the default 25% interval.

```
quantile(pumpkins)
```

```
    0%    25%    50%    75%   100%
 3.520  4.895  6.590  7.375 10.140
```

Instead of the default 25% interval, you can specify your preferred quantile(s), and they don't have to be equally divided. Here are some examples:

```
quantile(pumpkins, probs = c(0.1,0.9))
```

```
  10%   90%
3.736 7.872
```

Or, a specific quantile.

```r
quantile(pumpkins, probs = c(0.05))
```

```
   5%
3.628
```

```r
quantile(pumpkins, probs = c(0.95))
```

```
  95%
9.006
```

Inter-Quartile Region (IQR)

Here is the definition of IQR:

$$IQR = Q3 - Q1$$

```r
IQR(pumpkins)
```

```
[1] 2.48
```

IQR is used to determine the upper and lower whiskers in a boxplot. Data points below or above the lower or upper whisker are considered outliers.

$$Lowerwhisker = Q1 - 1.5 \times IQR$$
$$Upperwhisker = Q3 + 1.5 \times IQR$$

1.3.3 `summary()` Function

A data frame may consist of many variables of different types in various ranges. Therefore, it makes a lot of sense to take a glimpse of all variables before analysis. It can be done by using the `summary()` function to reveal:

1. The variable type of each column.
2. If the column is numeric, a summary of numeric values.
3. If the column is categorical (factor), what are the levels and order, if it is ordinal.

Let's see the summary of `iris`:

```
summary(iris)
```

```
  Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
 Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
 Median :5.800   Median :3.000   Median :4.350   Median :1.300
 Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
 3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
       Species
 setosa    :50
 versicolor:50
 virginica :50
```

The summary will show the range (min. and max.) and quantiles of a numeric column. For a categorical variable (factor), it will show the levels with the number of samples (rows) found for each level, e.g., `Species` is a nominal variable, with 50 samples per species.

1.4 Utility Functions

1.4.1 Inspection Functions

Before diving into data analysis, we often want to know some basic information about it, such as the number of samples, what is the range of a numeric variable, etc. Here we discuss a few R functions to do that, and we call them inspection functions.

We will use a sample of pumpkin weights for illustration.

```
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62)
```

The `length()` shows the number of items in a vector.

```
length(pumpkins)
```

```
[1] 10
```

To be clear, `length()` counts the number of items in an R composite object; the number of items in a vector or the number of columns in a data frame. **If the object is atomic, the function will return 1.** For example, in the following example, `dna` is an atomic, character variable. The function will return 1, not the number of characters.

```
dna <- 'ACGTAGC'
length(dna)
```

```
[1] 1
```

To count the number of characters in a character variable, use the `nchar()` function.

```
nchar(dna)
```

```
[1] 7
```

The `range()` prints the minimum and maximum values of an object[3].

```
range(pumpkins)
```

```
[1]  3.52 10.14
```

If you are interested in either the minimum or maximum, here are the corresponding functions:

```
# The lightest
min(pumpkins)
```

```
[1] 3.52
```

```
# The heaviest
max(pumpkins)
```

```
[1] 10.14
```

1.4.2 Printing and Formatting

R provides a function called `print()` that prints text to the console. E.g.,

[3]Note that `range()` works for a non-numeric vector, but we will skip such a special operation.

```
print("Hello World!")
```

```
[1] "Hello World!"
```

```
radius <- 2.5
diameter <- 2*pi*radius
print(diameter)
```

```
[1] 15.708
```

Suppose you want to improve the printout by making it more readable, say `"The diameter of a circle with radius 2.5 cm is 15.70796."` In such a case, you need the format function `paste()`. Here's how to use it:

```
radius <- 2.5
diameter <- 2*pi*radius
print(paste("The diameter of a circle with radius",radius,"cm is",diameter,"."))
```

```
[1] "The diameter of a circle with radius 2.5 cm is 15.708 ."
```

The above printout is almost perfect. If you're picky and don't like the extra space separating the diameter and the period in the above message, you can use `paste()`'s cousin function `paste0()`. The difference between the two is that `paste0()` will not automatically add an extra space between each component given to the function. Below is the previous print statement with `paste0()`:

```
print(paste0("The diameter of a circle with radius",radius,"cm is",diameter,"."))
```

```
[1] "The diameter of a circle with radius2.5cm is15.708."
```

As you can see, there is no more space between the diameter and the period. But it created other issues; there is no space between "The diameter of a circle with radius" and the actual radius as well. So, when you use `paste0()`, you have to take care of the space separation yourselves. Here is the final version for using `paste0()`:

```
print(paste0("The diameter of a circle with radius ",radius," cm is ",diameter,"."))
```

```
[1] "The diameter of a circle with radius 2.5 cm is 15.708."
```

1.4.3 `seq()` - create a series of numbers

There are situations where we need a series of equal-interval numbers. For example, override the default breaks - x-ticks - of a histogram, or a vector containing all possible outcomes of tossing two dice, i.e, from 2 to 12.

In R, the `seq()` function generates a list of equal-interval numbers, whole and decimal. `seq()` takes the following parameters:

1. `from = 1`, the default is 1.
2. `to = 1`, the default is 1.
3. `by = 1`, the size of the interval between two adjacent numbers. It takes integer and decimal numbers. If it is negative, the series is decreasing.
4. `length.out` specifies how many numbers are returned. This option is an alternative to `by` when you care only about how many numbers are generated instead of the interval between them. See the example below.
5. `along.with`, it generates the same number of numbers according to what is provided to the `along.with` parameter.

Examples

By default, `seq()` begins with 1 and has an interval of 1. E.g., generate 10 integers starting with 1.

```
seq(10)
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

Starting from and ending with.

```
seq(-10,10)
```

```
 [1] -10  -9  -8  -7  -6  -5  -4  -3  -2  -1   0   1   2   3   4   5   6   7   8
[20]   9  10
```

Specify the interval.

```
seq(-10,10, by=2)
```

```
 [1] -10  -8  -6  -4  -2   0   2   4   6   8  10
```

A decreasing sequence.

```
seq(10,by=-1)
```

```
 [1] 10  9  8  7  6  5  4  3  2  1
```

Just specify the desired numbers to be generated. As you can see, the interval is pretty precise.

```
seq(-5,7,length.out=20)
```

```
 [1] -5.00000000 -4.36842105 -3.73684211 -3.10526316 -2.47368421 -1.84210526
 [7] -1.21052632 -0.57894737  0.05263158  0.68421053  1.31578947  1.94736842
[13]  2.57894737  3.21052632  3.84210526  4.47368421  5.10526316  5.73684211
[19]  6.36842105  7.00000000
```

Generate a sequence that matches the same number in another sequence.

```
x <- seq(1,5)
cat("x =",x,"\n")
```

```
x = 1 2 3 4 5
```

```
y <- seq(-5, along.with=x)
cat("y =",y,"\n")
```

```
y = -5 -4 -3 -2 -1
```

Note: What is the difference between `cat(...)` and `print(paste(...))` you saw above? The main difference is that `print(paste(...))` creates an R object (a text) and then prints it, while `cat(...)` directly writes text to the console without creating a formal R object. The example below shows the nuance.

```
msg1 <- print(paste("Hello","World"))
```

```
[1] "Hello World"
```

```
print(msg1)
```

```
[1] "Hello World"
```

```
msg2 <- cat("Hello","World","\n")
```

```
Hello World
```

```
print(msg2)
```

```
NULL
```

Is it really important? No, unless you're an R developer.

1.4.4 `with()`

The function sets up the context of a data frame so it spares you from using the `$` to access the columns of a data frame, as it often leads to longer commands, which makes it harder to comprehend and troubleshoot. It is also commonly used together with a function applied to one or more columns. E.g., the median of `Petal.Width`. It can be done in two ways:

```
# Use $
median(iris$Petal.Width)
```

```
[1] 1.3
```

```
# Or, use with()
with(iris, median(Petal.Width))
```

```
[1] 1.3
```

For the straightforward example above, you do not see the value of `with()`. Let's consider a more complicated row selection example. Find `Sepal.Length` in `iris` where its `Petal.Length` is less than 4 and `Petal.Width` is greater than 1. The `$`-way is:

```
iris$Sepal.Length[iris$Petal.Length<4 & iris$Petal.Width>1]
```

```
[1] 5.2 5.6 5.6 5.5 5.8 5.1
```

Here's the `with()` version that saves typing `iris` repeatedly.

```
with(iris, Sepal.Length[Petal.Length<4 & Petal.Width>1])
```

```
[1] 5.2 5.6 5.6 5.5 5.8 5.1
```

In short, `with()` is English, so it improves the readability of R statements. Second, it saves some typing of the data frame's name. I recommend using `with()` whenever possible.

1.5 Summary

That's the bare minimum of R scripting required for data analysis. In the next chapter, we will focus on operations applied to data frames. Before you leave, it is useful to recap the R functions discussed in this chapter:

Table 1.1: A Summary of R Operators and Functions

`<-`	`head()`	`sd()`	+
`c()`	`tail()`	`range()`	-
`print()`	`dim()`	`min()`	*
`paste()`	`nrow()`	`max()`	/
`paste0()`	`ncol()`	`median()`	**
`cat()`	`data.frame()`	`quantile()`	^
`typeof()`	`summary()`	`IQR()`	==
`nchar()`	`mean()`	`seq()`	`<-`
`factor()`	`var()`	`with()`	`c()`

1.6 Exercises

Q1. Identify illegal variable names.

A. `fiveguys` B. `5guys` C. `five.guys` D. `five-guys` E. `five_guys` F. `_5guys` G. `don't care`

Q2. Identify bad variable name choices and why.

A. `head` B. `super` C. `okay` D. `c` E. `type` F. `pi` G. `end`

Q3. Identify data types not supported by R.

A. `numeric` B. `integer` C. `decimal` D. `text` E. `character` F. `logical` G. `file` H. `complex` I. `data`

Q4. Variables and arithmetic.

a) Create three variables: `a` , `b` , and `c` with values `3` , `2` , and `-8` , respectively.

b) Based on the following formula, determine the two values of `r`.

$$r = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Q5. Vectors and vectorization.

a) Create a vector containing the heights of your five best friends in inches.

b) Use the approach of **vectorization** to calculate the sample mean, sample variance, and sample standard deviation of your friends' heights, but you are **not allowed** to use R's built-in `mean()` , `var()` , and `sd()` functions. Here is the formula for sample variance:

$$s^2 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}$$

Q6. Use the `seq()` function to answer the following parts.

a) Create a vector that stores consecutive odd integers between -10 and 10.

b) Create a vector that stores consecutive odd integers between -10 and 10 in descending order.

c) Create a vector that stores 20 numbers between -10 and 10.

Q7. Factors.

a) Create a nominal categorical variable to represent the four properties of amino acids: `acidic` , `basic` , `hydrophilic` , and `hydrophobic` .

b) Create an ordinal categorical variable to represent the classification of blood pressure: `Hypotension` , `Normal`, `Elevated`, `Hypertension Stage 1`, and `Hypertension Stage 2`.

Q8. Use the built-in data set `mtcars` to answer this question.

a) List the first 10 rows of `mtcars`.

b) List the last 8 rows of `mtcars`.

c) Use an R function to show the number of rows and columns of `mtcars`.

d) Print the 13th row.

e) Print the horsepower of the vehicle in the 7th row.

f) Print the `mpg` and `hp` of all vehicles together in two columns.

g) Based on the previous part, show the minimum, maximum, and quantiles of `mpg` and `hp`.

Q9. Create a data frame of your friends that stores their name, phone number, and height.

Q10. Use the data set `mtcars` and the `with()` function, as much as possible, to answer this question.

a) Calculate the difference between the maximum and minimum horsepower `hp`.

b) Determine the median `hp`.

c) Calculate the IQR of `hp`.

d) Calculate the upper and lower whiskers of `hp`.

e) `vs` denotes engine type: 0 = V-shaped, 1 = straight. Use R to determine the number of cars with a V-shaped engine and a straight engine (don't count it manually).

2 Data Wrangling

Data wrangling refers to the process of viewing, changing, and reorganizing rows and columns in a data frame. Before we dive into the operations, it is important to know that there are two distinct schools of data wrangling: base R and `tidyverse`. The base R approach means that no additional packages are required, but the `tidyverse` approach requires the installation of `tidyverse`, even though the installation is very straightforward. In my opinion, `tidyverse` has substantially refined and enhanced data wrangling. You will see it in this chapter. Regardless, we will discuss both approaches. Let's start with the base R version.

Learning Objectives:

This chapter covers common data wrangling techniques. After finishing this chapter, you should be able to:

- Select rows that fulfill specified conditions.
- Randomly sample rows from a data frame.
- Sort rows in a data frame.
- Select a subset of columns.
- Add and remove columns to an existing data frame.
- Tabulate data.

2.1 Base R

2.1.1 Select Rows by Condition(s)

The `subset()` function selects rows from a data frame if they satisfy the condition(s). Below are six relational operators used in specifying the conditions:

1. `>` greater than.
2. `>=`[1] greater than or equal to.

[1] No spaces between the two symbols.

3. `<` less than.
4. `<=`[2] less than or equal to.
5. `==`[3] equal to.
6. `!=`[4] not equal to.

If the selection involves multiple conditions, they must be joined by relational operators. Here are the three logical operators required for multi-part conditions:

1. `|` denotes the logical OR.
2. `&` denotes the logical AND.
3. `!` denotes the logical NOT.

Let's begin how `subset()` works with vectors, followed by data frames.

2.1.1.1 `subset()` with Vectors

Let's define a vector.

```
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62, 7)
```

Select pumpkins heavier than 7 lbs.

```
subset(pumpkins, pumpkins>7)
```

```
[1]  7.55 10.14  7.62
```

Select pumpkins that are exactly 7 lbs.

```
subset(pumpkins, pumpkins==7)
```

```
[1] 7
```

Select pumpkins between 6 and 7 lbs, inclusive.

[2]No spaces between the two symbols.
[3]No spaces between the two symbols.
[4]No spaces between the two symbols.

```
subset(pumpkins, pumpkins>=6 & pumpkins<=7)
```

```
[1] 6.85 6.56 6.23 6.62 7.00
```

Let's use the logical NOT to negate the above query, meaning to select pumpkins either lighter than 6 or heavier than 7.

```
subset(pumpkins, !(pumpkins>=6 & pumpkins<=7))
```

```
[1]  7.55  4.45 10.14  3.52  3.76  7.62
```

The above query can be rewritten using the logical OR as follows:

```
subset(pumpkins, pumpkins<6 | pumpkins>7)
```

```
[1]  7.55  4.45 10.14  3.52  3.76  7.62
```

2.1.1.2 `subset()` with Data Frames

Here is an example that chooses only the `setosa` species from `iris` data frame and keeps them in a new data frame. Note that the equality check is case-sensitive.

```
setosa_iris <- subset(iris, Species == 'setosa')

# How many are selected?
print(paste("There are", nrow(setosa_iris), "rows."))
```

```
[1] "There are 50 rows."
```

```
# Peek into the first few rows of the result.
head(setosa_iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

More than one condition. Select rows with `Sepal.Length` is greater than 5 or `Sepal.Width` is smaller than 3.

```
iris_or_eg <- subset(iris, Sepal.Length > 5 | Sepal.Width < 3 )

# How many?
print(paste("There are", nrow(iris_or_eg), "rows."))
```

```
[1] "There are 124 rows."
```

```
head(iris_or_eg)
```

```
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           5.1         3.5          1.4         0.2  setosa
6           5.4         3.9          1.7         0.4  setosa
9           4.4         2.9          1.4         0.2  setosa
11          5.4         3.7          1.5         0.2  setosa
15          5.8         4.0          1.2         0.2  setosa
16          5.7         4.4          1.5         0.4  setosa
```

Select rows with `Petal.Length` is greater than 6 and `Petal.Width` is smaller than 2.

```
# More than one condition. The relational AND operator.
iris_and_eg <- subset(iris, Petal.Length > 6 & Petal.Width < 2 )

# How many?
print(paste("There are", nrow(iris_and_eg), "rows."))
```

```
[1] "There are 2 rows."
```

```
iris_and_eg
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
108          7.3         2.9          6.3         1.8 virginica
131          7.4         2.8          6.1         1.9 virginica
```

Select everything except for `setosa`.

```r
# No setosa iris
iris_no_setosa_eg <- subset(iris, Species != 'setosa')

# How many?
print(paste("There are", nrow(iris_no_setosa_eg), "rows."))
```

```
[1] "There are 100 rows."
```

```r
head(iris_no_setosa_eg)
```

```
   Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
51          7.0         3.2          4.7         1.4 versicolor
52          6.4         3.2          4.5         1.5 versicolor
53          6.9         3.1          4.9         1.5 versicolor
54          5.5         2.3          4.0         1.3 versicolor
55          6.5         2.8          4.6         1.5 versicolor
56          5.7         2.8          4.5         1.3 versicolor
```

2.1.2 Random Sampling

It is quite common to randomly sample a subset of items from a vector or rows from a data frame in statistics.

2.1.2.1 Sample Items Randomly from a Vector

```r
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62, 7)
```

Randomly pick 5 samples without replacement.

```r
sample(pumpkins, size=5, replace=F)
```

```
[1] 7.62 6.85 6.56 6.62 7.55
```

The `replace=` option can be twisted to simulate tossing a coin N times. E.g.,

```
sample(c('H','T'), size=10, replace=T)
```

```
 [1] "T" "H" "T" "H" "H" "H" "H" "T" "T" "H"
```

2.1.2.2 Sample Rows Randomly from a Data Frame

Let's say we want to pick 5 samples randomly from `iris`. It takes two steps to achieve that:

Step 1: Create a list (vector) of random row indexes by the `sample()` function by specifying the following parameters:

a. The range of indexes. In this example, the indexes must be in the range from 1 to the last row of `iris`, which can be obtained from `nrow(iris)`.

b. How many samples, e.g., `size = 5`.

c. Can a sample be drawn repeatedly? If not, `replace = F`.

```
rnd_idx <- sample(seq(nrow(iris)), size = 5, replace = F)
rnd_idx
```

```
[1]  61  85   7   5 142
```

`seq(nrow(iris))` above generates a vector containing 1, 2, 3, ..., 150, as `iris` contains 150 rows.

Step 2: Select rows from `iris` according to the random indexes in `rnd_idx`.

```
a_random_sample <- iris[rnd_idx,]

a_random_sample
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
61           5.0         2.0          3.5         1.0 versicolor
85           5.4         3.0          4.5         1.5 versicolor
7            4.6         3.4          1.4         0.3     setosa
5            5.0         3.6          1.4         0.2     setosa
142          6.9         3.1          5.1         2.3  virginica
```

The indexing method has been discussed in the previous chapter. Check out here: Section 1.2.5.1. A note about `iris[rnd_idx,]` . Do not omit the comma (`,`) after `rnd_idx` inside the square brackets. It means all columns are selected for the output.

2.1.3 Sort Items

2.1.3.1 Sort Items in a Vector

It can be done using the `sort()` function. By default, the sorting order is ascending (from small to large).

```
sort(pumpkins)
```

```
 [1]  3.52  3.76  4.45  6.23  6.56  6.62  6.85  7.00  7.55  7.62 10.14
```

Sort it in descending order (large to small).

```
sort(pumpkins, decreasing = TRUE)
```

```
 [1] 10.14  7.62  7.55  7.00  6.85  6.62  6.56  6.23  4.45  3.76  3.52
```

2.1.3.2 Sort Rows in a Data Frame

Unfortunately, the `sort()` function does not work for the two-dimensional data frame as conveniently as it does for vectors. We have to use a method similar to sampling rows. It takes two steps to sort rows in a certain order. Suppose you want to sort rows in `iris` by `Sepal.Length` in ascending order (small to large).

Step 1: Obtain a list (vector) of row indexes that reflects the sorting order of the data frame.

```
sorted_idx <- order(iris$Sepal.Length)
sorted_idx
```

```
  [1]  14   9  39  43  42   4   7  23  48   3  30  12  13  25  31  46   2  10
 [19]  35  38  58 107   5   8  26  27  36  41  44  50  61  94   1  18  20  22
 [37]  24  40  45  47  99  28  29  33  60  49   6  11  17  21  32  85  34  37
 [55]  54  81  82  90  91  65  67  70  89  95 122  16  19  56  80  96  97 100
 [73] 114  15  68  83  93 102 115 143  62  71 150  63  79  84  86 120 139  64
 [91]  72  74  92 128 135  69  98 127 149  57  73  88 101 104 124 134 137 147
[109]  52  75 112 116 129 133 138  55 105 111 117 148  59  76  66  78  87 109
[127] 125 141 145 146  77 113 144  53 121 140 142  51 103 110 126 130 108 131
[145] 106 118 119 123 136 132
```

Note that the length of `sorted_idx` is the same as the number of rows in `iris`.

Step 2: Rearrange the rows in the data frame according to the sorted row indexes.

```
# The 5 shortest Sepal Lengths
head(iris[sorted_idx,])
```

```
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
14          4.3         3.0          1.1         0.1  setosa
9           4.4         2.9          1.4         0.2  setosa
39          4.4         3.0          1.3         0.2  setosa
43          4.4         3.2          1.3         0.2  setosa
42          4.5         2.3          1.3         0.3  setosa
4           4.6         3.1          1.5         0.2  setosa
```

```
# The 5 longest Sepal Lengths
tail(iris[sorted_idx,])
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
106          7.6         3.0          6.6         2.1 virginica
118          7.7         3.8          6.7         2.2 virginica
119          7.7         2.6          6.9         2.3 virginica
123          7.7         2.8          6.7         2.0 virginica
136          7.7         3.0          6.1         2.3 virginica
132          7.9         3.8          6.4         2.0 virginica
```

Note that the steps above do NOT change the original order of the rows in the data frame. If you intend to change the row order permanently, use the assignment operator to update the existing data frame or create a new data frame to store the sorted rows. See below:

```
ascending_iris <- iris[sorted_idx,]

# The shortest Sepal Length
head(ascending_iris)
```

```
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
14          4.3         3.0          1.1         0.1  setosa
9           4.4         2.9          1.4         0.2  setosa
39          4.4         3.0          1.3         0.2  setosa
43          4.4         3.2          1.3         0.2  setosa
42          4.5         2.3          1.3         0.3  setosa
4           4.6         3.1          1.5         0.2  setosa
```

```r
# The longest Sepal Length
tail(ascending_iris)
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
106          7.6         3.0          6.6         2.1 virginica
118          7.7         3.8          6.7         2.2 virginica
119          7.7         2.6          6.9         2.3 virginica
123          7.7         2.8          6.7         2.0 virginica
136          7.7         3.0          6.1         2.3 virginica
132          7.9         3.8          6.4         2.0 virginica
```

To sort rows in descending order (large to small), add the parameter `decreasing = T` to the `order()` function.

```r
desc_idx <- order(iris$Sepal.Length, decreasing = T)
desc_idx
```

```
  [1] 132 118 119 123 136 106 131 108 110 126 130 103  51  53 121 140 142  77
 [19] 113 144  66  78  87 109 125 141 145 146  59  76  55 105 111 117 148  52
 [37]  75 112 116 129 133 138  57  73  88 101 104 124 134 137 147  69  98 127
 [55] 149  64  72  74  92 128 135  63  79  84  86 120 139  62  71 150  15  68
 [73]  83  93 102 115 143  16  19  56  80  96  97 100 114  65  67  70  89  95
 [91] 122  34  37  54  81  82  90  91   6  11  17  21  32  85  49  28  29  33
[109]  60   1  18  20  22  24  40  45  47  99   5   8  26  27  36  41  44  50
[127]  61  94   2  10  35  38  58 107  12  13  25  31  46   3  30   4   7  23
[145]  48  42   9  39  43  14
```

```r
decreasing_iris <- iris[desc_idx,]
head(decreasing_iris)
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
132          7.9         3.8          6.4         2.0 virginica
118          7.7         3.8          6.7         2.2 virginica
119          7.7         2.6          6.9         2.3 virginica
123          7.7         2.8          6.7         2.0 virginica
136          7.7         3.0          6.1         2.3 virginica
106          7.6         3.0          6.6         2.1 virginica
```

```
tail(decreasing_iris)
```

```
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
48          4.6         3.2          1.4         0.2  setosa
42          4.5         2.3          1.3         0.3  setosa
9           4.4         2.9          1.4         0.2  setosa
39          4.4         3.0          1.3         0.2  setosa
43          4.4         3.2          1.3         0.2  setosa
14          4.3         3.0          1.1         0.1  setosa
```

2.1.4 Rearranging Columns

Suppose you want to move the `Species` column to the leftmost position. It can be done in two ways.

```
# Reorder columns by name
new_iris <- iris[,c("Species","Sepal.Length","Sepal.Width","Petal.Length","Petal.Width")]
head(new_iris)
```

```
  Species Sepal.Length Sepal.Width Petal.Length Petal.Width
1  setosa          5.1         3.5          1.4         0.2
2  setosa          4.9         3.0          1.4         0.2
3  setosa          4.7         3.2          1.3         0.2
4  setosa          4.6         3.1          1.5         0.2
5  setosa          5.0         3.6          1.4         0.2
6  setosa          5.4         3.9          1.7         0.4
```

Do not overlook the comma (`,`). It is used to retrieve all the rows in the data frame.

An alternative approach is to make use of column indexes as below:

```
# Reorder columns by index
new_iris <- iris[,c(5,1,2,3,4)]
head(new_iris)
```

```
  Species Sepal.Length Sepal.Width Petal.Length Petal.Width
1  setosa          5.1         3.5          1.4         0.2
2  setosa          4.9         3.0          1.4         0.2
3  setosa          4.7         3.2          1.3         0.2
4  setosa          4.6         3.1          1.5         0.2
5  setosa          5.0         3.6          1.4         0.2
6  setosa          5.4         3.9          1.7         0.4
```

Note that if a data frame contains many columns, say >100, this method is daunting. A much better way can be done using `tidyverse`, so stay tuned.

2.1.5 Focus on a Subset of Columns

Suppose a data frame contains many columns, way more than you need for analysis. It is neat to focus on those columns and leave out the rest. Let's suppose you want to focus only on sepal information and species.

```
sepal_iris <- iris[,c("Sepal.Length","Sepal.Width","Species")]
dim(sepal_iris)
```

```
[1] 150   3
```

```
head(sepal_iris)
```

```
  Sepal.Length Sepal.Width Species
1          5.1         3.5  setosa
2          4.9         3.0  setosa
3          4.7         3.2  setosa
4          4.6         3.1  setosa
5          5.0         3.6  setosa
6          5.4         3.9  setosa
```

2.1.6 Add a New Column

Suppose we want to add a new column called `Sepal.Ratio`, which is the ratio `Sepal.Length` to `Sepal.Width`. Here is the code:

```
iris$Sepal.Ratio <- iris$Sepal.Length/iris$Sepal.Width
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species Sepal.Ratio
1          5.1         3.5          1.4         0.2  setosa    1.457143
2          4.9         3.0          1.4         0.2  setosa    1.633333
3          4.7         3.2          1.3         0.2  setosa    1.468750
4          4.6         3.1          1.5         0.2  setosa    1.483871
5          5.0         3.6          1.4         0.2  setosa    1.388889
6          5.4         3.9          1.7         0.4  setosa    1.384615
```

New columns are always added to the rightmost position, as you can see `Sepal.Ratio`.

Suppose you want to add a new column that records the name of the researcher who collected the sample. If one researcher does it for all samples, you don't have to repeat the same name in a vector. Here is the code:

```
iris$Researcher <- "Brian"
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species Sepal.Ratio
1          5.1         3.5          1.4         0.2  setosa    1.457143
2          4.9         3.0          1.4         0.2  setosa    1.633333
3          4.7         3.2          1.3         0.2  setosa    1.468750
4          4.6         3.1          1.5         0.2  setosa    1.483871
5          5.0         3.6          1.4         0.2  setosa    1.388889
6          5.4         3.9          1.7         0.4  setosa    1.384615
  Researcher
1      Brian
2      Brian
3      Brian
4      Brian
5      Brian
6      Brian
```

2.1.7 Remove a Column

You can remove a column from a data frame, e.g., remove the `Sepal.Ratio` and `Researcher` added above.

```
iris$Sepal.Ratio <- NULL
iris$Researcher <- NULL

# the columns are gone
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

2.1.8 Tabulation

Tally the number of samples per one or more categorical variables (factors) by the `table()` function.

```
table(iris$Species)
```

```
    setosa versicolor  virginica
        50         50         50
```

```
# Alternatively, use with()
with(iris, table(Species))
```

```
Species
    setosa versicolor  virginica
        50         50         50
```

The data frame can be tallied by more than one categorical variable. Let's use another built-in dataset `CO2` to illustrate the point.

```
with(CO2, table(Plant, Type))
```

```
     Type
Plant Quebec Mississippi
  Qn1      7           0
  Qn2      7           0
  Qn3      7           0
  Qc1      7           0
  Qc3      7           0
  Qc2      7           0
  Mn3      0           7
  Mn2      0           7
  Mn1      0           7
  Mc2      0           7
  Mc3      0           7
  Mc1      0           7
```

That's all for data wrangling using base R.

2.2 Exercises for Base R

Q1. Use the built-in data set `islands` to answer this question. `islands` is a named vector containing the landmass areas of the world's major continents and islands, where the unit of landmass is in thousands of square miles.

a) Show places with landmass less than 50.

b) Show places with landmass between 100 and 200, inclusive.

c) Show places with landmass either greater than 1000 or smaller than 100, inclusive.

d) Show places with landmass 30.

e) Randomly sample 10 places with no replacement.

f) Show the five largest places.

g) Show the five smallest places.

Q2. Use the built-in data set `mtcar` to answer this question.

a. Show cars with more than 4 gears `gear`.

b. Show cars with `mpg` between 15 and 25, inclusive.

c. Show cars with horsepower `hp` greater than 100 and the number of cylinders `cyl` less than 6.

d. Show cars that do not have 6 cylinders.

e. `qsec` stands for quarter-mile time in seconds. It measures acceleration; the shorter the time, the faster the acceleration. Show three cars that have the highest acceleration.

f. Show three cars that have the lowest acceleration.

g. Show two cars whose accelerations are closest to the median.

Q3. Rearranging columns.

a. Create a new data frame called `mtcars_small` that contains only three columns: `mpg`, `cyl`, and `wt`. FYI, `wt` is weight in 1,000 lbs.

b. Create a new column in `mtcars_small` called `mpg_by_wt`, which is defined as `mpg` divided by `wt`.

c. Show the top three cars with the highest `mpg` by `wt`.

d. Remove `mpg_by_wt` from `mtcars_small`.

e. Create a new variable in `mtcars_small` called `efficiency`. If the `mpg` of a car is greater than 20, set `efficiency` to `"gas guzzler"`; otherwise, set it to `"economy"`. Tabulate `efficiency`.

2.3 `tidyverse`

`tidyverse` is an alternative, and importantly, less cryptic way to perform data wrangling on data frames. One design goal of is to name functions using **verbs**, which improves code readability, setting it apart from using unpronounceable symbols, such as `$`, `[`, etc., as used by base R as shown in the previous section. **However, `tidyverse` works with data frames only.** You still need base R to process vectors.

Before we dive into `tidyverse`, it is noteworthy to clarify that `tidyverse` is an integrated ecosystem of R packages designed for data science. Two packages actually provide the data wrangling functions discussed below: `dplyr`, and `tidyr`. As this book is a little book, you can assume the umbrella package provides everything.

As `tidyverse` is an add-on package, you have to install it manually. But the installation is simple. In RStudio, click the *Tools* menu option, select the first option: *Install Packages*. In the *Packages* box, type "tidyverse", and click the "Install" button. You only need to install it once, until after a major R upgrade.

2.3.1 Activate `tidyverse`

After opening a new session, you need to activate `tidyverse` using the `require()` function.

```
require(tidyverse)
```

```
Loading required package: tidyverse
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

The `require()` function prints messages telling you what packages (9 of them) are loaded together.

Alternatively, you can use the `library()` function to activate a package. Putting aside technical nuances, these two functions are largely doing the same thing. Since activation is an action, using a verb (`require()`) makes more sense than using a noun (`library()`). Note that you only need to activate a package once per R session.

Let's use `tidyverse` to repeat the data wrangling tasks in base R above.

2.3.2 Select Rows by Condition(s)

The function is `filter()`. It chooses rows that satisfy the specified condition(s). The syntax is `filter(data_frame, conditions)`. E.g.,

```
# select all setosa iris
setosa_iris <- filter(iris, Species == 'setosa')

# How many?
print(paste("There are", nrow(setosa_iris), "rows."))
```

```
[1] "There are 50 rows."
```

```
# More than one condition. The symbol for the logical OR operator is a vertical bar |
iris_or_eg <- filter(iris, Sepal.Length > 5 | Sepal.Width < 3 )

# How many?
print(paste("There are", nrow(iris_or_eg), "rows."))
```

```
[1] "There are 124 rows."
```

```
head(iris_or_eg)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          5.4         3.9          1.7         0.4  setosa
3          4.4         2.9          1.4         0.2  setosa
4          5.4         3.7          1.5         0.2  setosa
5          5.8         4.0          1.2         0.2  setosa
6          5.7         4.4          1.5         0.4  setosa
```

Specifying more than one condition. Use a relational operator to join conditions together.

```
iris_and_eg <- filter(iris, Petal.Length > 6 & Petal.Width < 2 )

# How many?
print(paste("There are", nrow(iris_and_eg), "rows."))
```

```
[1] "There are 2 rows."
```

```
iris_and_eg
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
1          7.3         2.9          6.3         1.8 virginica
2          7.4         2.8          6.1         1.9 virginica
```

```
# No setosa iris
iris_not_eg <- filter(iris, Species != 'setosa')

# How many?
print(paste("There are", nrow(iris_not_eg), "rows."))
```

```
[1] "There are 100 rows."
```

```
head(iris_not_eg)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
1          7.0         3.2          4.7         1.4 versicolor
2          6.4         3.2          4.5         1.5 versicolor
3          6.9         3.1          4.9         1.5 versicolor
4          5.5         2.3          4.0         1.3 versicolor
5          6.5         2.8          4.6         1.5 versicolor
6          5.7         2.8          4.5         1.3 versicolor
```

2.3.3 Select Rows Randomly

With `tidyverse`, you can do it in one step by using the `sample_n()` function.

```r
a_random_sample <- sample_n(iris, size = 5)

a_random_sample
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
1          6.7         3.1          5.6         2.4  virginica
2          5.2         2.7          3.9         1.4 versicolor
3          7.2         3.2          6.0         1.8  virginica
4          5.0         3.4          1.6         0.4     setosa
5          6.5         3.0          5.2         2.0  virginica
```

2.3.4 Sort Rows in a Data Frame

Use `arrange()`. Note that the function will not change the original data frame. Use the assignment operator (`<-`) to make permanent changes to the original data frame or store the changes in a new data frame. For example, sort `iris` by `Sepal.Length`, and store the sorted result in a new data frame.

```r
sorted_iris <- arrange(iris, Sepal.Length)

# The shortest Sepal Length
head(sorted_iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          4.3         3.0          1.1         0.1  setosa
2          4.4         2.9          1.4         0.2  setosa
3          4.4         3.0          1.3         0.2  setosa
4          4.4         3.2          1.3         0.2  setosa
5          4.5         2.3          1.3         0.3  setosa
6          4.6         3.1          1.5         0.2  setosa
```

```r
# The longest Sepal Length
tail(sorted_iris)
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
145          7.6         3.0          6.6         2.1 virginica
146          7.7         3.8          6.7         2.2 virginica
147          7.7         2.6          6.9         2.3 virginica
148          7.7         2.8          6.7         2.0 virginica
149          7.7         3.0          6.1         2.3 virginica
150          7.9         3.8          6.4         2.0 virginica
```

To sort rows in descending order, mark the sorting column with a minus `-`. E.g.,

```
decreasing_iris <- arrange(iris, -Sepal.Length)
head(decreasing_iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
1          7.9         3.8          6.4         2.0 virginica
2          7.7         3.8          6.7         2.2 virginica
3          7.7         2.6          6.9         2.3 virginica
4          7.7         2.8          6.7         2.0 virginica
5          7.7         3.0          6.1         2.3 virginica
6          7.6         3.0          6.6         2.1 virginica
```

```
tail(decreasing_iris)
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width Species
145          4.6         3.2          1.4         0.2  setosa
146          4.5         2.3          1.3         0.3  setosa
147          4.4         2.9          1.4         0.2  setosa
148          4.4         3.0          1.3         0.2  setosa
149          4.4         3.2          1.3         0.2  setosa
150          4.3         3.0          1.1         0.1  setosa
```

Specifying more than one sorting column is much easier in `tidyverse` than base R. E.g. sort `iris` in descending order of `Sepal.Length` and ascending order of `Petal.Length`.

```
double_sorted_iris <- arrange(iris, -Sepal.Length, Petal.Length)
head(double_sorted_iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
1          7.9         3.8          6.4         2.0 virginica
2          7.7         3.0          6.1         2.3 virginica
3          7.7         3.8          6.7         2.2 virginica
4          7.7         2.8          6.7         2.0 virginica
5          7.7         2.6          6.9         2.3 virginica
6          7.6         3.0          6.6         2.1 virginica
```

```
tail(double_sorted_iris)
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width Species
145          4.6         3.1          1.5         0.2  setosa
146          4.5         2.3          1.3         0.3  setosa
147          4.4         3.0          1.3         0.2  setosa
148          4.4         3.2          1.3         0.2  setosa
149          4.4         2.9          1.4         0.2  setosa
150          4.3         3.0          1.1         0.1  setosa
```

For rows with equal `Sepal.Length` (rows 2-5, 147-149), you can see their `Petal.Length` are in ascending order.

2.3.5 Reorganize columns of a data frame

Suppose you want to move the `Species` column to the leftmost position. You're going to love `tidyverse`, as it can be done easily by using two functions: `select()` and `everything()`.

```
# Reorder columns by name
new_iris <- select(iris, Species, everything())
head(new_iris)
```

```
  Species Sepal.Length Sepal.Width Petal.Length Petal.Width
1  setosa          5.1         3.5          1.4         0.2
2  setosa          4.9         3.0          1.4         0.2
3  setosa          4.7         3.2          1.3         0.2
4  setosa          4.6         3.1          1.5         0.2
5  setosa          5.0         3.6          1.4         0.2
6  setosa          5.4         3.9          1.7         0.4
```

2.3.6 Focus on a Subset of Columns

If a data frame contains many columns, way more than you need for analysis. It is neat to focus on those columns and remove the rest. E.g., use the `select()` function to keep only sepal information and species.

```
sepal_iris <- select(iris, Sepal.Length, Sepal.Width, Species)
dim(sepal_iris)
```

```
[1] 150   3
```

```
head(sepal_iris)
```

```
  Sepal.Length Sepal.Width Species
1          5.1         3.5  setosa
2          4.9         3.0  setosa
3          4.7         3.2  setosa
4          4.6         3.1  setosa
5          5.0         3.6  setosa
6          5.4         3.9  setosa
```

2.3.7 Add a New Column

Use the `mutate()` function to add columns to a data frame.

```
iris <- mutate(iris, Sepal.Ratio = Sepal.Length/Sepal.Width)
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species Sepal.Ratio
1          5.1         3.5          1.4         0.2  setosa    1.457143
2          4.9         3.0          1.4         0.2  setosa    1.633333
3          4.7         3.2          1.3         0.2  setosa    1.468750
4          4.6         3.1          1.5         0.2  setosa    1.483871
5          5.0         3.6          1.4         0.2  setosa    1.388889
6          5.4         3.9          1.7         0.4  setosa    1.384615
```

2.3.8 Remove a Column

Use the `select()` function and put a minus `'-'` symbol before the column name(s) to be removed.

```
iris <- select(iris, -Sepal.Ratio)
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

2.3.9 Rename a Column

So far, you have seen all the data wrangling tasks performed by base R using `tidyverse`. Starting from here, you are going to demonstrate tasks that are harder to do with base R. The first task is to rename a column. Suppose you want to rename `Species` to `Iris_Species`. You can do it with the `rename()` function:

```
# new_name = existing_name
iris <- rename(iris, Iris_Species = Species)
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Iris_Species
1          5.1         3.5          1.4         0.2       setosa
2          4.9         3.0          1.4         0.2       setosa
3          4.7         3.2          1.3         0.2       setosa
4          4.6         3.1          1.5         0.2       setosa
5          5.0         3.6          1.4         0.2       setosa
6          5.4         3.9          1.7         0.4       setosa
```

Let's revert the renaming so it will not affect the following examples.

```
iris <- rename(iris, Species = Iris_Species)
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

2.3.10 Pipe

Often, data wrangling is a multi-step process with many intermediate steps before producing the final result. E.g., select iris where its `Petal.Length` is between 0.5 and 2, and sort the results in descending order of `Petal.Length`. As you can see, it involves filtering, followed by sorting. It can be done in two steps via an intermediate data frame as below:

```
tmp_iris <- filter(iris, Petal.Length >= 0.5 & Petal.Length <=2)
final_iris <- arrange(tmp_iris, -Petal.Length)
dim(final_iris)
```

```
[1] 50  5
```

```
head(final_iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          4.8         3.4          1.9         0.2  setosa
2          5.1         3.8          1.9         0.4  setosa
3          5.4         3.9          1.7         0.4  setosa
4          5.7         3.8          1.7         0.3  setosa
5          5.4         3.4          1.7         0.2  setosa
6          5.1         3.3          1.7         0.5  setosa
```

The final result contains 50 rows.

`tidyverse` provides a neater way with no intermediate data frames produced. What is new is the **pipe** concept. You can imagine a pipe is like a factory assembly line, where an upstream workstation passes (pipes) intermediate results to the next workstation for further processing.

The pipe symbol is represented by **%>%**, with no space between them. Let's see how to apply the pipe to the above example without creating the intermediate data frame `tmp_iris`.

```
final_results <- iris %>%
                   filter(Petal.Length >= 0.5 & Petal.Length <=2) %>%
                   arrange(-Petal.Length)
dim(final_iris)
```

```
[1] 50  5
```

```
head(final_iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          4.8         3.4          1.9         0.2  setosa
2          5.1         3.8          1.9         0.4  setosa
3          5.4         3.9          1.7         0.4  setosa
4          5.7         3.8          1.7         0.3  setosa
5          5.4         3.4          1.7         0.2  setosa
6          5.1         3.3          1.7         0.5  setosa
```

Let's walk through the code above line by line.

- `iris %>%` sends all rows, 150 in total, to the downstream `filter()` function.
- `filter(Petal.Length >= 0.5 & Petal.Length <=2) %>%` receives the data and checks each row to see if it satisfies the specified conditions. If so, it sends it to the downstream process; otherwise, it does not pass it down.
- `arrange(-Petal.Length)` received rows that satisfy the previous filtering process, and sort them in descending order of `Petal.Length`.
- `final_results <-` the sorted result is assigned to the variable `final_results`.

By using pipe (`%>%`), you can avoid intermediate variables, making the code cleaner and easier to read.

2.3.11 Tabulation and Summarize

Another useful but difficult task in base R is producing summary information by group. E.g., you want to produce a table that shows the average sepal length for each species. Here's how to do it using two new functions `group_by()` and `summarize()`, and pipe them together.

E.g., tally the number of samples by `Plant` and 'Type' in `CO2`. The function that does the tallying is `n()`.

```
CO2 %>%
  group_by(Plant, Type) %>%
  summarize(n=n())
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by Plant and Type.
i Output is grouped by Plant.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(Plant, Type))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.

# A tibble: 12 x 3
# Groups:   Plant [12]
   Plant Type            n
   <ord> <fct>       <int>
 1 Qn1   Quebec          7
 2 Qn2   Quebec          7
 3 Qn3   Quebec          7
 4 Qc1   Quebec          7
```

```
 5 Qc3   Quebec          7
 6 Qc2   Quebec          7
 7 Mn3   Mississippi     7
 8 Mn2   Mississippi     7
 9 Mn1   Mississippi     7
10 Mc2   Mississippi     7
11 Mc3   Mississippi     7
12 Mc1   Mississippi     7
```

You do not see much difference between base R and `tidyverse` in the example above. But if you want to do more than tallying, `summarize()` is a treat, as `summarize()` supports the descriptive statistics functions mentioned before, such as mean(), sd(), median(), quantile(), etc. E.g,

```
iris %>%
  group_by(Species) %>%
  summarize(u=mean(Sepal.Length))
```

```
# A tibble: 3 x 2
  Species        u
  <fct>      <dbl>
1 setosa      5.01
2 versicolor  5.94
3 virginica   6.59
```

You can apply multiple functions in `summarize()`. E.g., shows the number of samples and standard deviation.

```
iris %>%
  group_by(Species) %>%
  summarize(n=n(),
            u=mean(Sepal.Length),
            s=sd(Sepal.Length))
```

```
# A tibble: 3 x 4
  Species        n     u     s
  <fct>      <int> <dbl> <dbl>
1 setosa        50  5.01 0.352
2 versicolor    50  5.94 0.516
3 virginica     50  6.59 0.636
```

You can group by more than one column. Let's switch to the `CO2` dataset to illustrate this point.

```
CO2 %>%
  group_by(Plant, Type, Treatment) %>%
  summarize(n=n(),
            u=mean(conc),
            s=sd(conc))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by Plant, Type, and Treatment.
i Output is grouped by Plant and Type.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(Plant, Type, Treatment))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 12 x 6
# Groups:   Plant, Type [12]
   Plant Type        Treatment      n     u     s
   <ord> <fct>       <fct>      <int> <dbl> <dbl>
 1 Qn1   Quebec      nonchilled     7   435  318.
 2 Qn2   Quebec      nonchilled     7   435  318.
 3 Qn3   Quebec      nonchilled     7   435  318.
 4 Qc1   Quebec      chilled        7   435  318.
 5 Qc3   Quebec      chilled        7   435  318.
 6 Qc2   Quebec      chilled        7   435  318.
 7 Mn3   Mississippi nonchilled     7   435  318.
 8 Mn2   Mississippi nonchilled     7   435  318.
 9 Mn1   Mississippi nonchilled     7   435  318.
10 Mc2   Mississippi chilled        7   435  318.
11 Mc3   Mississippi chilled        7   435  318.
12 Mc1   Mississippi chilled        7   435  318.
```

Two remarks:

1. `group_by()` is never used alone. It is often followed by `summarize()`.
2. The type of variables fed to `group_by()` must be factors, i.e., categorical.

2.3.12 A Summary of `tidyverse` functions

- `filter()`: select rows that satisfy the specified conditions.
- `select()`: choose or drop certain columns.
- `arrange()`: sort rows by the specified columns.
- `rename()`: change the name of a column.
- `mutate()`: add a column or modify the values of an existing column.
- `group_by()`, followed by `summarize()`
- pipe `%>%`: funnel intermediate result from an upstream process to the input of an immediate downstream process.

2.4 Exercises for `tidyverse`

Q4. Repeat the exercises in Q1 using `tidyverse`.

Q5. Repeat the exercises in Q2 using `tidyverse`.

Q6. Repeat the exercises in Q3 using `tidyverse`.

Q7. Use the built-in data set `esoph` to answer this question about pipe `%>%`, `group_by()`, and `summarise()`. It is an epidemiological data set from a case-control study of esophageal cancer in Ille-et-Vilaine, France. Here is a description of the fields:

- agegp: age group.
- alcgp: alcohol consumption group (grams/day).
- tobgp: tobacco consumption group (grams/day).
- ncases: Number of cancer cases.
- ncontrols: Number of healthy controls.

a. List the number of subjects for each age group.

b. List the number of subjects for each alcohol consumption group.

c. List the number of subjects for each age group and tobacco consumption group. Based on the results, how many subjects are 35-44 years old and consumed 10-19 grams per day?

d. List the number of cases and controls for each age group and alcohol consumption group. Based on the results, which combination of age group and alcohol consumption group has the smallest number of controls?

2.5 Summary

In this chapter, you have seen how to perform data wrangling with basic R and `tidyverse`. What you have seen is only a fraction of what `tidyverse` can do. Readers who want to explore the full capability of `tidyverse` should consult the most authoritative source: **R for Data Science** by Hadley Wickham & Garrett Grolemund. In the next chapter, you will see how to plot data. A picture is worth a thousand words. Data visualization is an indispensable step for data analysis. But before the next topic, let's recall all the new R commands discussed in this chapter.

Table 2.1: A Summary of R Operators and Functions

|	`sample()`	`filter()`	`summarize()`
`&`	`order()`	`select()`	`n()`
`==`	`NULL` (a keyword)	`arrange()`	pipe `%>%`
`!`	`table()`	`rename()`	
`!=`	`library()`	`mutate()`	
`subset()`	`require()`	`group_by()`	

3 Data Visualization

Now that you have cleaned, organized, and analyzed the data, the next step is to share the findings and insights with the audience, raising their attention to the study. Rows and columns of numbers may not be palatable to most people. To help the audience unpack the message embedded in the data, a graph is recommended. In this chapter, we will discuss various types of graphs, what they are good at and not so good at, and how to make them in R using two different graphical systems.

Learning Objectives:

After finishing this chapter, you should be able to:

- Choose the most appropriate graph based on the number of variables and their types.
- Convey the message you want to convey to the audience embedded in the data through graphs.
- Decorate graphs with labels, a main title, color, and other graphical attributes using the base R graphical system.
- Create publication-quality graphs using the `ggplot2` package.

3.1 Types of Graphs

The best way to choose the most appropriate graph depends on the data. Specifically, the types of variables (numeric or categorical) and how many of them there are. Here, you will plot graphs that entail:

1. One numeric variable.
2. One categorical variable.
3. One categorical variable and one numeric variable.
4. Two or more numeric variables.
5. A summary table.

3.2 Base R Graphics

This graphical system is built into base R; no additional package is required. It is simple, ideal for creating graphs promptly for prototyping purposes and getting the job done. The flip side is that the graphs may be bare-bones and unrefined. Some features are hard to implement. There is no one graphical system for all data visualization tasks. It is highly recommended to master both systems. Let's dive into each graph one by one.

3.2.1 One Numeric Variable

If you want to examine the distribution of a numeric variable (such as the spread, peak(s), symmetry, and flatness), a histogram is the choice. On the other hand, if your concern is robust statistics (median, outliers, etc.), a box plot will do the job. Let's start with plotting a histogram.

3.2.1.1 Histogram

Let's examine the distribution of `iris`'s `Sepal.Width`, a numeric variable, using a histogram. Below are the codes for creating four histograms that display the same information but with different refinements.

```
par(mfrow=c(2,2))

# Top left
hist(iris$Sepal.Width)

# Top right
hist(iris$Sepal.Width,
     xlab="Sepal Width",
     ylab = "Count",
     main = "Histogram of Sepal Width")

# Bottom left
hist(iris$Sepal.Width,
     col = 'skyblue',
     border = 'white',
     xlab="Sepal Width",
     ylab = "Count",
     main = "Color options")

# Bottom right
```

```r
hist(iris$Sepal.Width,
     breaks = seq(2,4.5,by=0.1),
     col = 'skyblue',
     border = 'white',
     xlab="Sepal Width",
     ylab = "Count",
     main = "breaks option")
```

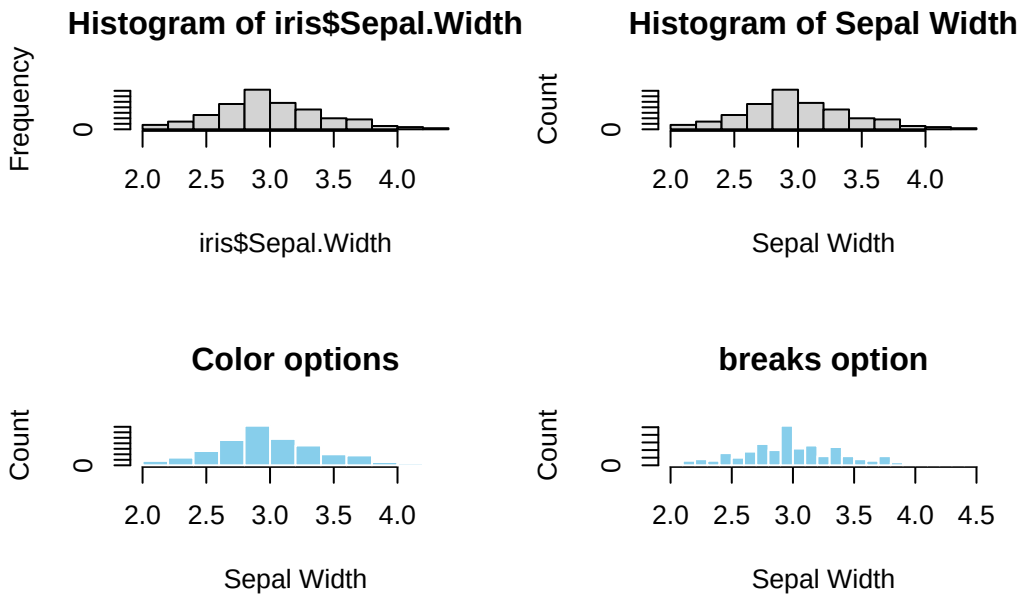


Figure 3.1: Base R Histogram. Top left: default. Top right: Customized X-Y labels and the main title. Bottom left: change color of bars and border. Bottom right: change the bin width (`breaks=`)

For the bottom-right graph, check out this line: `breaks = seq(2,4.5,by=0.1)`. The aim is to customize the bar widths. The `seq()` function (Section 1.4.3) generates a list of numbers (`2.0, 2.1 ... 4.4, 4.5`) that represents the x-ticks.

Note: `par(mfrow=c(2,2))` splits the plotting area into a 2x2 grid such that multiple graphs can be displayed together. If your plot contains only one graph, it can be ignored.

3.2.1.2 Box Plot

If robust statistics of the data are the key message, the most appropriate graph is a box plot. In a box plot, the y-axis represents the numeric variable, and the x-axis has no meaning.

```r
par(mfrow=c(2,2))

# Top left
boxplot(iris$Sepal.Width)

# Top right
boxplot(iris$Sepal.Width,
        ylab="Sepal Width",
        main="Box plot of Sepal Width")

# Bottom left
boxplot(iris$Sepal.Width,
            col = "skyblue",
            ylab="Sepal Width",
            outcol = 'red',
            outpch = 19,
            main="Color options")

# Bottom right
boxplot(iris$Sepal.Width,
        horizontal = TRUE,
        xlab="Sepal Width",
        main="Horizontal Box plot")
```

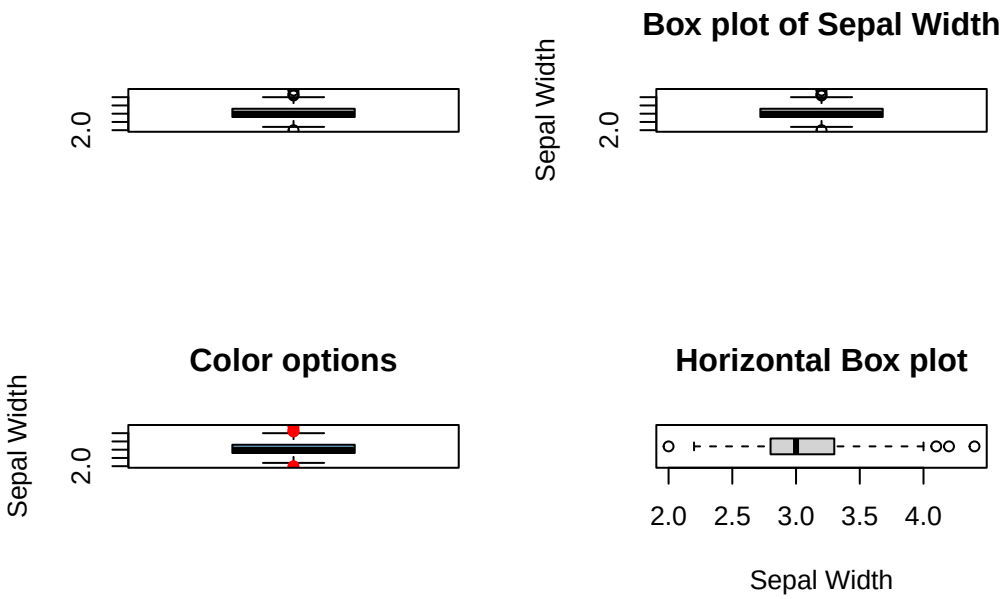


Figure 3.2: Base R Box plot. Top left: Default, Top right: labels and main title. Bottom left: color options. Bottom right: `horizontal = TRUE`

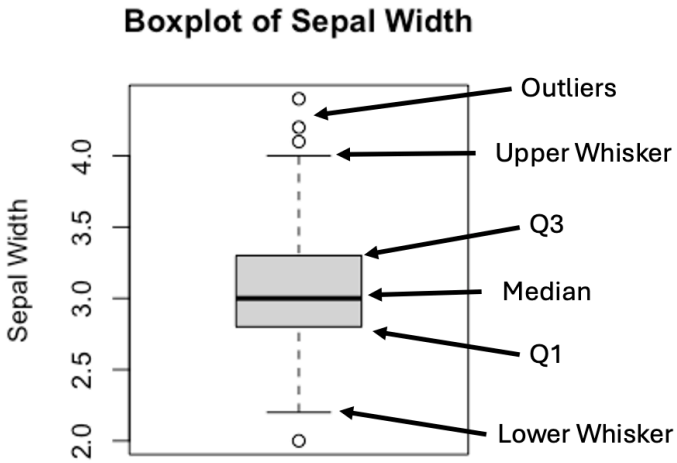


Figure 3.3: Anatomy of a Box plot

The upper and lower whiskers are defined as:

$$Upper\ Whisker = Q3 + 1.5 \times IQR$$
$$Lower\ Whisker = Q1 - 1.5 \times IQR$$

, where $IQR = (Q3 - Q1)$.

As a practice, data points above the upper whisker or below the lower whiskers are labeled as outliers.

While a box plot summarizes data into quartiles, a histogram portrays the underlying distribution—such as peaks and symmetry—that is otherwise difficult to spot. Is it a great idea to combine the best of both plots in a single graph? To ensure the combined plot makes sense, we need to make sure the axes of the two plots are in the same range. Here's an example below:

```
par(mfrow=c(2,1))

hist(iris$Sepal.Width,
     breaks = seq(2,4.5,by=0.1),
     col = 'skyblue',
     border = 'white',
     xlim = c(2,4.5),
     xlab = "",
     ylab = "Count",
     main = "Stacking Up Graphs")

boxplot(iris$Sepal.Width,
        horizontal = TRUE,
        ylim = c(2,4.5),
        xlab="Sepal Width",
        main="")
```

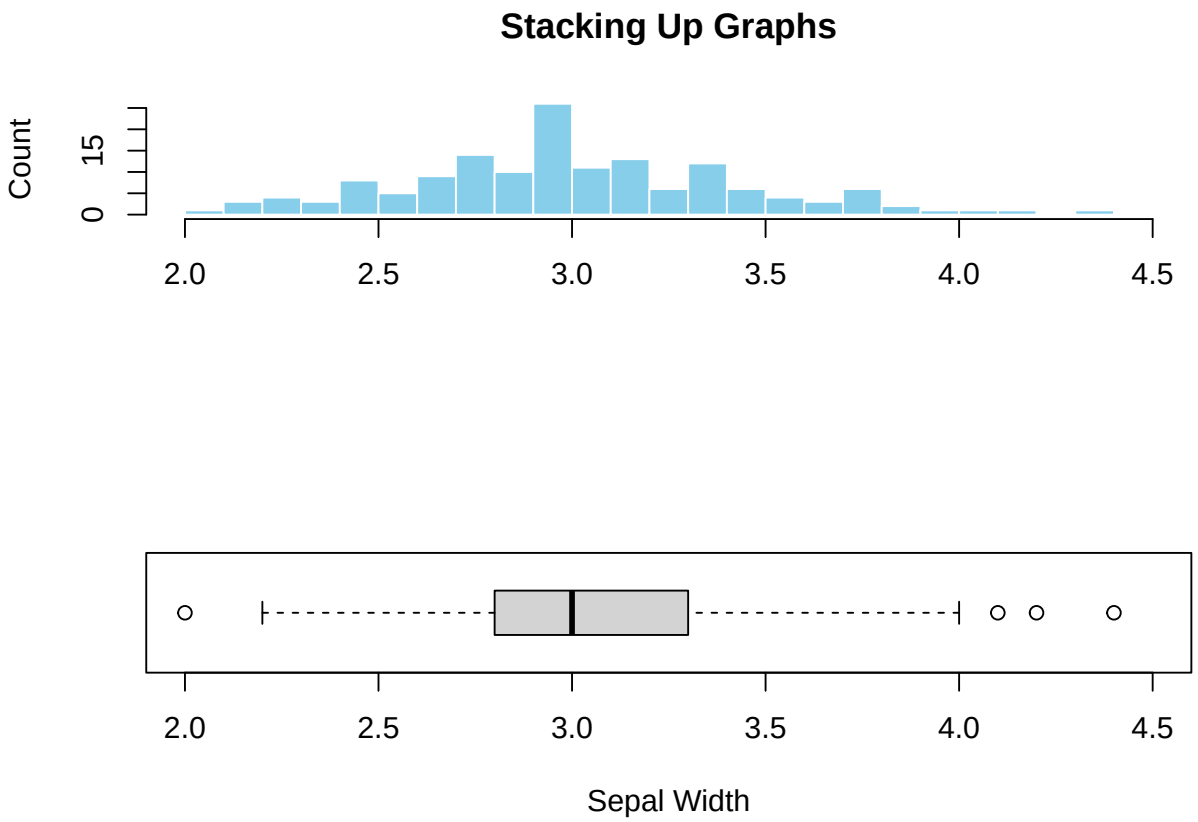


Figure 3.4: Stacking up a histogram and a box plot.

The `ylim` of the histogram and the `xlim=` of the box plot are the specific parameters used to align the two graphs on the same scale.

Figure 3.4 illustrates the added advantage of combining graphs. The peak in the histogram is almost perfectly aligned with the median shown in the box plot, suggesting the data distribution is nearly symmetric. Additionally, the outliers, hardly visible in the histogram, can easily be spotted in the box plot. This example demonstrates the 1+1>2 offered by combining different types of graphs, and the technique used to implement it.

3.2.2 One Categorical Variable

When it comes to categorical variables, we are usually interested in the number of samples that fall in each category. As discussed in Chapter 2, the tallying can be done using the `table()` function (Section 2.1.8) or `group_by()` followed by `summarize(n=n())` (Section 2.3.11).

3.2.2.1 Bar Plot

Bar plot is the standard choice for visualizing tallied data. The only input to R's `barplot()` graph function is a tabulated count of the categorical variable. Here's an example:

```
par(mfrow=c(2,2))

# Top left
barplot(with(iris, table(Species)))

# Top right
barplot(with(iris, table(Species)),
        xlab="Species",
        ylab="Count",
        main="A Barplot of Species")

# Bottom left
barplot(with(iris, table(Species)),
        col='skyblue',
        border='blue',
        xlab="Species",
        ylab="Count",
        main="Color Options")

# Bottom right
barplot(with(iris, table(Species)),
        horiz = TRUE,
        col='skyblue',
        border='blue',
        xlab="Count",
        ylab="Species",
        main="A Horizontal Barplot")
```

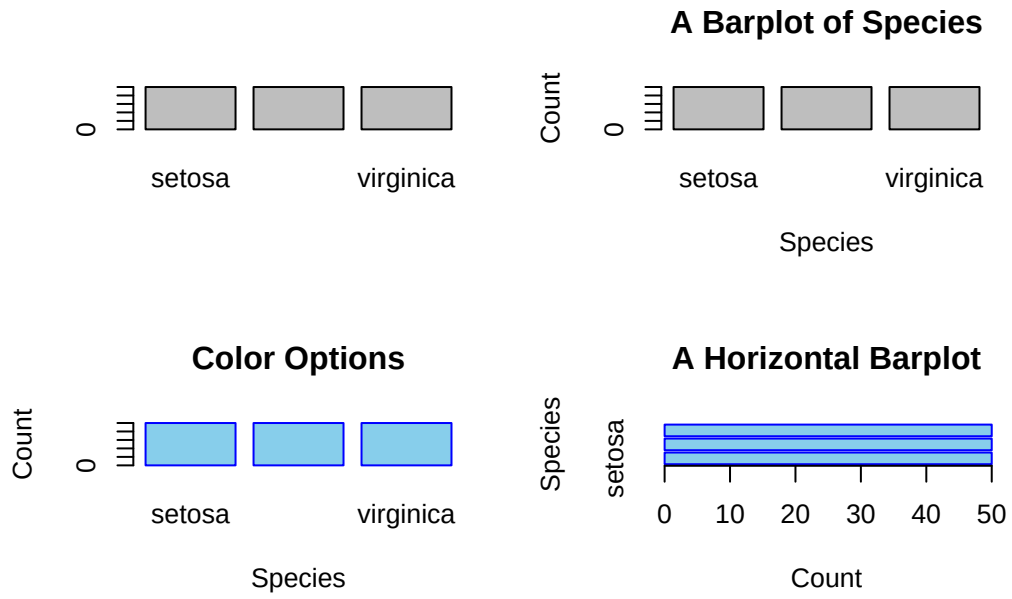


Figure 3.5: A simple bar plot. Top left: Default. Top right: with tables

Sometimes you might want to reorder the categories along the x- or y-axis. By default, `barplot()` places the bars from left to right according to the order of the levels. E.g., show the levels of `iris$Species` by:

```
levels(iris$Species)
```

```
[1] "setosa"     "versicolor" "virginica"
```

You can change the order of the levels by updating the factor variable using the `factor()` function (Section 1.2.4). Suppose the desired order is `virginica`, `setosa`, and `veriscolor`. Here's the code:

```
iris$Species <- factor(iris$Species,
                       levels = c("virginica", "setosa", "versicolor"),
                       ordered = TRUE)
levels(iris$Species)
```

```
[1] "virginica"  "setosa"     "versicolor"
```

Here's the revised bar plot.

```r
barplot(with(iris, table(Species)),
        col='skyblue',
        border='blue',
        xlab="Species",
        ylab="Count",
        main="Reordered Species")
```

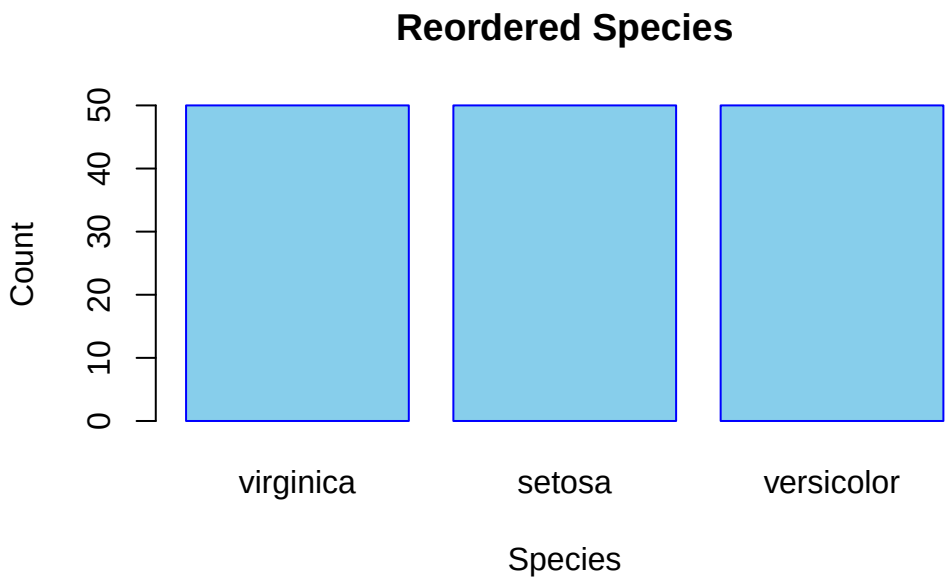


Figure 3.6: Reorder the Bars of a Barplot

3.2.2.2 Pie Chart

An alternative to bar plots is pie charts.

```r
par(mfrow=c(2,2))

# Top left
pie(with(iris, table(Species)))

# Top right
pie(with(iris, table(Species)),
    col = c('skyblue', 'orange', 'white'),
    main="Species")

# Bottom left
```

```r
counts <- with(iris, table(Species))
my_labs <- paste0(names(counts),"(",counts,")")
pie(counts,
    labels = my_labs,
    col = c('skyblue', 'orange', 'white'),
    main = "Species Counts")


# Bottom right
counts <- with(iris, table(Species))
fractions <- round(counts/sum(counts),2)
my_labs <- paste0(names(counts),"(",fractions,")")
pie(counts,
    labels = my_labs,
    col = c('skyblue', 'orange', 'white'),
    main = "Species Fractions")
```

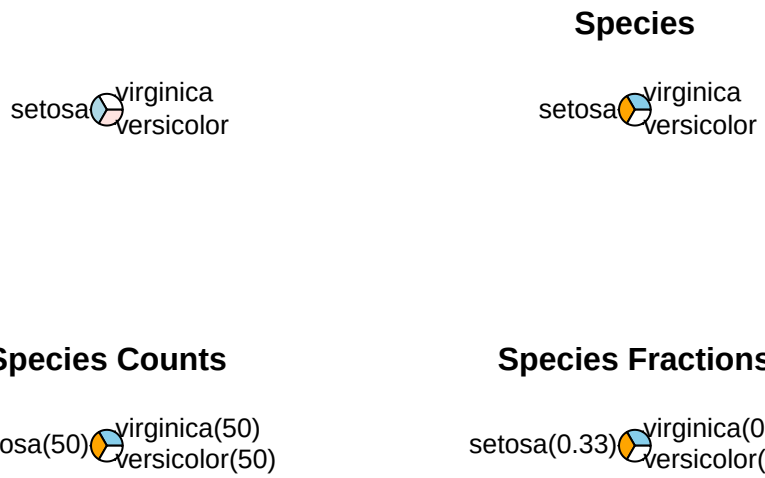


Figure 3.7: A simple pie chart. Top left: Default. Top right: with tables. Bottom left: Add a title. Show count for each category. Bottom right: Add a fraction or proportion for each category.

3.2.3 One Categorical Variable and One Numeric Variable

The categorical variable in this situation usually serves as a group identifier that splits the data into groups so the numerical variable can be compared between groups. This kind of graph is generally named a **side-by-side plot**. You can choose a side-by-side box plot or a

side-by-side bar plot. For example, a side-by-side box plot that visually compares the robust statistics of `Sepal.Length` between different species of iris.

3.2.3.1 Side-by-side box plot

```
par(mfrow=c(2,1))

boxplot(Sepal.Length ~ Species,
        data=iris,
        main="A Side-by-Side Box plot")

boxplot(Sepal.Length ~ Species,
        data=iris,
        horizontal = TRUE,
        main="A Horizontal Side-by-Side Box plot")
```

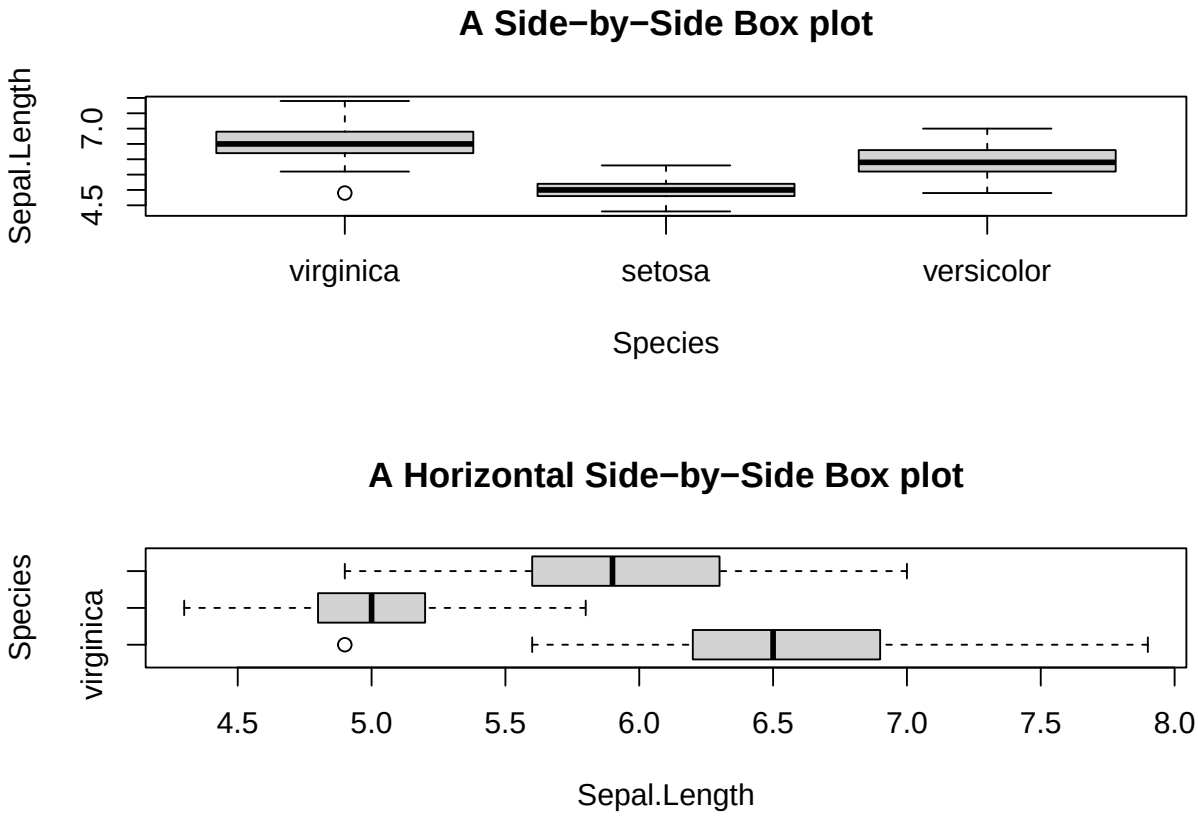


Figure 3.8: A side-by-side box plot

`Sepal.Length ~ Species` is read as "Sepal length **by** species", the tilde symbol "`~`" means "by".

However, there is a glitch in the horizontal box plot (bottom): the y-labels collide. It can be fixed by:

1. Expand the left margin of the plotting area.
2. Add the `las=2` parameter to the `boxplot()` function, and turn off the default `ylab=`.
3. Manually add the y-label.

```
# Expand the left margin so the text doesn't truncate
# The numbers correspond to margins: c(bottom, left, top, right)
# Default is c(5, 4, 4, 2). We are increasing the left margin from 4 to 7.
par(mar = c(5, 7, 4, 2) + 0.1)

# Draw the plot, but explicitly tell R NOT to draw the y-axis label yet
boxplot(Sepal.Length ~ Species,
        data=iris,
        horizontal = TRUE,
        las = 2,
        ylab = "", # This suppresses the default colliding label
        main="Y-tick Labels Reorientated")

# Add the y-axis label manually, pushing it further away from the axis
# The 'line' argument controls how many text lines away from the axis it sits
title(ylab = "Species", line = 5)
```

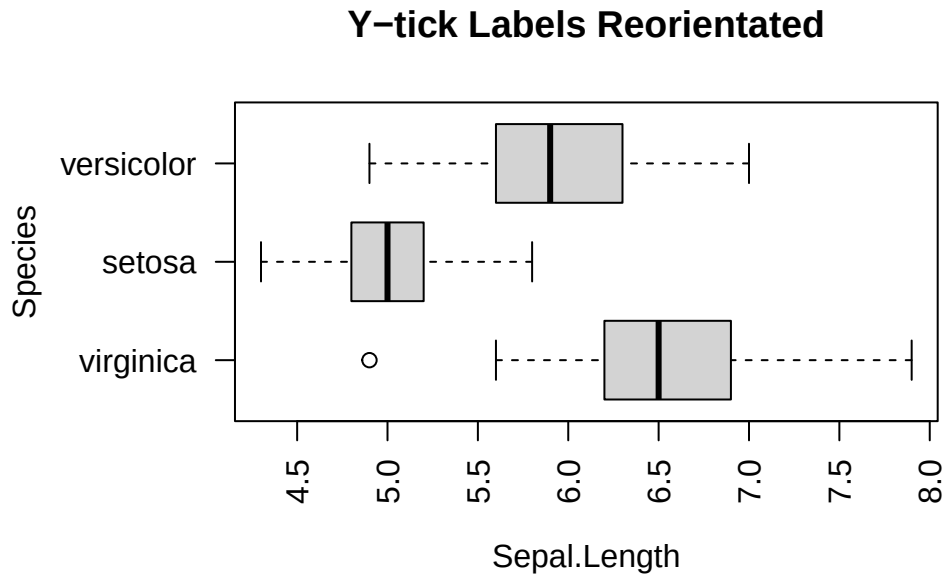


Figure 3.9: A Side-by-Side Box Plot with Horizontal Y-Labels

3.2.3.2 Side-by-side Bar Plot

Suppose you want to compare the average sepal length among different iris species visually. You will begin to feel the cumbersome nature of base R, and appreciate the power of `tidyverse`. Anyway, you will see how to make such a graph with a new function, `tapply()`, in two steps.

First, make a table of average sepal length by species. Apparently, the `table()` function may help. But it does not work for this case, as it only tallies the number of samples by species. Hence, you need to use a new function `tapply()`. Here's an example:

```
mean_data <- tapply(iris$Sepal.Length, iris$Species, FUN = mean)
mean_data
```

```
 virginica     setosa versicolor
     6.588      5.006      5.936
```

How does `tapply()` work? The "`t`" represents a table. The function applies the `mean` function to the input data `iris$Sepal.Length` grouped by `iris$Species`. In other words, the `mean()` is applied to the sepal lengths for each species separately.

And then, pass the `mean_data` object to `barplot()`.

```r
barplot(mean_data,
        xlab="Species",
        ylab="Average Sepal.Length",
        main="Side-by-side Barplot")
```

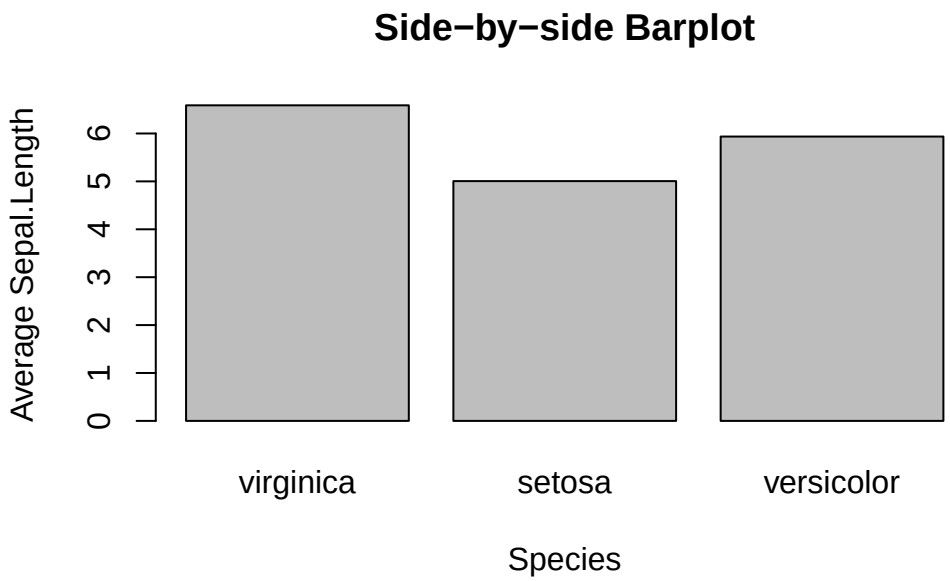


Figure 3.10: A Side-by-Side Bar Plot

3.2.3.3 Side-by-side Bar Plot with Error Bar

This topic is slightly advanced. It is much easier to use `ggplot2`*. So, feel free to skip this or jump to Section 3.4.4.3.*

The error bar represents the standard error of the mean (SEM). Here is the formula:

$$\bar{x} \pm \frac{s}{\sqrt{n}}$$

, where s and n are the standard deviation and sample size, respectively. In the bar plot you are going to plot, the height of the bar is $\bar{x}$, and the error bars span above and below the bar's top by $\frac{s}{\sqrt{n}}$ units.

First, calculate the mean and standard error using the `tapply()` function.

```r
mean_data <- tapply(iris$Sepal.Length, iris$Species, FUN = mean)
mean_data
```

```
 virginica     setosa versicolor
     6.588      5.006      5.936
```

```r
sem <- function(x) {sd(x)/sqrt(length(x))}
sem_data <- tapply(iris$Sepal.Length, iris$Species, FUN = sem)
sem_data
```

```
 virginica     setosa versicolor
0.08992695 0.04984957 0.07299762
```

First, create the bar plot and store the height of bars in the object `bar_midpoints`. Use the `arrows()` function to create/add the error bars to the bar plot, where each error bar consists of the upper whisker (x_1, y_1) and lower whisker (x_0, y_0).

```r
bar_midpoints <- barplot(mean_data,
                  xlab="Species",
                  ylab="Average Sepal.Length",
                  main="Side-by-side Barplot")

arrows(x0 = bar_midpoints,
       y0 = mean_data - sem_data, # Start of the bar
       x1 = bar_midpoints,
       y1 = mean_data + sem_data, # End of the bar
       angle = 90,                # Makes the ends of the bars flat
       code = 3,                  # Draws ticks at both ends
       length = 0.5, col='red')   # Sets the length of the ticks
```

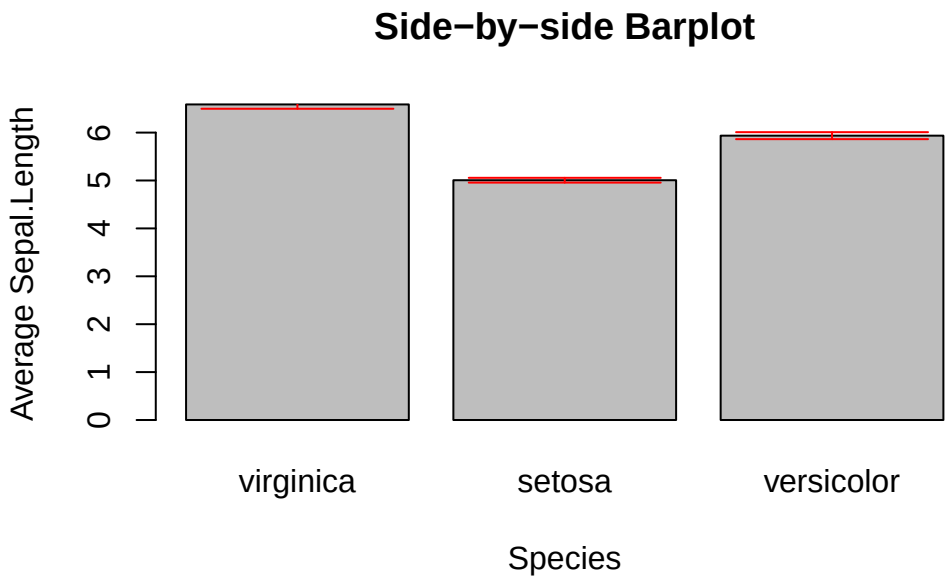


Figure 3.11: A Side-by-side Bar Plot with Error Bars

Once again, if you found this confusing, see the `ggplot` version of error bars (Section 3.4.4.3).

3.2.3.4 Line Graph with Error Bar

If your message is to show a trend from left to right, a line graph is the default choice. Here is an example to create a line graph with solid dots.

```
# Default line plot
plot(mean_data,
      type='b',
      pch=19,
      xlab="Species",
      ylab="Average Sepal.Length",
      main="A Line Plot")
```

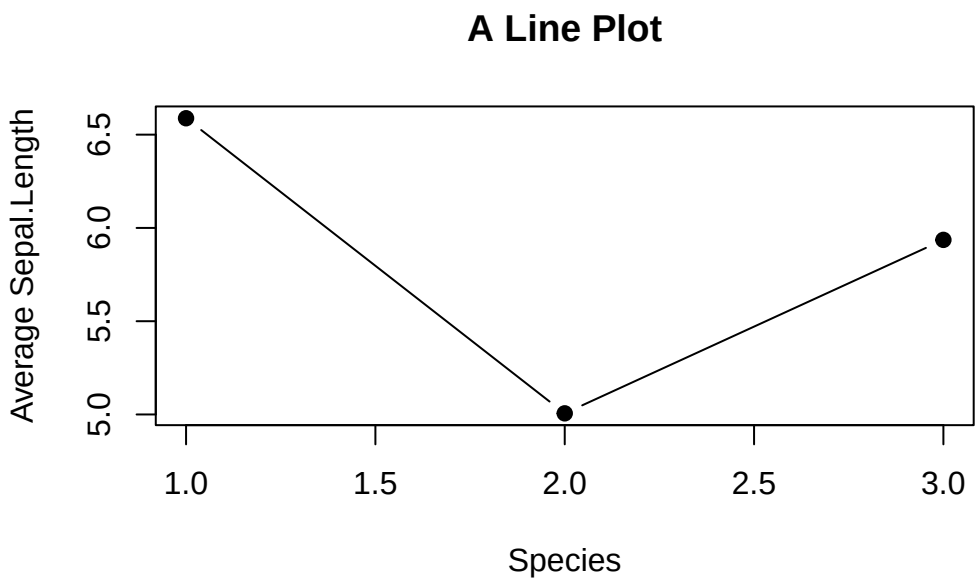


Figure 3.12: A Line Plot

- Use `pch=19` to specify the shape of the data points. `pch` stands for **p**lotting **ch**aracter. The value `19` denotes a solid dot (•).
- To join the dots together in a line plot, specify `type='b'`, the `b` means **both** dots and line are plotted. More options of `type` can be found using the help function: `help(plot)` documentation.

But there is a glitch in the x-tick. The x-tick should show the actual species names instead of numbers. It can be fixed in two steps: 1. Suppress the default x-tick labeling feature of the `plot()` function by setting the parameter `xaxt="n"`. And then, paste the customized x-tick back with the `axis()` function. The species names can be obtained by `names(mean_data)`.

```
# Fix x-tick labels
plot(mean_data,
      type='b',
      pch=19,
      xlab="Species",
      ylab="Average Sepal.Length",
      main="Change x-tick Labels",
      xaxt="n")
axis(side=1, at=1:length(mean_data), labels=names(mean_data))
```

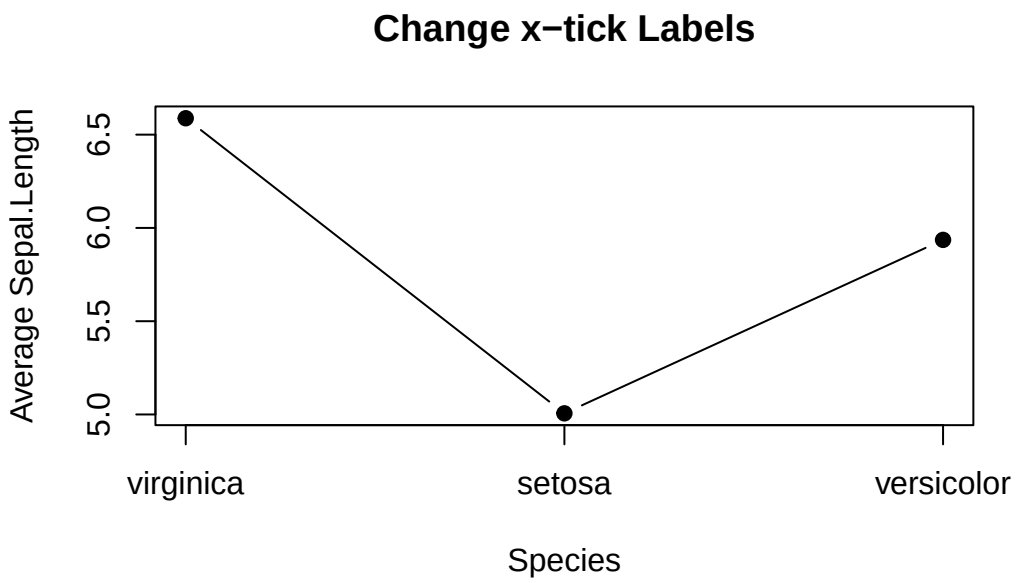


Figure 3.13: A Line plot with proper x-tick labels

Now we can add error bars to the line plot. It is similar to the bar plot above, except that `x0` and `x1` should take the values `1,2,3`.

```
# Add Error Bar
plot(mean_data,
      type='b',
      pch=19,
      xlab="Species",
      ylab="Average Sepal.Length",
      main="Change x-tick Labels",
      xaxt="n")
axis(side=1, at=1:length(mean_data), labels=names(mean_data))

arrows(x0 = seq(length(mean_data)),
       y0 = mean_data - sem_data, # Start of the bar
       x1 = seq(length(mean_data)),
       y1 = mean_data + sem_data, # End of the bar
       angle = 90,                # Makes the ends of the bars flat
       code = 3,                  # Draws ticks at both ends
       length = 0.05, col='red')   # Sets the length of the ticks
```

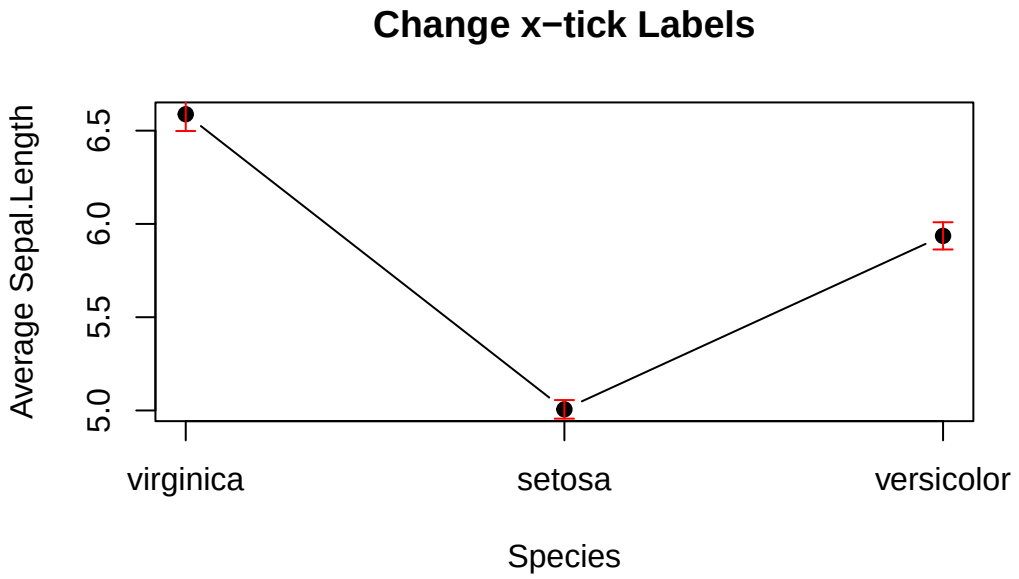


Figure 3.14: A Line plot with Error Bars

3.2.3.5 Overlapping Histogram

The code for creating this plot can be convoluted. It is easier to make it using `ggplot2`*. Feel free to skip to Section 3.4.4.5.*

Suppose you are interested in visualizing and comparing the distribution of sepal lengths among the three iris species. Here are the steps to do it:

1. Split the data by individual species.
2. Calculate the range for the x- and y-axis to ensure the histograms are on the same scale.
3. Create a histogram in certain color for a species from step 1, and use it as a template.
4. Overlay the histograms from other species on top of each other.

Let's do it step by step below:

```
# Step 1: Split the data by individual species

setosa <- subset(iris, Species == "setosa")
versicolor <- subset(iris, Species == "versicolor")
virginica <- subset(iris, Species == "virginica")
```

```r
# Step 2: Set up the range for the x- and y-axis

x_range <- range(iris$Sepal.Length)
y_range <- c(0, 25) # 25 is arbitrary.
```

Below is the code for plotting the template histogram and adding histograms to it. To distinguish species, different colors are attributed to different species. It is done by `col = rgb(...)`. The `rgb()` function stands for **r**ed, **g**reen, and **b**lue. It takes four parameters: the intensity of red, green, blue, and transparency. All are in the range of 0 and 1. E.g., `rgb(1, 0, 0, 0.5)` means 100% red, no green and blue at all, and transparency is 50%.

```r
# Step 3: Create a template

hist(setosa$Sepal.Length,
     main = "Overlapping Histogram of Sepal Length",
     xlab = "Sepal Length",
     col = rgb(1, 0, 0, 0.5), # Red with 50% transparency
     breaks = seq(4,9,by=0.25),
     xlim = x_range,
     ylim = y_range)

# Step 4: Overlay histograms from other species

hist(versicolor$Sepal.Length, col=rgb(0, 1, 0, 0.5), add = TRUE) # green with 50% transparen
hist(virginica$Sepal.Length, col=rgb(0, 0, 1, 0.5), add = TRUE)  # blue with 50% transparency
```

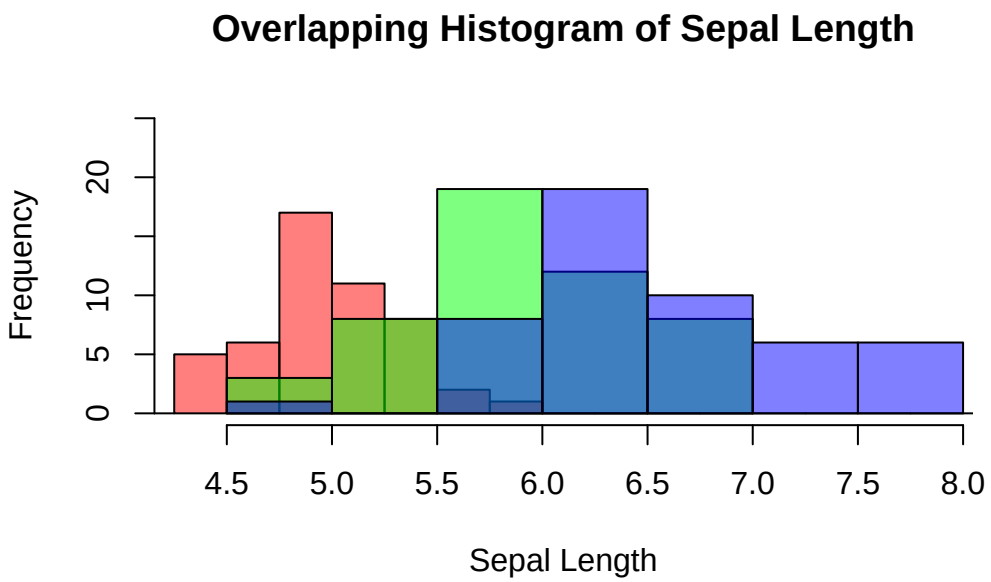


Figure 3.15: Overlapping histograms

3.2.4 Contingency Table

Here we deal with a data set that comprises two categorical variables. A two-dimensional contingency table is used to tally the number of samples for different combinations of categories of the two categorical variables.

The distribution of the counts can reveal whether or not the two categorical variables are dependent. To illustrate how to visualize a contingency table, we will use the built-in data set `penguins`. The data set contains two categorical variables: `species` and `island`.

```
summary(penguins)
```

```
      species          island       bill_len        bill_dep
 Adelie   :152   Biscoe   :168   Min.   :32.10   Min.   :13.10
 Chinstrap: 68   Dream    :124   1st Qu.:39.23   1st Qu.:15.60
 Gentoo   :124   Torgersen: 52   Median :44.45   Median :17.30
                                 Mean   :43.92   Mean   :17.15
                                 3rd Qu.:48.50   3rd Qu.:18.70
                                 Max.   :59.60   Max.   :21.50
                                 NAs    :2       NAs    :2
  flipper_len      body_mass       sex            year
 Min.   :172.0   Min.   :2700   female:165   Min.   :2007
 1st Qu.:190.0   1st Qu.:3550   male  :168   1st Qu.:2007
```

```
 Median :197.0   Median :4050   NAs    : 11   Median :2008
 Mean   :200.9   Mean   :4202                 Mean   :2008
 3rd Qu.:213.0   3rd Qu.:4750                 3rd Qu.:2009
 Max.   :231.0   Max.   :6300                 Max.   :2009
 NAs    :2       NAs    :2
```

Suppose you are interested in exploring whether or not different species have shown preference in habitat or geographical location (`island`).

Let's begin by creating a 2D table as below:

```
mytab <- with(penguins, table(species, island))
mytab
```

```
           island
species     Biscoe Dream Torgersen
  Adelie        44    56        52
  Chinstrap      0    68         0
  Gentoo       124     0         0
```

Obviously, `species` and `island` are not independent, as you can see not all three species are present in all islands. If `species` and `island` are independent, the counts in the contingency table should be below:

```
indep_tab
```

```
           island
species     Biscoe Dream Torgersen
  Adelie        74    55        23
  Chinstrap     33    25        10
  Gentoo        61    45        19
```

Two popular graphs are suitable for visualizing a contingency table: a side-by-side bar plot, or a mosaic plot.

```
par(mfrow=c(3,1))

barplot(mytab,
        beside=T,
        xlab = "Island",
        ylab = "Count",
```

```
        main = "A 2D Table",
        col = c("skyblue", "lightgreen", "lightcoral"),
        legend.text = rownames(mytab))

mosaicplot(mytab, main = "Mosaic Plot of Two Dependent Variables")

mosaicplot(indep_tab, main = "Mosaic Plot of Two Independent Variables")
```

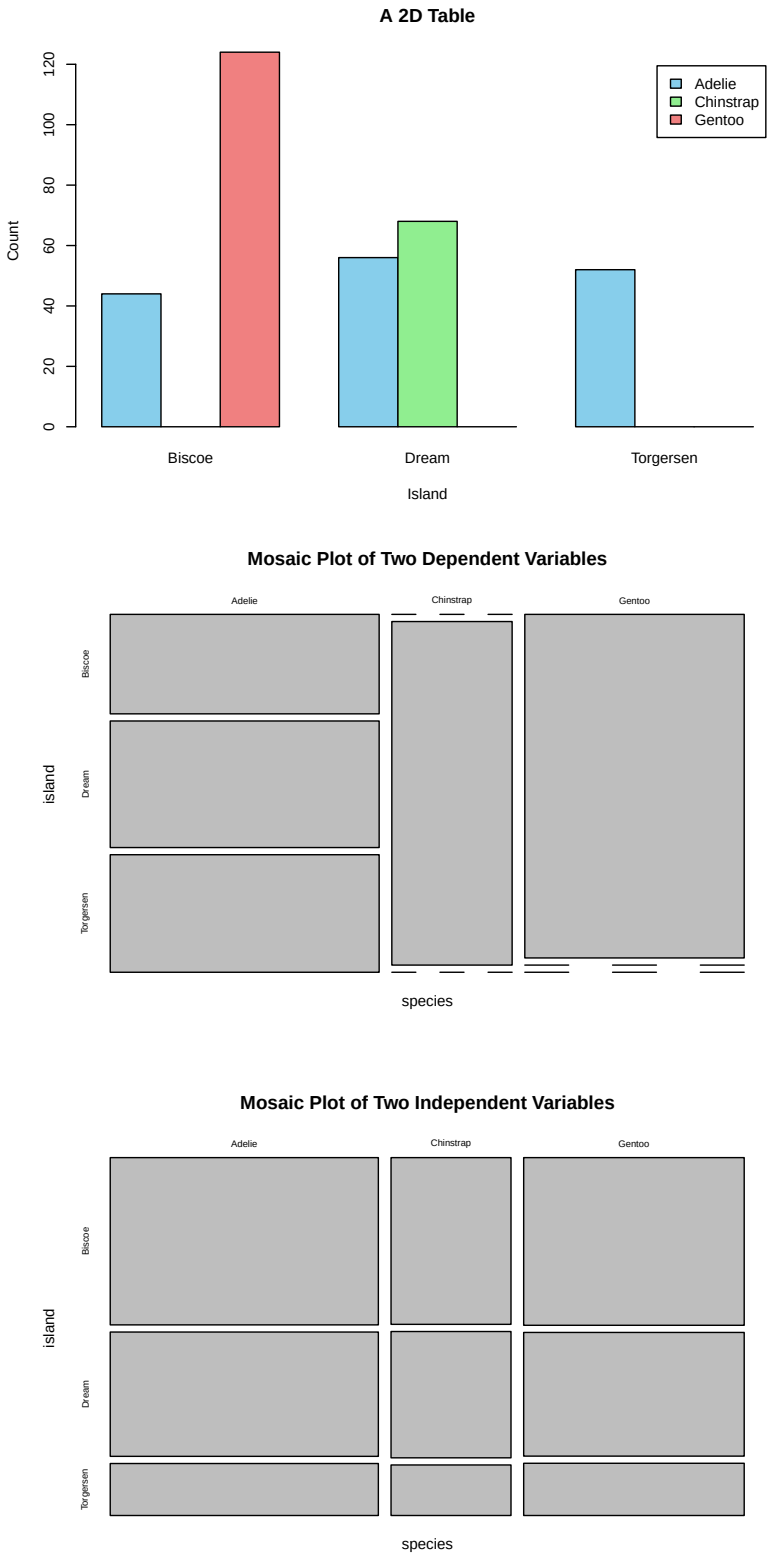


Figure 3.16: Visualization of a 2D Contingency Table. Top. A side-by-side barplot. Middle. Mosaic plot. Bottom. Mosaic plot for independent categorical variables

3.2.5 Two Numeric Variables

3.2.5.1 Scatterplot

A scatter plot is the default choice to visualize the relationship between two numeric variables. The relationship can vary, from linear to cyclical. Visualizing how the data points are distributed in a 2D plane is an intuitive way to understand their relationship. Importantly, visualization should be done before performing any statistical analysis, e.g., linear regression. Here is how to create a scatter plot that shows the relationship between sepal width and sepal length. Note that each point (x_i, y_i) is a pair of values from one sample.

```
par(mfrow=c(1,2))

# Top left
plot(x=iris$Sepal.Width,
     y=iris$Sepal.Length,
     xlab = "Sepal Width",
     ylab = "Sepal Length",
     main = "Default")

# Top right
plot(x=iris$Sepal.Width,
     y=iris$Sepal.Length,
     pch=3,
     xlab = "Sepal Width",
     ylab = "Sepal Length",
     main = "Change the Point Style")
```

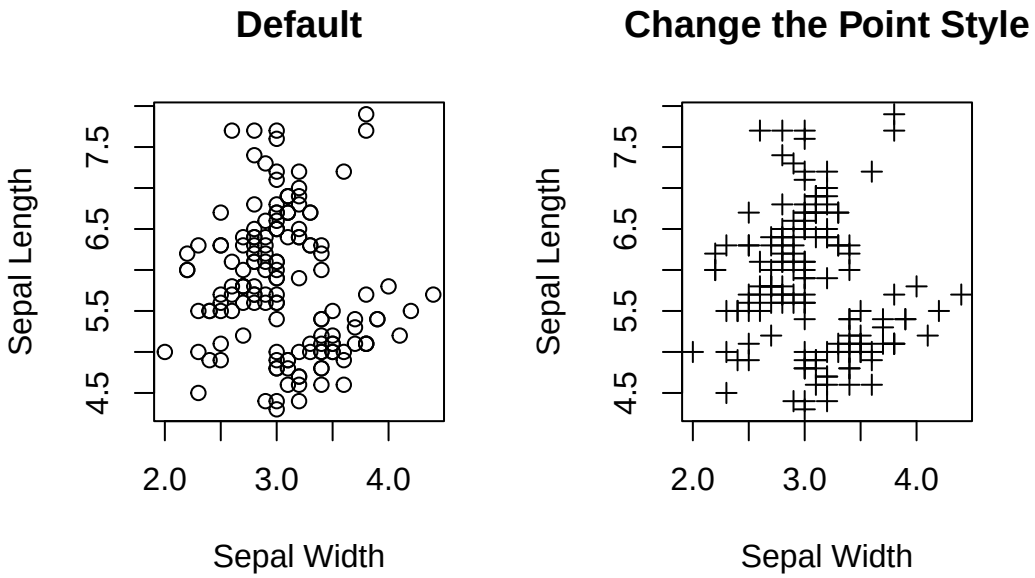


Figure 3.17: Visualizing the relationship between two numeric variables.

```
plot(x=iris$Sepal.Width,
     y=iris$Sepal.Length,
     pch=3,
     xlab = "Sepal Width",
     ylab = "Sepal Length",
     main = "Segregate Data Points by Color (Species)",
     col = c("skyblue", "lightgreen", "lightcoral")[as.numeric(iris$Species)])

legend("topright",
       legend = levels(iris$Species),
       col = c("skyblue", "lightgreen", "lightcoral"),
       pch = 3,
       title = "Species")
```

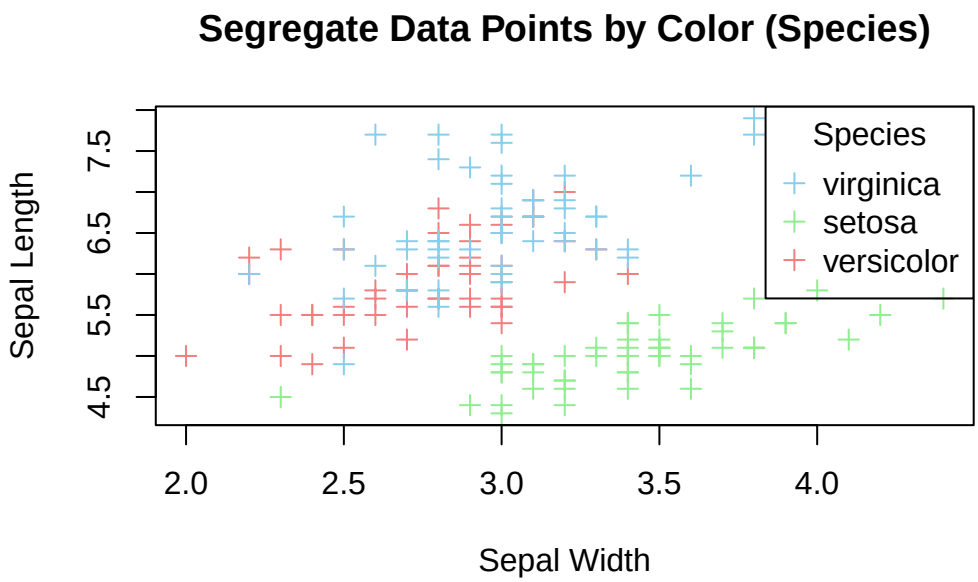


Figure 3.18: Color data points by species.

If you aim to explore the linear relationship between the two variables, you can emphasize it by adding a regression line for the two variables on top of the data points. Here is an example: regress `Sepal.Length` (response) on `Sepal.Width` (independent).

```
lm_model <- lm(Sepal.Length ~ Sepal.Width, data=iris)

# intercept
intercept <- lm_model$coefficients[1]

# slope
slope = lm_model$coefficients[2]

print(paste("Slope =", slope, "Intercept =", intercept))
```

```
[1] "Slope = -0.2233610611299 Intercept = 6.52622255089448"
```

```
plot(x=iris$Sepal.Width,
     y=iris$Sepal.Length,
     pch=3,
     xlab = "Sepal Width",
     ylab = "Sepal Length",
     main = "Add a Regression Line",
     col = c("skyblue", "lightgreen", "lightcoral")[as.numeric(iris$Species)])
```

```
legend("topright",
       legend = levels(iris$Species),
       col = c("skyblue", "lightgreen", "lightcoral"),
       pch = 3,
       title = "Species")

# Add a line
abline(a=intercept, b=slope, col='red')
```

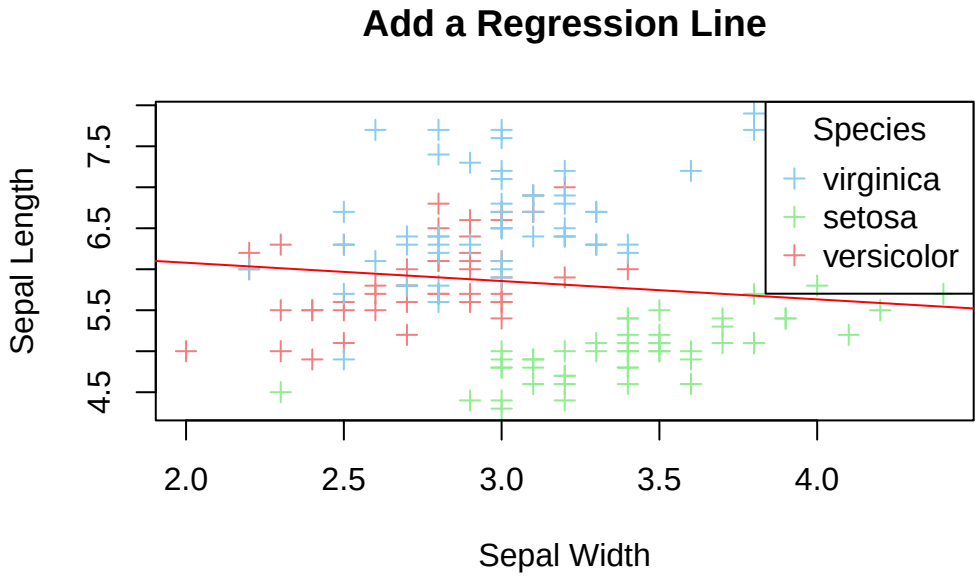


Figure 3.19: A scatter plot with a regression line.

3.2.5.2 Side-by-side Boxplot

If the comparison is between the descriptive statistics of two or more groups, a side-by-side plot is suggested. Suppose our interest is the sepal width between `setosa` and `versicolor`.

```
setosa <- subset(iris, Species == 'setosa')
versicolor <- subset(iris, Species == 'versicolor')

par(mfrow=c(1,2))

# Left
boxplot(setosa$Sepal.Width, versicolor$Sepal.Width)
```

```
# Right
boxplot(setosa$Sepal.Width, versicolor$Sepal.Width, names=c('setosa','versicolor'), xlab="Spe
```

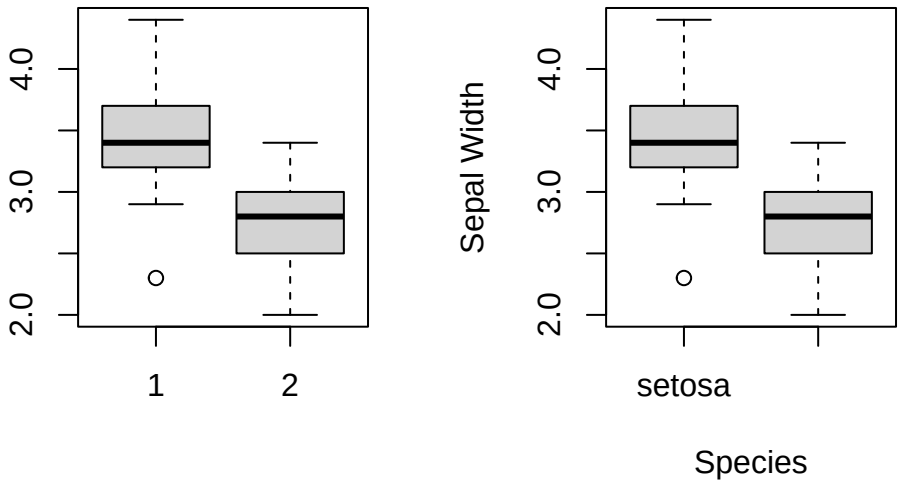


Figure 3.20: A Side-by-side Boxplot

3.2.6 Multipanel Scatter Plot

A multipanel scatter plot offers a glimpse of the relationship between all possible pairs of numeric variables. Suppose you are interested in the pairwise relationship between all numeric variables in `iris`. It can easily be done using the `pairs()` plot function.

```
pairs(iris[,1:4], main="A Multi-Panel Scatter Plot")
```

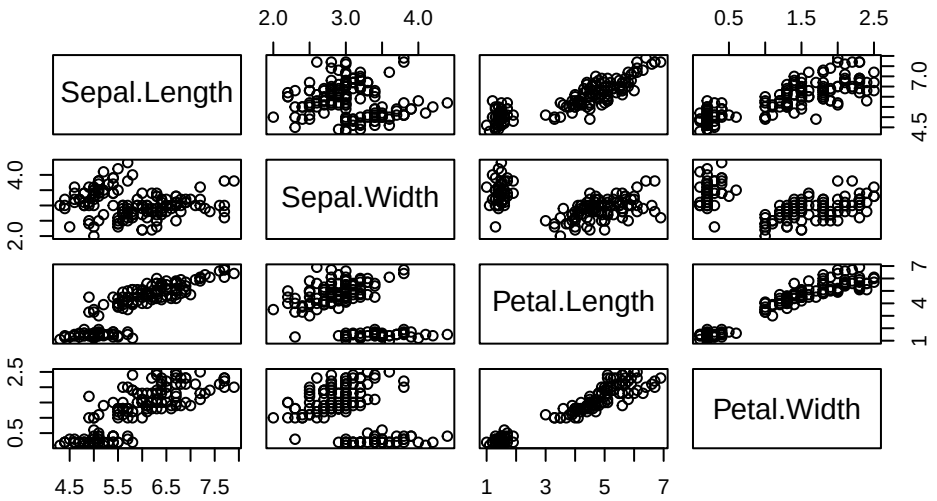


Figure 3.21: A Multipanel Scatter Plot

Here concludes our journey of data visualization using base R functions. As you can see, some plots can be made easily using base R (e.g., multipanel scatter plot), while others are less direct (e.g., bar plot with error bars). There is no one system you should use; choose whatever fits your requirements.

3.3 Exercises for base R

Q1. Use the built-in data `mtcars` to answer this question.

a. Make a histogram of `mpg`. Give meaningful labels to the axes. Create a title for the plot. Fill the histogram with your favorite color.

b. Create a box plot of horsepower by the number of cylinders. As above, modify the axis labels and main title. Color outliers in red.

c. Create a bar plot showing the number of cars by the number of gears `gear`. Label the x and y axes.

d. Create a pie chart of the number of cylinders with labels like Figure 3.7.

e. Use a mosaic plot to illustrate whether the number of cylinders and gears are independent.

f. Use a side-by-side bar for part e).

g. Create a scatter plot of acceleration `qsec` (x) by horsepower `hp` (y) with proper axis labels.

h. Create a multipanel plot of `mpg`, `hp`, `cyl`, `wt`, and `qsec` . Identify pairs of variables with a linear relationship.

Q2. Create a line plot of the number of cylinders (x) vs. acceleration `qsec` (y) with error bars. See Section 3.2.3.3 and Figure 3.14 for reference.

3.4 ggplot2

`ggplot2` is bundled in `tidyverse`. Once you have activated `tidyverse`, `ggplot2` is available. Here, the focus is on using `ggplot2` to plot the graphs that were created using base R above. We will not repeat the discussion of strengths and weaknesses of each graph in this section. Readers should refer to the corresponding section above for the characteristics and functions of the graphs.

If `tidyverse` has not been activated, load it into R before using `ggplot2`.

```
require(tidyverse)
```

```
Loading required package: tidyverse
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

3.4.1 Grammar of ggplot2

The two 'g's of `ggplot2` stand for **Grammar of Graphics**. In `ggplot2`, the graphical widgets are arranged in layers. That idea gives developers the flexibility to decorate a graph by adding layers to an existing graph. The **data** layer forms the foundation of a graph. The **Aesthetics** layer defines the association between data variable(s) and the axes. **Geometries** are the types of graphs, e.g., line plot, histogram, etc. **Facets** split a graph into multiple

panels. **Statistics** defines how to aggregate data, e.g., in proportion or take the face value in a barplot. **Coordinates** support an X-Y Cartesian coordinate system and a polar coordinate system. (See the Pie Chart section below) **Themes** offer different color schemes, font type, font size, etc.

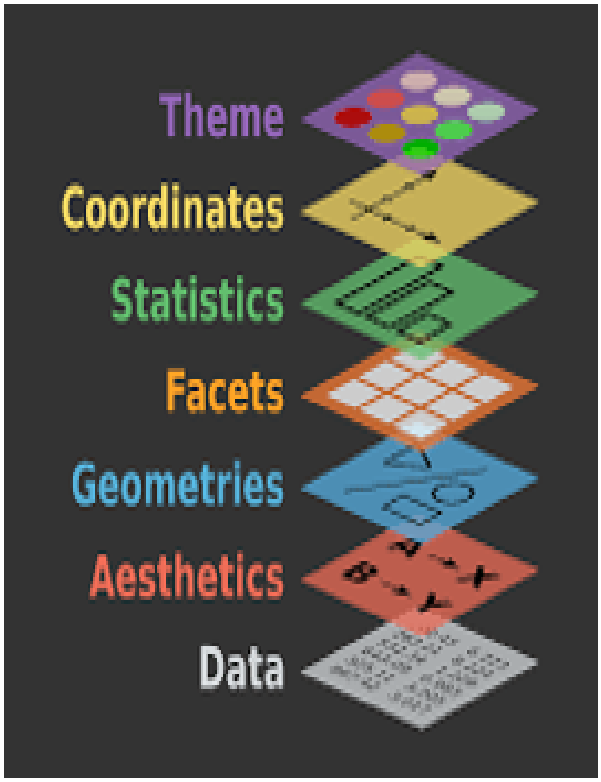


Figure 3.22: Anatomy of `ggplot2`

3.4.2 One Numeric Variable

You can plot a histogram of the numeric variable to see its distribution, such as the spread, the peak, symmetry, and flatness. If your concern is robust statistics, make a box plot instead.

3.4.2.1 Histogram

Let's redraw the histogram in Figure 3.1 using `ggplot2`. The first line is the *data layer* that specifies the data source, and it must be a data frame (or tibble), e.g., `iris`. The *aesthetic* option designates the x-axis for `Sepal.Width`. The second line is the *geometry* of the graph. For a histogram, the function name is `geom_histogram()`. Here is the bare minimum of a `ggplot2` histogram:

```
# Default
ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram()
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

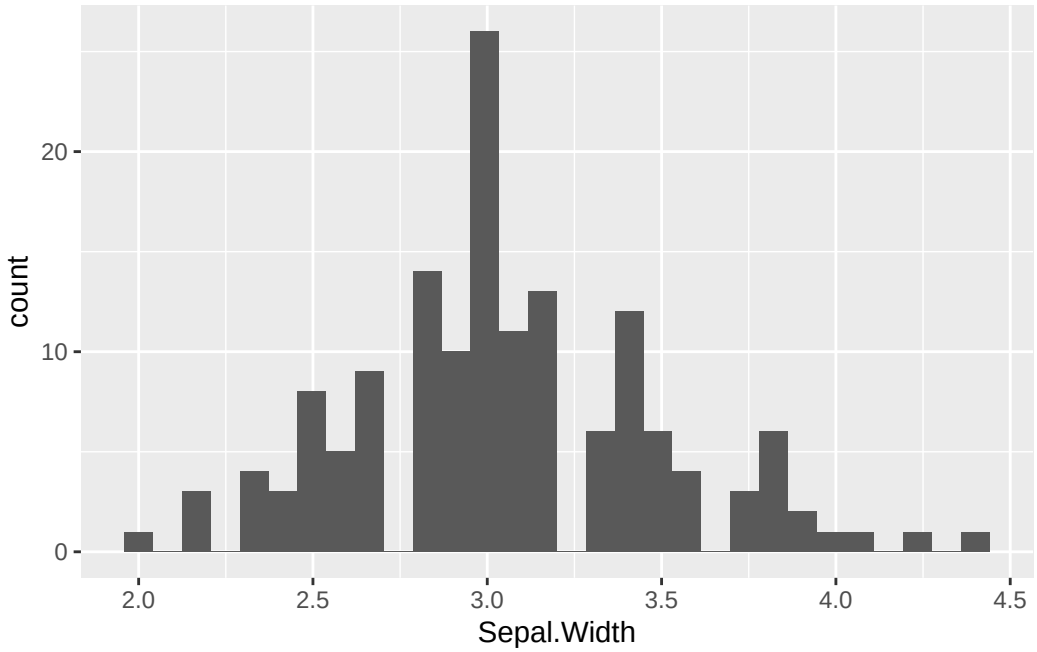


Figure 3.23: `ggplot2` Histogram.

If you see this message: "`stat_bin()` using `bins = 30`. Pick better value `binwidth`", R just informs you that 30 bins was used by default. You can ignore it or specify a new `binwidth` value.

It is always a good idea to give the graph a title at the top so people know what the key message is. Equally important are the x- and y-labels. The function `labs()` can do the job. By default, the title is left-justified. To center it, see the last line and the comment above it.

```
# Overwrite default x-/y-label and title
ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram() +
  labs(x="Sepal Width", y="Count", title="Histogram of Sepal Width") +
  # hjust = 0.5 means the horizontal adjustment is the midpoint.
  theme(plot.title = element_text(hjust = 0.5))
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

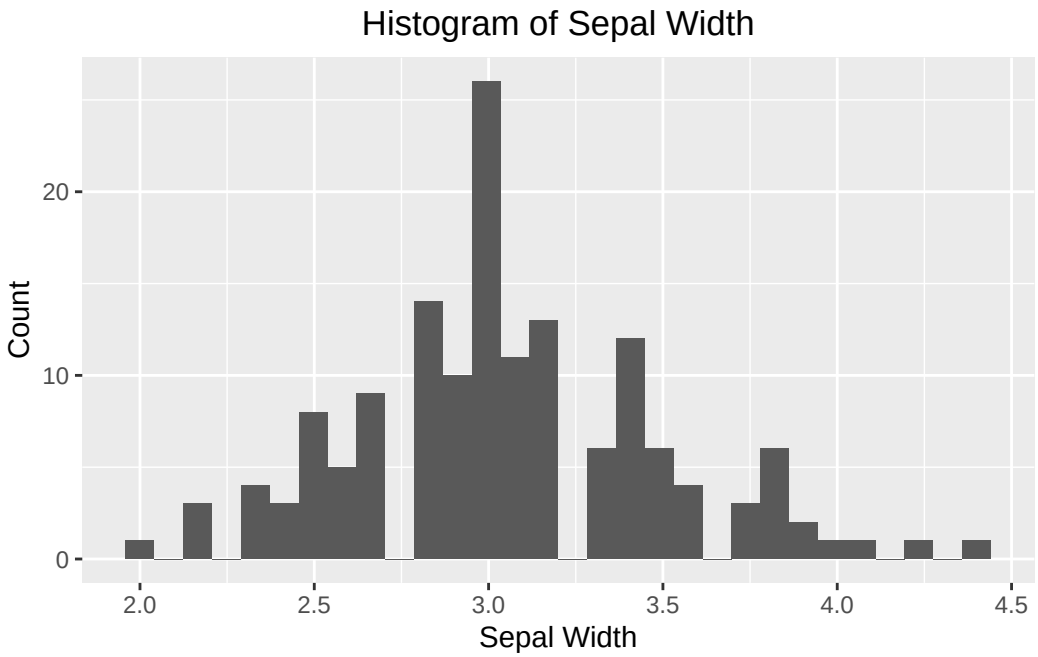


Figure 3.24: A `ggplot2` Histogram with customized labels and title.

Change the fill color and border color. It can be done by providing additional parameters to the `geom_histogram()`.

```
# fill = filled color
# color = border color
ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram(fill="skyblue", color = "white") +
  labs(x="Sepal Width", y="Count", title="Color Options") +
  # hjust = 0.5 means the horizontal adjustment is the midpoint.
  theme(plot.title = element_text(hjust = 0.5))
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

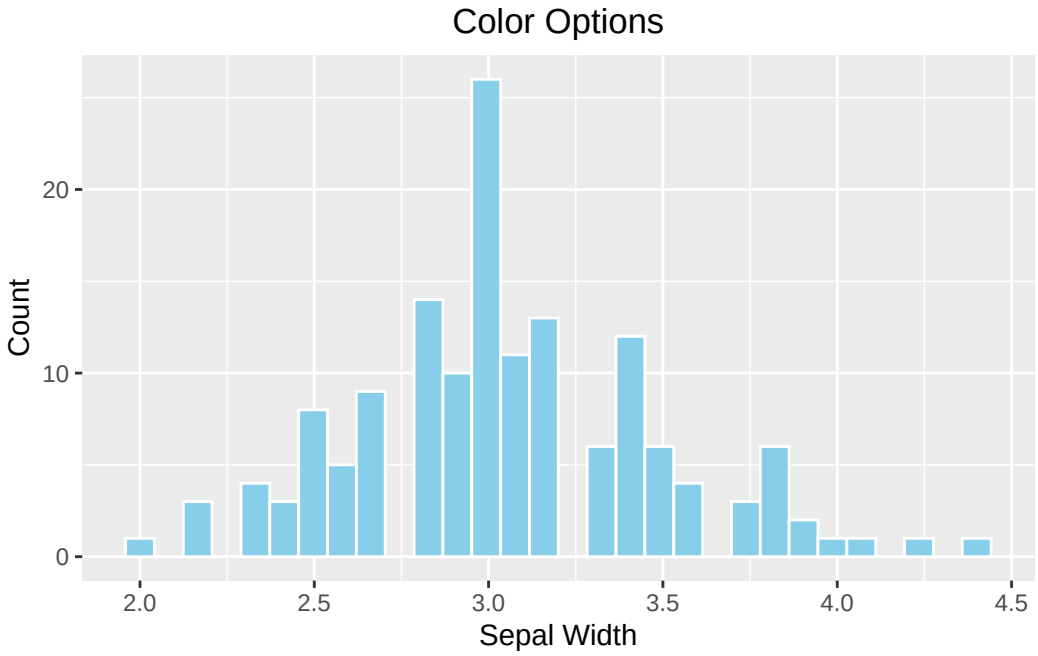


Figure 3.25: A `ggplot2` Histogram with a customized color.

The bin width of a histogram is calculated automatically by `ggplot2` according to the range of x. You can use the `breaks` parameter to overwrite it.

```
# breaks
ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram(fill="skyblue", color = "white", breaks=seq(2,4.5,by=0.1)) +
  labs(x="Sepal Width", y="Count", title="`breaks` option") +
  # hjust = 0.5 means the horizontal adjustment is the midpoint.
  theme(plot.title = element_text(hjust = 0.5))
```

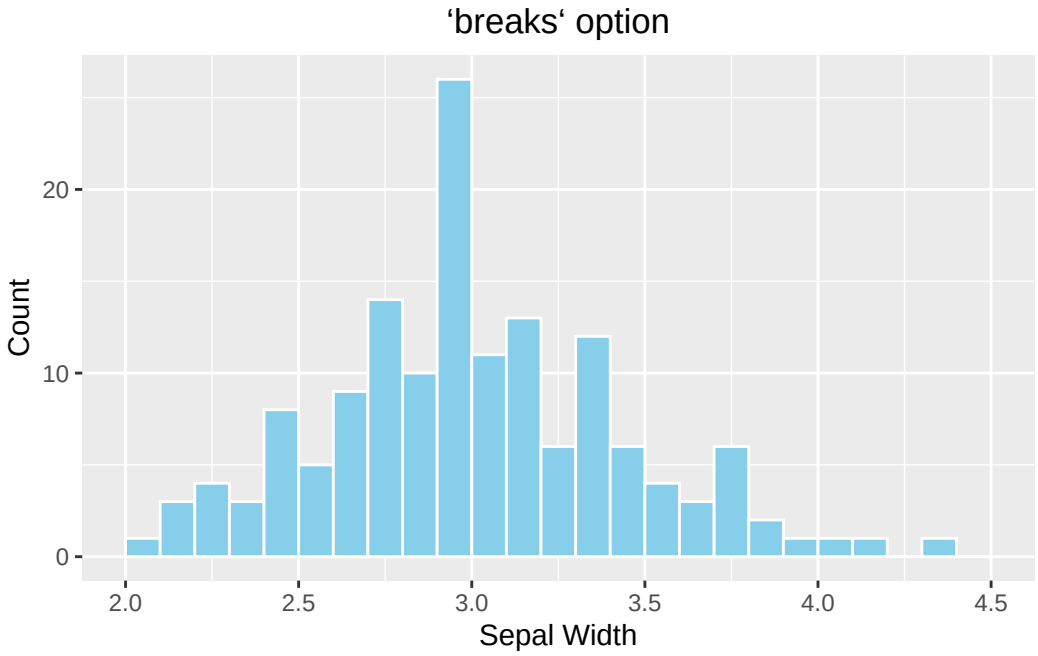


Figure 3.26: A `ggplot2` Histogram with a non-default bin width.

Optional topic

As you might notice, each of the four histograms occupies the entire plotting area instead of displaying in a 2x2 grid as in Figure 3.1. It is because `ggolot2` does not work with `par(mfrow(2,2))`. To divide the plotting area into a grid that is compatible with `ggplot2`, use the `patchwork` package. Use the menu option **Tools** → Install Packages to install it.

Another new concept is that a graph can be stored in a variable so it can be displayed later in the grid. In the example below, the four histograms above are initially stored in variables, `p1`, `p2`, `p3`, and `p4`. See the example below:

```
require(patchwork)

# Default
p1 <- ggplot(data=iris, aes(x=Sepal.Width)) +
        geom_histogram()

# With labels and title
p2 <- ggplot(data=iris, aes(x=Sepal.Width)) +
        geom_histogram() +
        labs(x="Sepal Width", y="Count", title="Histogram of Sepal Width") +
        # hjust = 0.5 means the horizontal adjustment is the midpoint.
```

```
        theme(plot.title = element_text(hjust = 0.5))

# fill = filled color
# colour = border color
p3 <- ggplot(data=iris, aes(x=Sepal.Width)) +
        geom_histogram(fill="skyblue", color = "white") +
        labs(x="Sepal Width", y="Count", title="Color Options") +
        # hjust = 0.5 means the horizontal adjustment is the midpoint.
        theme(plot.title = element_text(hjust = 0.5))

# Change the number of bins
p4 <- ggplot(data=iris, aes(x=Sepal.Width)) +
        geom_histogram(fill="skyblue", color = "white", breaks=seq(2,4.5,by=0.1)) +
        labs(x="Sepal Width", y="Count", title="`breaks` option") +
        # hjust = 0.5 means the horizontal adjustment is the midpoint.
        theme(plot.title = element_text(hjust = 0.5))

# Arrange them in a 2x2 grid
# '+' places plots next to each other, '/' stacks them
(p1 + p2) / (p3 + p4)
```

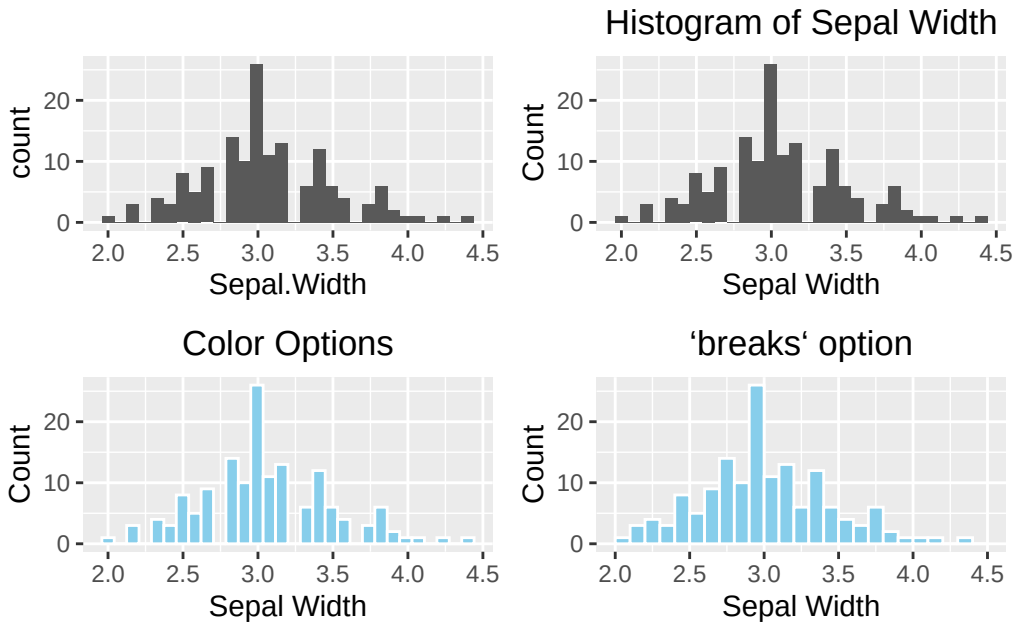


Figure 3.27: `ggplot2` Grid System by `patchwork`. Top left: Default (bare minimum). Top right: with labels and title. Bottom left: change the color. Bottom right: change the number of bins.

3.4.2.2 Box plot

The anatomy of a box plot can be found in Figure 3.3. The geometry function of a box plot is `geom_boxplot()`. Unlike a histogram, the numeric variable is mapped to the y-axis for a vertical box plot. If you want a horizontal box plot, map the numeric variable to the x-axis, and don't forget to specify an x-axis label.

```
require(patchwork)

# Default
p1 <- ggplot(data=iris, aes(y=Sepal.Width)) +
        geom_boxplot()

# Add y-label and title
p2 <- ggplot(data=iris, aes(y=Sepal.Width)) +
        geom_boxplot() +
        labs(y="Sepal Width", title = "box plot of Sepal Width") +
        theme(plot.title = element_text(hjust = 0.5))

# Change color
p3 <- ggplot(data=iris, aes(y=Sepal.Width)) +
        geom_boxplot(fill="skyblue", outlier.color = "red") +
        labs(y="Sepal Width", title = "Color options") +
        theme(plot.title = element_text(hjust = 0.5))

# Horizontal box plot
p4 <- ggplot(data=iris, aes(x=Sepal.Width)) +
        geom_boxplot() +
        labs(x="Sepal Width", title = "Horizontal box plot") +
        theme(plot.title = element_text(hjust = 0.5))

(p1 + p2)/(p3 + p4)
```

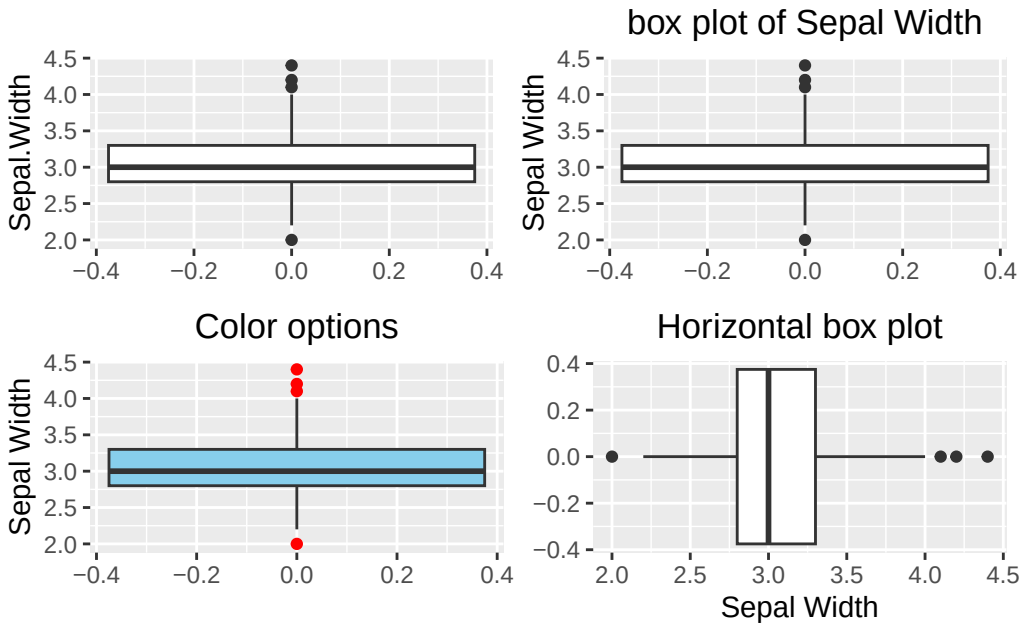


Figure 3.28: box plot using `ggplot2`. Top left: default (bare minimum). Top right: with y-label and title. Bottom left: change color. Bottom right: horizontal box plot.

3.4.3 One Categorical Variable

3.4.3.1 Bar Plot

Here we will show how to make a bar plot and a pie chart using `ggplot2`.

```
require(patchwork)

# Default
p1 <- ggplot(data=iris, aes(x=Species)) +
        geom_bar()

# With labels and title
p2 <- ggplot(data=iris, aes(x=Species)) +
        geom_bar() +
        labs(x="Species", y="Count", title = "A Barplot of Species") +
        theme(plot.title = element_text(hjust = 0.5))

# Color option
p3 <- ggplot(data=iris, aes(x=Species)) +
        geom_bar(fill="skyblue", color="blue") +
```

```
        labs(x="Species", y="Count", title = "A Barplot of Species") +
        theme(plot.title = element_text(hjust = 0.5))

# Horizontal barplot
p4 <- ggplot(data=iris, aes(y=Species)) +
        geom_bar(fill="skyblue", color="blue") +
        labs(y="Species", x="Count", title = "A Horizontal Barplot of Species") +
        theme(plot.title = element_text(hjust = 0.5))

(p1 + p2)/(p3 + p4)
```

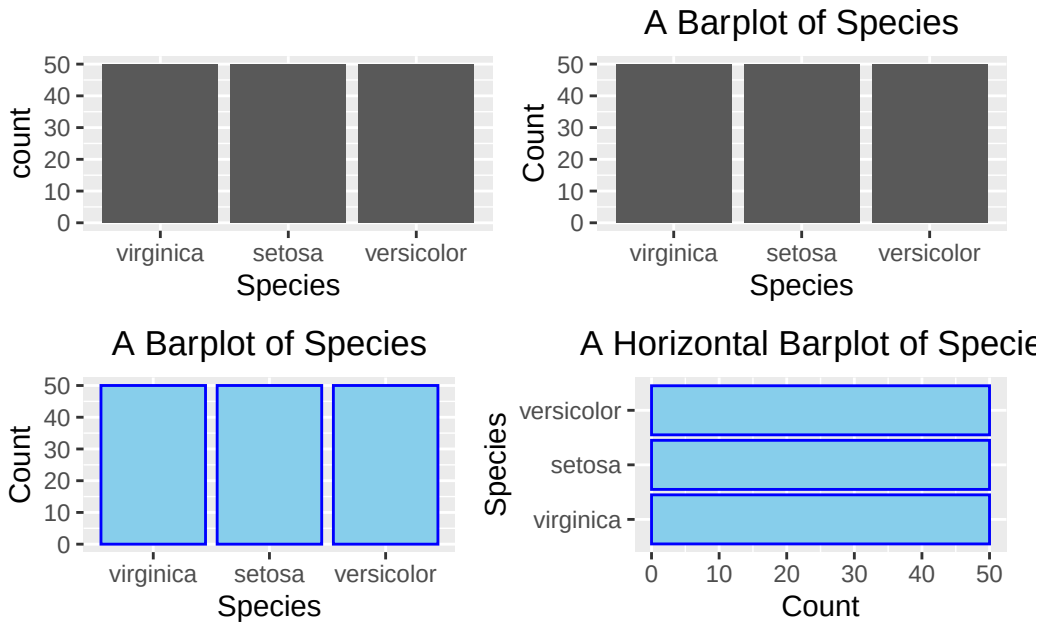


Figure 3.29: Barplot using `ggplot2`. Top left: default (bare minimum).

3.4.3.2 Pie Chart

Here's a curve ball. You may give an impression that `ggplots` is an upgrade. Plotting a pie chart should be a piece of cake. The fact is NO.

Despite the disappointment, it is worth seeing how a pie chart can be plotted using the foundational building-block functions. To give you a heads-up, `ggplot2` view a pie chart as a twisted bar plot. There is an internal logic to justify such a view, but we will not discuss it in detail.

Suppose we want to create a pie chart like the one below:

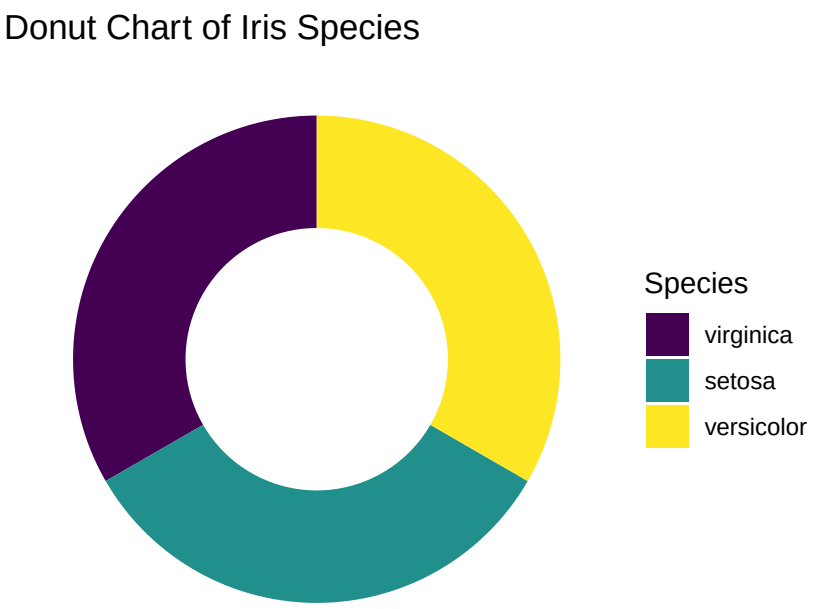


Figure 3.30: Pie chart or a donut chart

Here we will walk through each of the intermediate steps until getting the final pie chart.

Let's begin with a stacked barplot, where the bars are stacked up into a single column, and the fill color is an attribute to distinguish different iris species (`fill=Species`). The stacked bar is placed at the position 2 units on the right from the origin (`aes(x=2)`), but the exact position does not matter.

```
ggplot(data=iris, aes(x=2, fill=Species)) +
  geom_bar()
```

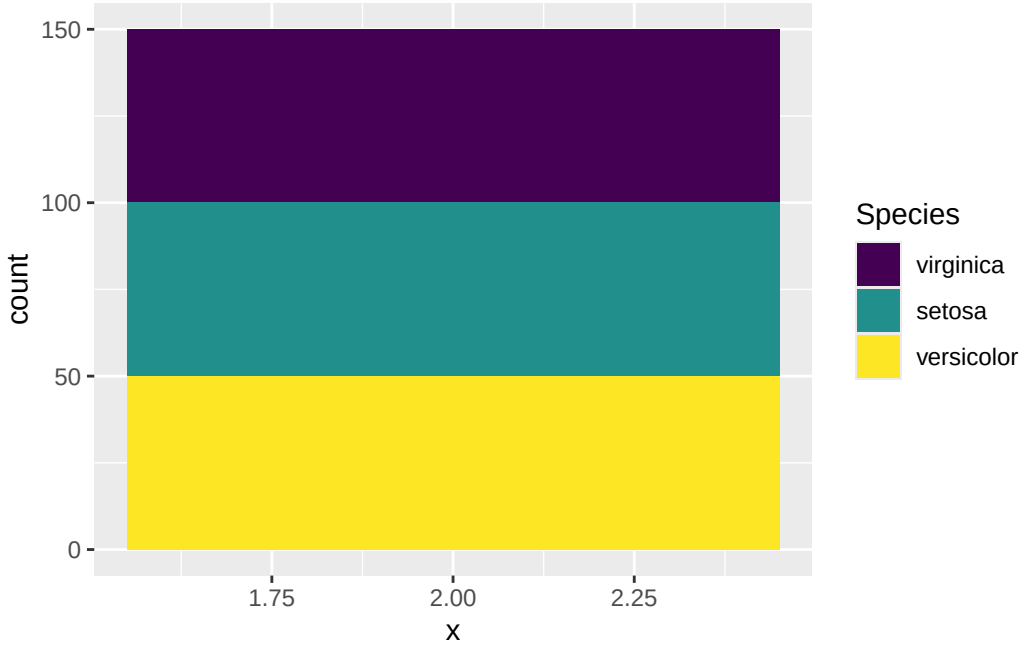


Figure 3.31: A stacked barplot

Now, use your imagination to picture that the stacked bar is bent into a ring. Does it look like the pie chart in Figure 3.30? One way to "bend" it in `ggplot2` is to change the x-y Cartesian coordinate system to the polar (circular) coordinate system in which a point is characterized by two parameters: the distance from the origin (r), and the angle (θ) from the horizontal **polar axis** in a counter-clockwise direction.

By adding this line `coord_polar(theta="y", start = 0)` in the code below, it transforms the height of the bar for each species into an angle (θ) in the polar coordinate, where the zero position is the 12 o'clock (`start = 0`). In this example, one third of the samples is `setosa`. As such, the angle θ is $360^\circ \times \frac{1}{3} = 120^\circ$. If the proportion is a quarter, the angle $\theta = 360^\circ \times \frac{1}{4} = 90^\circ$.

```
ggplot(data=iris, aes(x="", fill=Species)) +
  geom_bar() +
  coord_polar(theta="y", start = 0)
```

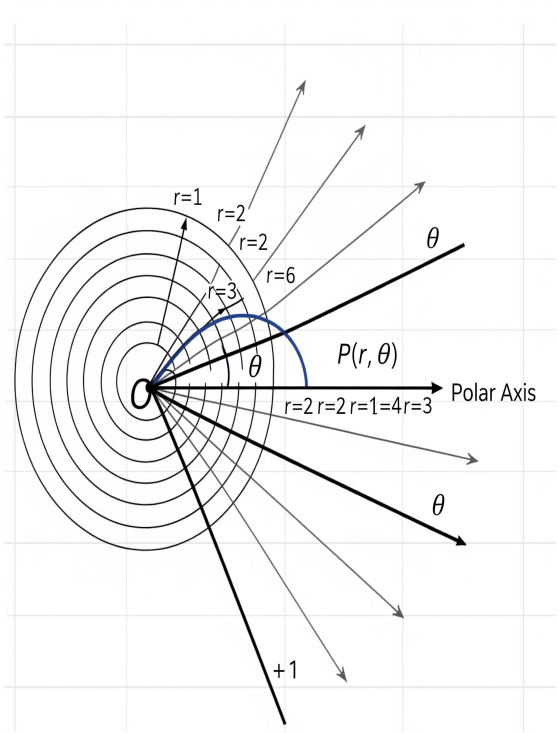


Figure 3.32: Polar Coordinate. (generated by Gemini)

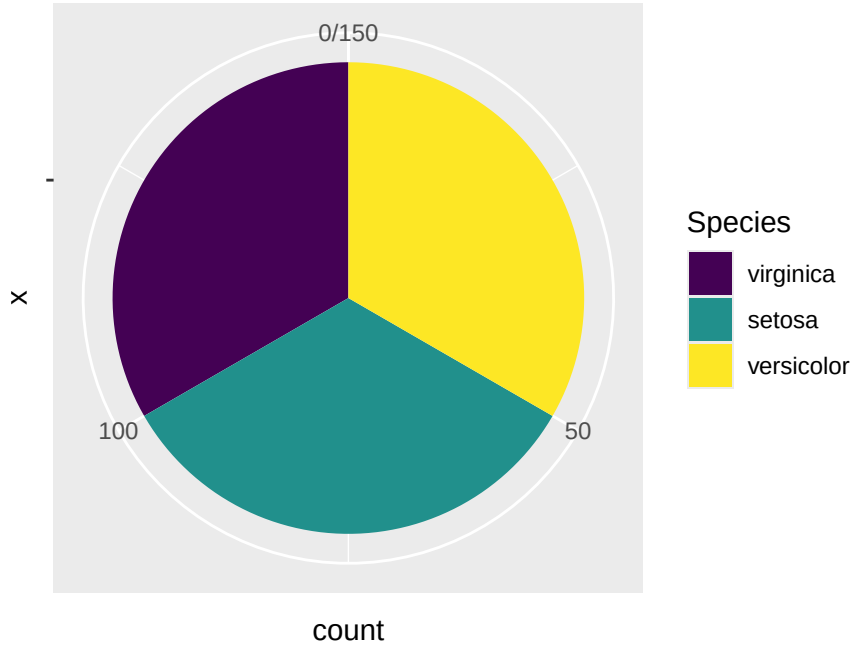


Figure 3.33: A Piechart by changing to the polar coordinate system.

The final step is to open a hole in the pie chart if you like, for aesthetic reasons. But this modification is optional.

```
ggplot(iris, aes(x = 2, fill = Species)) +
  geom_bar(width = 1, stat = "count") +
  coord_polar(theta = "y", start = 0) +
  # This is the key part that creates the hole
  xlim(0.5, 2.5) +
  labs(title = "Donut Chart of Iris Species")
```

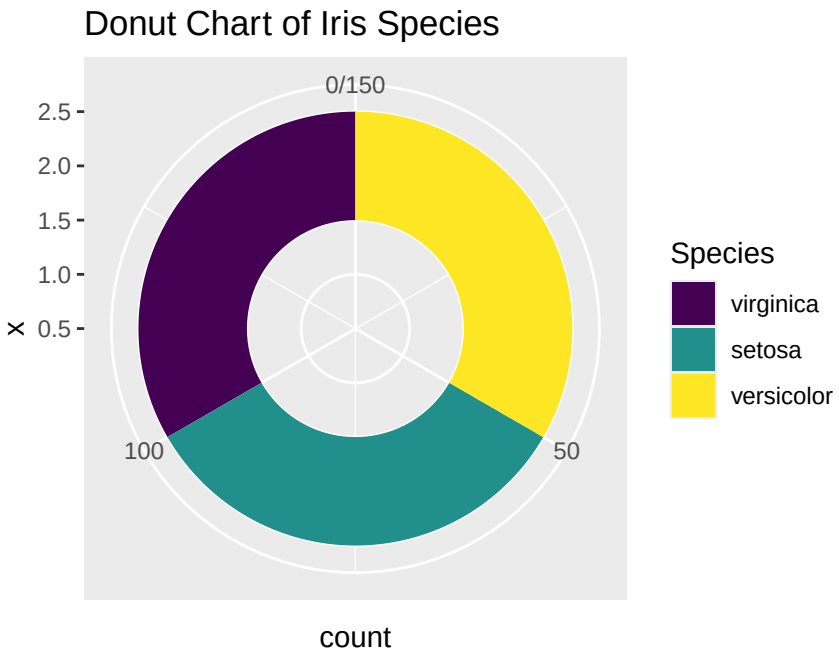


Figure 3.34: A Piechart using `ggplot2`.

If you prefer to have a clean background, add the line `theme_void()`. Finally, here's the desired pie chart. What a journey!

```
ggplot(iris, aes(x = 2, fill = Species)) +
  geom_bar(width = 1, stat = "count") +
  coord_polar(theta = "y", start = 0) +
  # This is the key part that creates the hole
  xlim(0.5, 2.5) +
  theme_void() +
  labs(title = "Donut Chart of Iris Species")
```

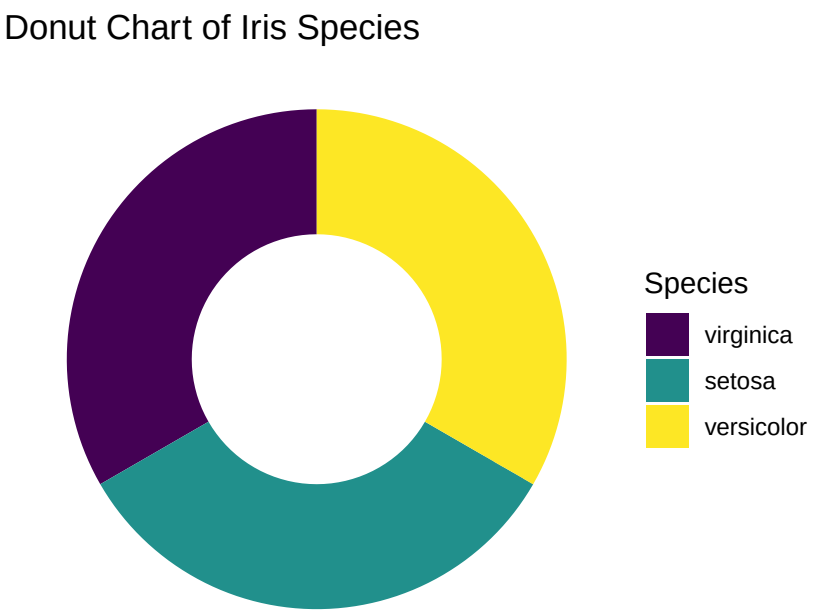


Figure 3.35: Piechart using `ggplot2` without no background.

3.4.4 One Categorical Variable and One Numeric Variable

3.4.4.1 Side-by-side Box Plot

Here you want to compare the robust statistics among different species.

```
require(patchwork)

p1 <- ggplot(data=iris, aes(x=Species, y=Sepal.Length)) +
        geom_boxplot() +
        labs(title = "A Side-by-Side box plot") +
        theme(plot.title = element_text(hjust = 0.5))

p2 <- ggplot(data=iris, aes(x=Sepal.Length, y=Species)) +
        geom_boxplot() +
        labs(title = "A Horizontal Side-by-Side box plot") +
        theme(plot.title = element_text(hjust = 0.5))

p1/p2
```

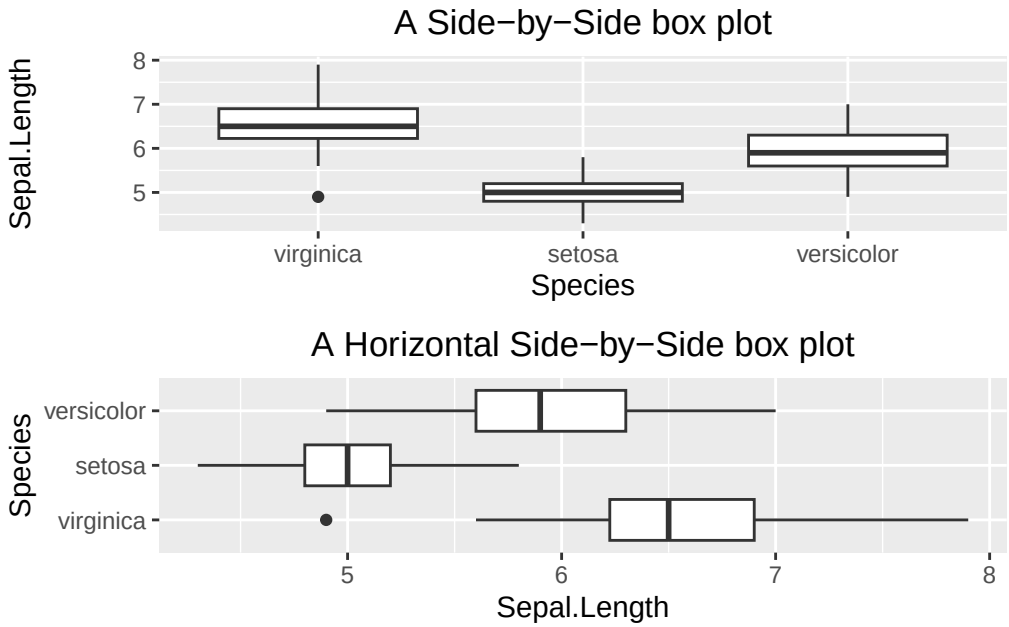


Figure 3.36: A side-by-side box plot

3.4.4.2 Side-by-side Bar Plot

Here, you want to compare the average `Sepal.Length` among iris species. As discussed above, you calculate the averages and store them in a separate table. It uses pipe, group_by, and summarize concepts discussed in Section 2.3.11.

```
iris_summary1 <- iris %>%
                  group_by(Species) %>%
                  summarize(mean_sepal_length = mean(Sepal.Length))
iris_summary1
```

```
# A tibble: 3 x 2
  Species    mean_sepal_length
  <ord>                  <dbl>
1 virginica               6.59
2 setosa                  5.01
3 versicolor              5.94
```

And then, you can plot the summary table. The geometry function is `geom_bar(stat = "identity")`. The `stat = "identity"` parameter tells the `geom_bar()` function to set up the height of the bars according to the face value of `mean_sepal_length`.

```
ggplot(data=iris_summary1, aes(x=Species, y=mean_sepal_length)) +
  geom_bar(stat = "identity") +
  labs(x="Species", y="Average Sepal Length", title = "Side-by-side Barplot") +
  theme(plot.title = element_text(hjust = 0.5))
```

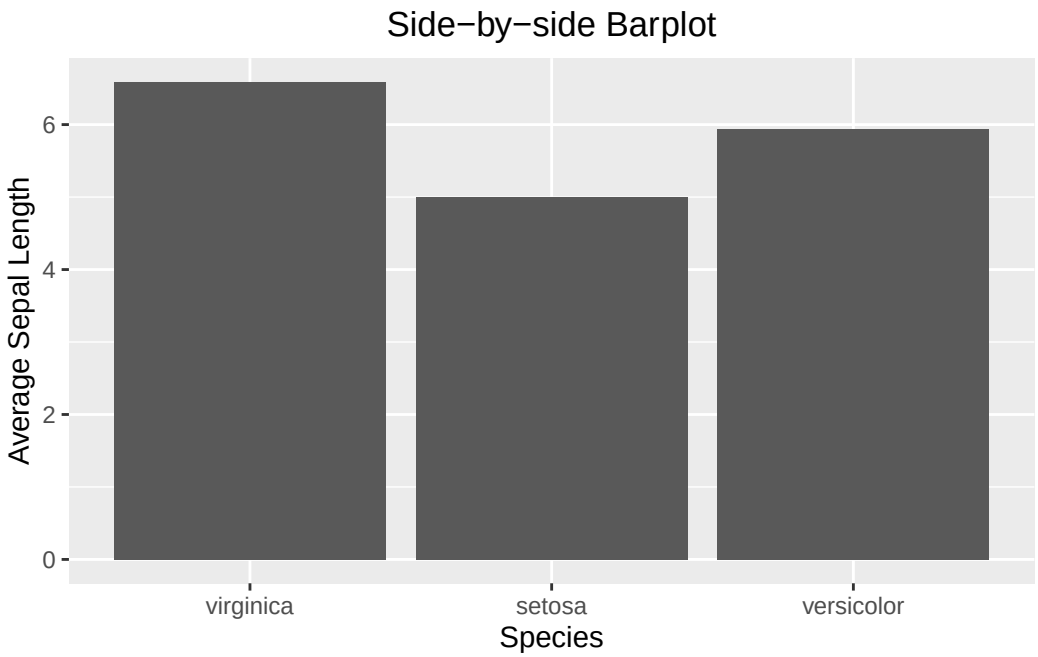


Figure 3.37: A side-by-side bar plot

3.4.4.3 Side-by-side Bar Plot with Error Bar

The calculation of the error bar (i.e., the standard error `se`) requires standard deviation and sample size. Once `se` is calculated, it can be used to calculate the minimum (lower) and maximum (upper) of the error bar.

```
iris_summary2 <- iris %>%
                  group_by(Species) %>%
                  summarize(mean_sepal_length = mean(Sepal.Length),
                            N=n(),
                            se=sd(Sepal.Length)/sqrt(N)) %>%
                  # add the y minimum and y maximum columns
                  mutate(ymin = mean_sepal_length - se,
                         ymax = mean_sepal_length + se)
iris_summary2
```

```
# A tibble: 3 x 6
  Species    mean_sepal_length     N     se  ymin  ymax
  <ord>                  <dbl> <int>  <dbl> <dbl> <dbl>
1 virginica               6.59    50 0.0899  6.50  6.68
2 setosa                  5.01    50 0.0498  4.96  5.06
3 versicolor              5.94    50 0.0730  5.86  6.01
```

```
ggplot(data=iris_summary2, aes(x=Species, y=mean_sepal_length)) +
  geom_bar(stat = "identity") +
  geom_errorbar(aes(ymin = ymin, ymax = ymax), width = 0.2, color='red') +
  labs(x="Species", y="Average Sepal Length", title = "Side-by-side Barplot") +
  theme(plot.title = element_text(hjust = 0.5))
```

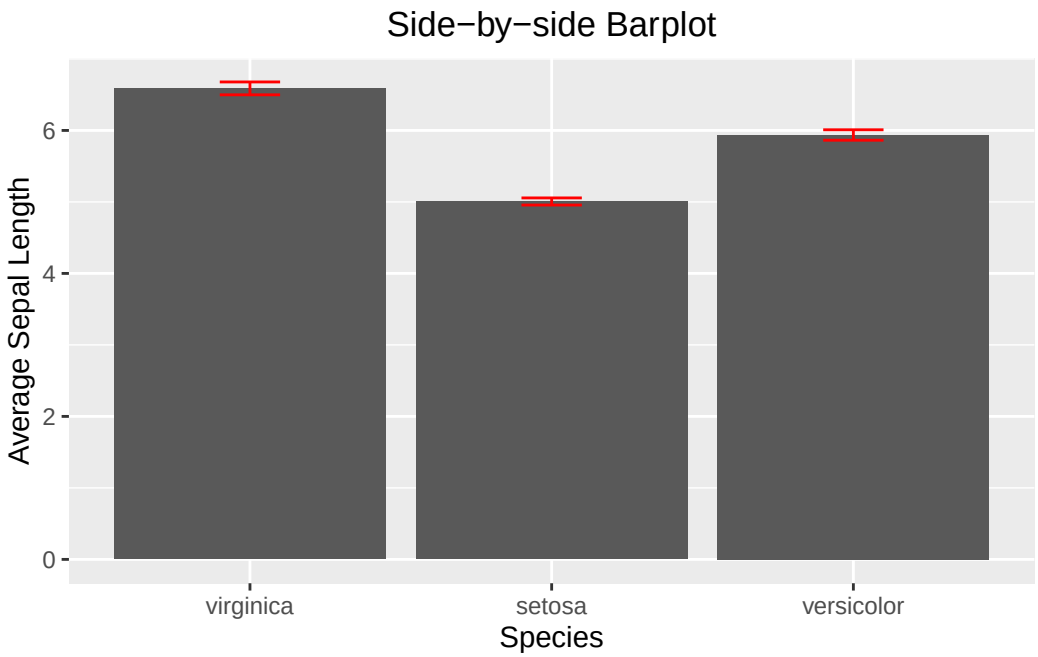


Figure 3.38: A side-by-side bar plot with error bar.

3.4.4.4 Line Graph with Error Bar

Calculate the SE as above:

```
iris_summary3 <- iris %>%
                 group_by(Species) %>%
                 summarize(mean_sepal_length = mean(Sepal.Length),
```

```
                            N=n(),
                            se=sd(Sepal.Length)/sqrt(N)) %>%
                  mutate(ymin=mean_sepal_length-se,
                         ymax=mean_sepal_length+se)
iris_summary3
```

```
# A tibble: 3 x 6
  Species    mean_sepal_length     N     se  ymin  ymax
  <ord>                  <dbl> <int>  <dbl> <dbl> <dbl>
1 virginica               6.59    50 0.0899  6.50  6.68
2 setosa                  5.01    50 0.0498  4.96  5.06
3 versicolor              5.94    50 0.0730  5.86  6.01
```

Incorporate the SE into a line plot.

```
ggplot(iris_summary3, aes(x=Species, y=mean_sepal_length, group = 1)) +
  geom_line(color = 'skyblue') +
  geom_point(size = 3, color = 'skyblue') +
  geom_errorbar(aes(ymin = ymin, ymax = ymax), width = 0.1, linewidth = 1) +
  labs(x="Species", y="Sepal Length", title = "Line Plot with Erorr Bar") +
  theme(plot.title = element_text(hjust = 0.5))
```

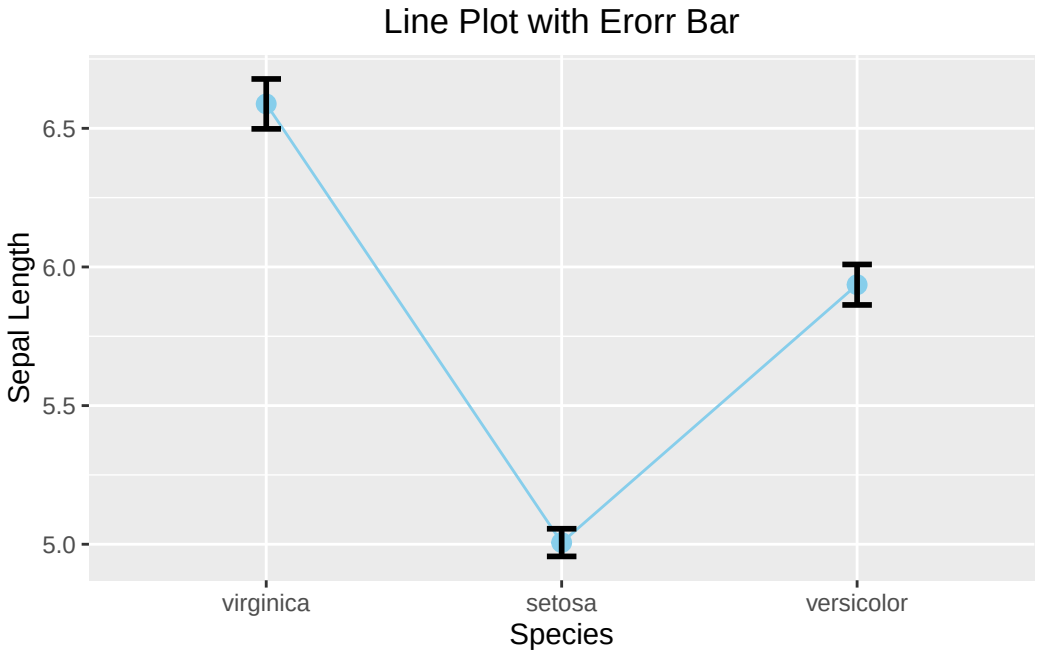


Figure 3.39: A line plot with error bar.

3.4.4.5 Overlapping Histogram

Like the base R side-by-side histogram, you will set the transparency to 0.5 and bin width to 0.25. Note that there is no need to care about the x- and y-limits of `ggplot2`.

```
ggplot(data=iris, aes(x=Sepal.Length, fill=Species)) +
  geom_histogram(alpha=0.5, binwidth = 0.25) +
  labs(x="Sepal Length", title = "Overlapping Histogram") +
  theme(plot.title = element_text(hjust = 0.5))
```

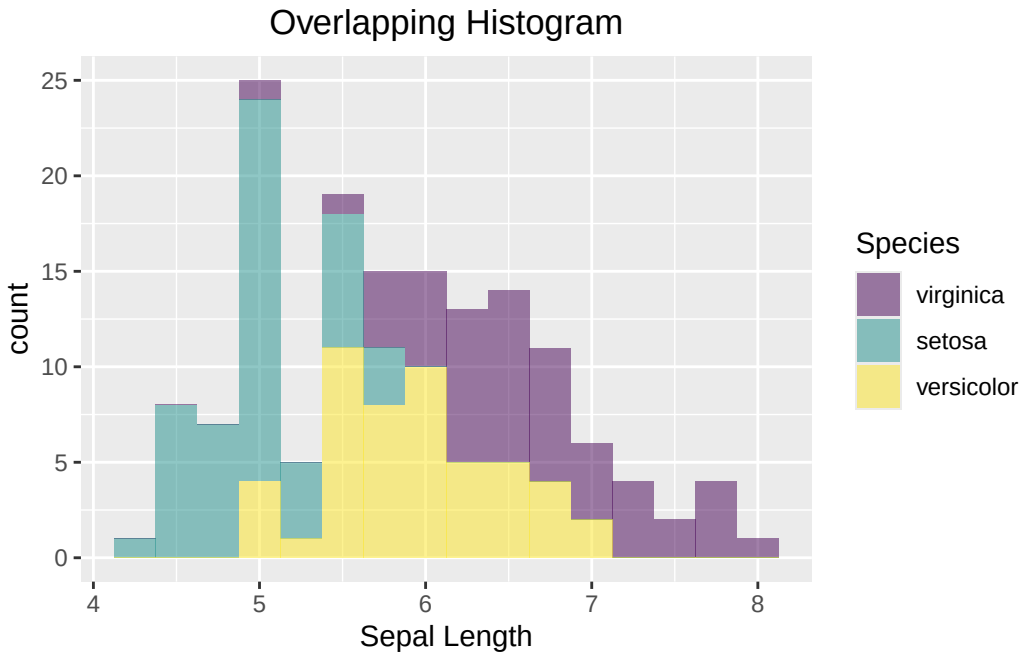


Figure 3.40: A side-by-side histogram.

3.4.5 Contingency Table

The `penguins` data set is used to illustrate how to create a side-by-side bar plot or a mosaic plot for displaying count data.

First, let's create a contingency table as below:

```
mytab <- with(penguins, table(species, island))
mytab
```

```
           island
species     Biscoe Dream Torgersen
  Adelie        44    56        52
  Chinstrap      0    68         0
  Gentoo       124     0         0
```

As `ggplot2` only works with data frames, `mytab` is **not** a data frame. Therefore, the first step is to convert the contingency table into a data frame.

```
mydf <- as.data.frame(mytab)
colnames(mydf) <- c("species","island","Count")
mydf
```

```
    species    island Count
1    Adelie    Biscoe    44
2 Chinstrap    Biscoe     0
3    Gentoo    Biscoe   124
4    Adelie     Dream    56
5 Chinstrap     Dream    68
6    Gentoo     Dream     0
7    Adelie Torgersen    52
8 Chinstrap Torgersen     0
9    Gentoo Torgersen     0
```

Now we can plot `mydf`.

```
ggplot(data=mydf, aes(x=island, y=Count, fill=species)) +
  geom_col(position = "dodge") +
  labs(title = "Side-by-side Barplot", x = "Island", y = "Count")
```

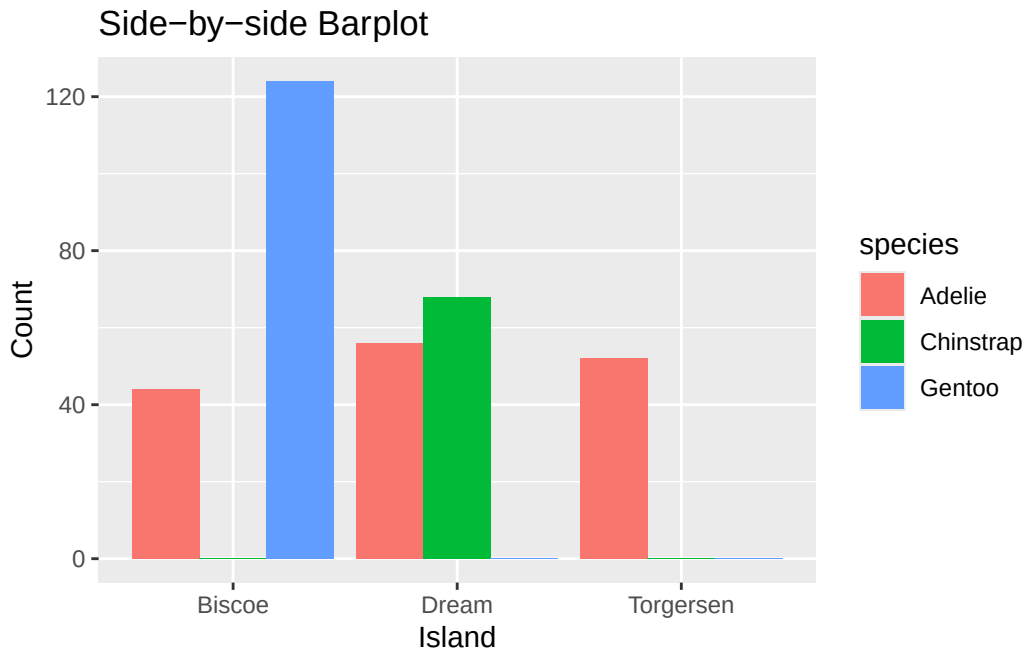


Figure 3.41: Plotting a contingency table using a side-by-side bar plot.

3.4.6 Two Numeric Variables

Use a scatter plot to visualize the relationship between two numeric variables. As mentioned above, the relationship can be linear or non-linear.

```
ggplot(iris, aes(x=Sepal.Width, y=Sepal.Length)) +
  geom_point() +
  labs(x="Sepal Width", y="Sepal Length", title = "A Scatter Plot") +
  theme(plot.title = element_text(hjust = 0.5))
```

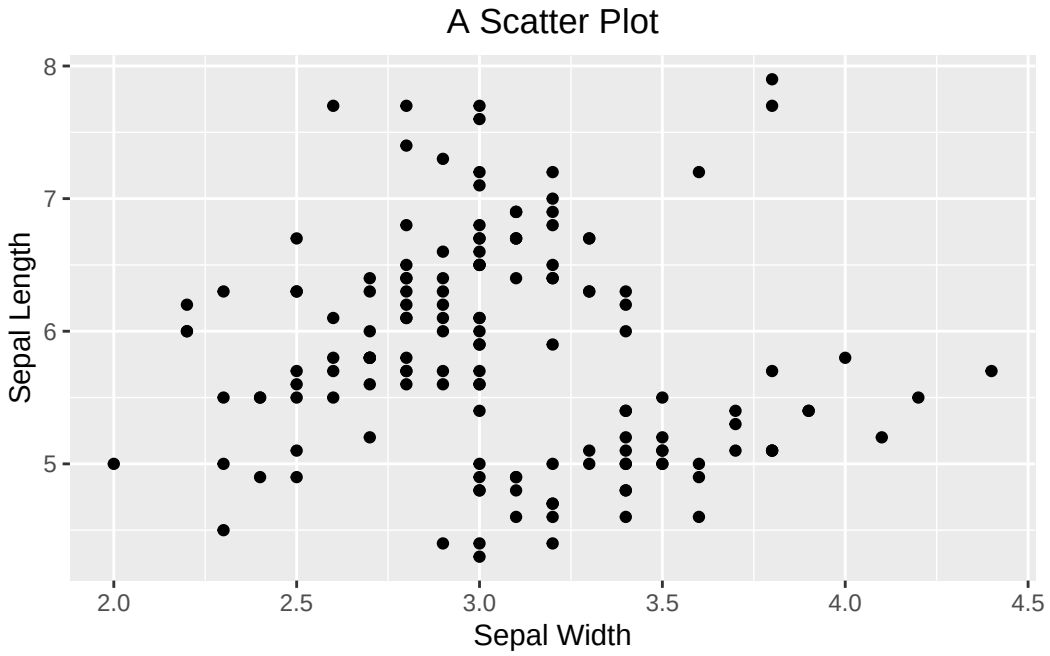


Figure 3.42: Visualizing the relationship between two numeric variables using a scatter plot.

If you want to emphasize their linear relationship, you can do this convincingly by adding a regression line on top of the scatter plot. `ggplot2` spares you the effort of creating a linear model in base R. Instead, it can be done by adding the `geom_smooth()`. You tell `geom_smooth(method = "lm", se = FALSE, color = "red")` to use a linear method (`lm`) for smoothing the data points. See the code below:

```
ggplot(iris, aes(x=Sepal.Width, y=Sepal.Length)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(x="Sepal Width", y="Sepal Length", title = "Scatter Plot with the Regression Line") +
  theme(plot.title = element_text(hjust = 0.5))
```

```
`geom_smooth()` using formula = 'y ~ x'
```

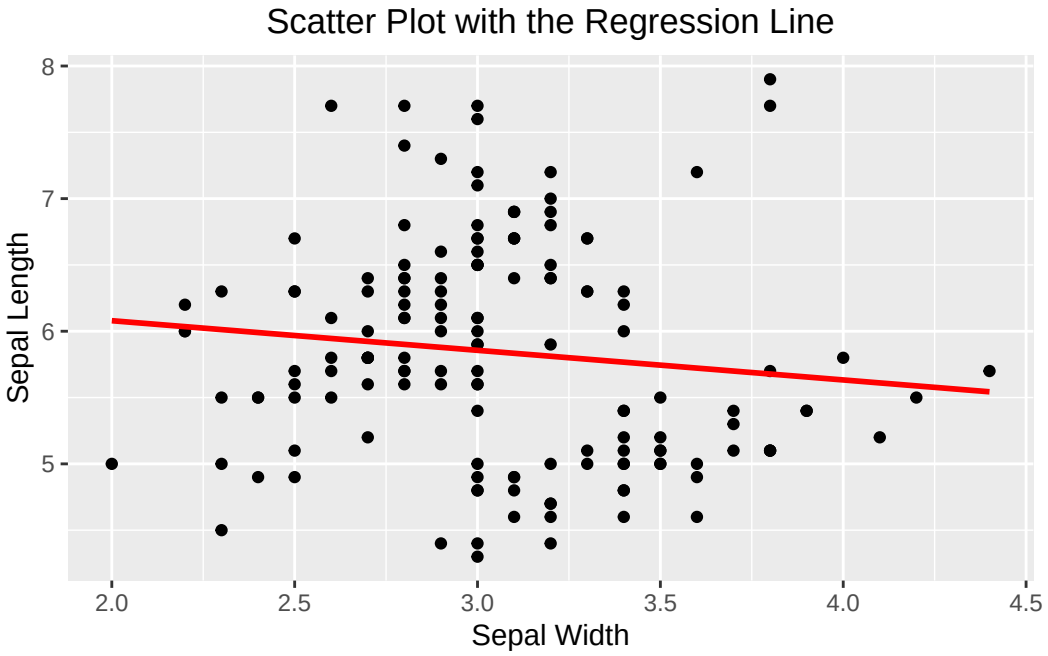


Figure 3.43: A scatter plot overlaid with a regression line.

If you see this message "`geom_smooth()` using formula = 'y ~ x'", it is harmless and informational. `ggplot2` is just to let you know what the response and predictor are used by `geom_smooth()` in creating the regression line.

3.4.7 Multipanel Scatter Plot

Oftentimes before multiple linear regression, it is recommended to visualize the relationship between the response variable and the independent variables pairwisely, or the linear relationship between independent variables. The latter is important, as it may cause the **collinearity** issue if independent variables are linearly correlated.

A multipanel scatter plot can do this. However, `ggplot2` does not have a geometry function for it. The simplest way is to use the `ggpairs()` function provided by the `GGally` package, which requires installation if you have not done so.

Suppose you are interested in the pairwise relationship between all numeric variables in `iris`. It can easily be done using the `ggpairs()` plot function. Color is the attribute used to distinguish species. `progress = FALSE` is to suppress the printing of the progress bar.

```
library(GGally)
ggpairs(iris, aes(color = Species), title = "A Multi-panel Scatter Plot", progress = FALSE)
```

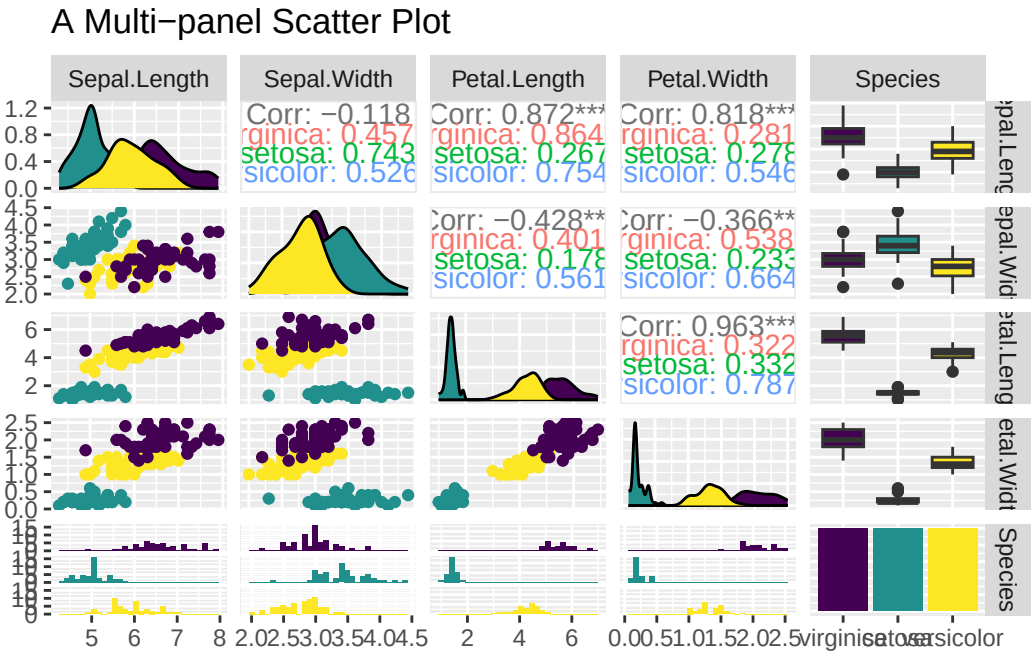


Figure 3.44: A Multipanel Scatter Plot

3.5 Exercises for `ggplot2`

Q3. Repeat Q1 using `ggplot2`.

Q4. Repeat Q2 using `ggplot2`.

3.6 Summary

This is a big chapter that has discussed an array of plot functions both from base R and `ggplot2`. It is more like a cookbook of various visualization recipes. So, instead of listing all the functions here, it is helpful to list several useful resources that you can fine-tune the graphs yourselves.

pch

If you type `help(pch)` at the console, R shows the `pch` section of the help page for `points`. Below is a snapshot of `pch` values and the corresponding plotting characters:

Color

R supports a large number of colors and palettes. R Graph Gallery is a great resource to see what choices are available and their names.

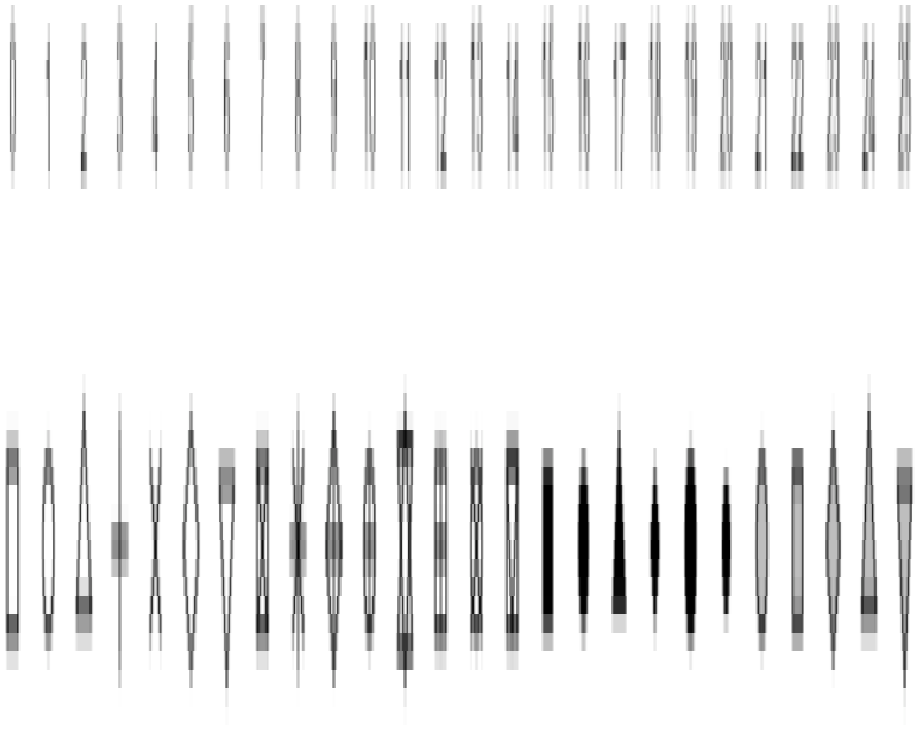


Figure 3.45: `pch` values

Part II

Part II

Part II covers topics beyond basic data manipulation functions and data visualization. It begins with Chapter 4, which discusses common statistical tests (a classical frequentist perspective). For readers who want to dive deeper into intermediate R functions, Chapter 5 covers R functions that create or import data sets from files, restructure data sets, and speed up execution. Chapter 6 provides an overview of RStudio, an Interactive Development Environment (IDE) for crafting R scripts.

4 Statistical Tests

R was inherently designed for statistical analysis. Therefore, it is justified to devote a chapter to statistical tests. Here, we will discuss a list of classic statistical tests with an emphasis on the Null Hypothesis Significance Test (NHST). We assume that you have a sound background in undergraduate-level statistics, i.e., you know the t statistic, χ^2 statistic, confidence interval, p-value, correlation, etc. For readers who need to refresh basic statistical concepts, there are many excellent introductory statistics textbooks freely available on the Internet[1] [2] [3]. Interested readers are recommended to consult them for further detail.

The goal of this chapter, as well as the whole book, emphasizes using R - the how. Additionally, the tests assume the availability of raw data, i.e., data sets contain individual samples. If only summary statistics (sample size, mean, standard deviation, etc.) are available, you have to use formulas to calculate the corresponding statistics, standard errors, critical values, and confidence intervals. These calculations are basically arithmetic operations and will not be discussed in this chapter.

Learning Objectives:

- Test a quantitative variable in one, two, or more groups.
- Test for one or two categorical variables.
- Linear regression

4.1 Housekeeping

We use two new data sets in this chapter: `FlightDelays`, and `Groceries` . Follow the steps below to import them into RStudio.

```
if (!require(resampledata)) {
  install.packages('resampledata')
}
```

[1] Introductory Statistics for Life and Biomedical Sciences: https://openintro.org/book/biostat/
[2] Introduction to Modern Statistics: https://openintro.org/book/ims/
[3] Handbook of Biological Statistics: https://www.biostathandbook.com/

```
Loading required package: resampledata


Attaching package: 'resampledata'

The following object is masked from 'package:datasets':

    Titanic
```

```
require(resampledata)
data("FlightDelays")
data("Groceries")
```

Take a glimpse of the data sets.

```
summary(FlightDelays)
```

```
       ID          Carrier       FlightNo       Destination    DepartTime
 Min.   :   1   AA:2906   Min.   :  71.0   BNA: 172    4-8am   : 699
 1st Qu.:1008   UA:1123   1st Qu.: 371.0   DEN: 264    8-Noon  :1053
 Median :2015             Median : 691.0   DFW: 918    Noon-4pm:1048
 Mean   :2015             Mean   : 827.1   IAD:  55    4-8pm   : 972
 3rd Qu.:3022             3rd Qu.: 787.0   MIA: 610    8-Mid   : 257
 Max.   :4029             Max.   :2255.0   ORD:1785
                                           STL: 225
  Day         Month        FlightLength       Delay         Delayed30
 Sun:551   May :1999   Min.   : 68.0   Min.   :-19.00   No :3432
 Mon:630   June:2030   1st Qu.:155.0   1st Qu.: -6.00   Yes: 597
 Tue:628               Median :163.0   Median : -3.00
 Wed:564               Mean   :185.3   Mean   : 11.74
 Thu:566               3rd Qu.:228.0   3rd Qu.:  5.00
 Fri:637               Max.   :295.0   Max.   :693.00
 Sat:453
```

It contains 10 variables, out of which two are quantitative variables (`FlightLength`, and `Delay`), and the rest are categorical variables. Notice that there is a glitch: R mistakenly considered `FlightNo` as a quantitative variable. However, it does not affect us, as we do not use this variable here. If you want to convert it to a categorical variable (factor), here is the step:

```
FlightDelays$FlightNo <- as.factor(FlightDelays$FlightNo)
```

The `Groceries` data set contains the product prices from Target and Walmart.

```
summary(Groceries)
```

```
                                                 Product          Size
 Annie's Macaroni & Cheese                           : 1   Length   :30
 Barilla Angel Hair Pasta                            : 1   N.unique :24
 Betty Crocker Super Moist Chocolate Fudge Cake Mix: 1   N.blank  : 0
 Campbell's Chicken Noodle Soup                      : 1   Min.nchar: 3
 Cheez-it Original Baked                             : 1   Max.nchar: 8
 Dinty Moore Hearty Meals Beef Stew                  : 1
 (Other)                                             :24
    Target          Walmart            Units          UnitType
 Min.   :0.990   Min.   :1.000   Length   :30   Length   :30
 1st Qu.:1.827   1st Qu.:1.760   N.unique :16   N.unique : 3
 Median :2.545   Median :2.340   N.blank  : 0   N.blank  : 0
 Mean   :2.762   Mean   :2.706   Min.nchar: 1   Min.nchar: 2
 3rd Qu.:3.140   3rd Qu.:2.955   Max.nchar: 2   Max.nchar: 5
 Max.   :7.990   Max.   :6.980
```

Lastly, we need to activate `tidyverse`.

```
suppressMessages(require(tidyverse))
```

4.2 Tests for Quantitative Variables

4.2.1 One-sample t-tests

The goal of a one-sample t-test is to test if the mean of a quantitative variable is equal to a certain value. For example, if the aviation regulatory agency states that acceptable flight delay is 12 minutes. We want to see if the flights meet the requirement. The null hypothesis (H_0) is that the mean delay is equal to 12 minutes. The alternative hypothesis (H_A) is that the mean delay is not equal to 12 minutes. Here are the steps to perform such a test:

```r
t.test(FlightDelays$Delay, mu = 12)
```

```
    One Sample t-test

data:  FlightDelays$Delay
t = -0.39963, df = 4028, p-value = 0.6895
alternative hypothesis: true mean is not equal to 12
95 percent confidence interval:
 10.45205 13.02375
sample estimates:
mean of x
  11.7379
```

As the p-value (0.6895) is greater than $\alpha = 0.05$ at a 95% confidence level, we fail to reject the null hypothesis H_0, meaning that the mean flight delay meets the standard.

4.2.2 One-sided t-tests

In the above flight delay example, the condition to reject the H_0 is that the delay time is either much longer or much shorter than 12 minutes (that is why the test is called two-sided). Indisputably, the latter case does not make much sense, as short delay times indicate the flights are punctual. Hence, the test must focus on long delay times by replacing the above two-sided test with a one-sided test in which the rejection region (H_A) lies at larger t-statistics (on the right). The parameter `alternative="greater"` suggests a one-sided test where the rejection region lies at the greater value.

```r
t.test(FlightDelays$Delay, mu = 12, alternative = "greater")
```

```
    One Sample t-test

data:  FlightDelays$Delay
t = -0.39963, df = 4028, p-value = 0.6553
alternative hypothesis: true mean is greater than 12
95 percent confidence interval:
 10.65885      Inf
sample estimates:
mean of x
  11.7379
```

The p-value 0.6553 is larger than $\alpha = 0.05$, fail to reject the H_0 at a 95% significance level.

4.2.3 Two-sample t-tests

A two-sample t-test tests whether the mean of one group is the same as the other group. The H_0 states the two groups have the same mean, and the H_A states otherwise.

For example, we want to compare the mean delays of the two carriers to determine whether they are the same.

Solution 1:

```
AA_delay <- subset(FlightDelays, Carrier=="AA")
UA_delay <- subset(FlightDelays, Carrier=="UA")
t.test(AA_delay$Delay, UA_delay$Delay)
```

```
    Welch Two Sample t-test

data:  AA_delay$Delay and UA_delay$Delay
t = -3.8255, df = 1843.8, p-value = 0.0001349
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -8.903198 -2.868194
sample estimates:
mean of x mean of y
 10.09738  15.98308
```

Based on the p-value (0.0001349) being less than $\alpha = 0.05$, the delays of the two carriers are different. The bottom of the test results shows that the mean delay for AA (group x) is 10.09738, shorter than UA's 15.98308.

Solution 2:

This approach makes use of the categorical variable `carrier` to split the data set into two, avoiding the creation of two temporary data frames.

```
t.test(Delay ~ Carrier, data=FlightDelays)
```

```
    Welch Two Sample t-test
```

```
data:  Delay by Carrier
t = -3.8255, df = 1843.8, p-value = 0.0001349
alternative hypothesis: true difference in means between group AA and group UA is not equal t
95 percent confidence interval:
 -8.903198 -2.868194
sample estimates:
mean in group AA mean in group UA
        10.09738         15.98308
```

The symbol `~` means "split by". `Delay ~ Carrier` means to split `Delay` by `Carrier`.

4.2.4 One-way ANOVA

A one-way ANOVA tests whether the means of three or more groups are the same. Two key differences from the two-sample t-test above: 1. a two-sample t-test can handle only two groups, but ANOVA can handle two or more groups. 2. The H_0 of ANOVA is rejected if any pair(s) of groups have unequal means.

For example, test whether mean delays vary by destination.

```
aov_results <- aov(Delay ~ Destination, data=FlightDelays)
summary(aov_results)
```

```
              Df  Sum Sq Mean Sq F value   Pr(>F)
Destination    6   62891   10482   6.094 2.29e-06 ***
Residuals   4022 6918028    1720
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

If you only care whether the H_0 can be rejected, focus on the overall p-value (2.29e-06) under the `Pr(>F)` column on the right. It is undeniably smaller than $\alpha = 0.05$, so the H_0 can be rejected.

In some situations, you may be interested in finding out which pair(s) do not have the same mean. Tukey's Honest Significant Difference (HSD) test can answer this question.

```
TukeyHSD(aov_results)
```

```
  Tukey multiple comparisons of means
    95% family-wise confidence level
```

```
Fit: aov(formula = Delay ~ Destination, data = FlightDelays)

$Destination
               diff         lwr        upr     p adj
DEN-BNA   7.1454369  -4.8424432 19.1333171 0.5765602
DFW-BNA   3.2646932  -6.8999706 13.4293570 0.9647119
IAD-BNA  19.4461945   0.4951700 38.3972191 0.0399134
MIA-BNA  12.9137057   2.3518660 23.4755453 0.0058064
ORD-BNA   8.4338186  -1.3335463 18.2011835 0.1430105
STL-BNA   0.9181137 -11.4728506 13.3090780 0.9999910
DFW-DEN  -3.8807437 -12.4245377  4.6630503 0.8331281
IAD-DEN  12.3007576  -5.8325642 30.4340793 0.4140643
MIA-DEN   5.7682688  -3.2444158 14.7809533 0.4884472
ORD-DEN   1.2883817  -6.7786774  9.3554408 0.9991892
STL-DEN  -6.2273232 -17.3274143  4.8727678 0.6463375
IAD-DFW  16.1815013  -0.8016809 33.1646835 0.0736782
MIA-DFW   9.6490125   3.2584254 16.0395995 0.0001755
ORD-DFW   5.1691254   0.2003667 10.1378841 0.0352035
STL-DFW  -2.3465795 -11.4473015  6.7541425 0.9885164
MIA-IAD  -6.5324888 -23.7563253 10.6913477 0.9225580
ORD-IAD -11.0123759 -27.7607938  5.7360421 0.4540417
STL-IAD -18.5280808 -36.9303655 -0.1257961 0.0471723
ORD-MIA  -4.4798870 -10.2175372  1.2577631 0.2425378
STL-MIA -11.9955920 -21.5378772 -2.4533067 0.0039784
STL-ORD  -7.5157049 -16.1704244  1.1390145 0.1382159
```

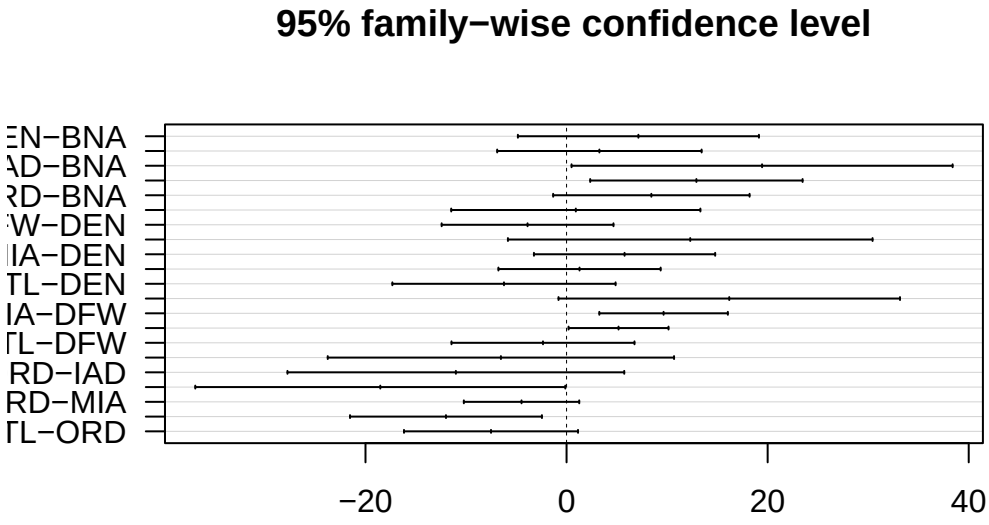
It lists all possible pairs of destinations. Since there are seven destinations, the total number of pairs is 21 (`choose(7,2)`).

Each row shows the difference of means (`diff`), the lower (`lwr`) and upper (`upr`) bounds of the confidence interval, and the adjusted p-value (`p adj`). Examining the p-value is the quickest way to identify pairs with unequal means. E.g., IAD-BNA in which the p-value is less than 0.05.

To ease analysis, it is suggested to plot the HSD result as below:

```r
hsd_results <- TukeyHSD(aov_results)

# las=1 causes the plot to print the pair name horizontally.
plot(hsd_results, las=1)
```



Confidence intervals not crossing zero (the vertical line) indicate pairs have unequal means.

4.2.5 Paired t-tests

A paired t-test evaluates whether the mean difference between two measurements taken on the same subject significantly differs from zero. For a drug trial example, a subject's blood pressure is measured before and after taking the tested drug. Each sample is a subject. The measurements are blood pressure. The H_0 is that the mean difference between the two measurements is zero.

Here we will examine whether the product prices between Target and Walmart are different. The subject is a product. The two measurements are the prices of the same product from Target and Walmart.

```
with(Groceries, t.test(Target, Walmart, paired=T))
```

```
    Paired t-test

data:  Target and Walmart
t = 0.47046, df = 29, p-value = 0.6415
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -0.1896825  0.3030159
sample estimates:
```

```
mean difference
     0.05666667
```

The p-value of $0.6415 > \alpha = 0.05$ suggests that there is no statistical difference between the two stores at the 95% confidence level. The mean price difference of all the products in the sample is 0.057, which is not considered a significant difference from zero.

4.3 Tests for Categorical Variables

4.3.1 One Categorical Variable

Here, we want to check whether the proportions of categories match a specified expected ratio. In standard textbooks, this test is called the **Chi-square (χ^2) goodness-of-fit** test. For example, `Delayed30` indicates whether the flight was delayed by more than 30 minutes; it is either Yes or No. Suppose the regulatory standard recommends not more than 10% of a random sample of flights. First, figure out how many Yes and No in the sample.

```
table(FlightDelays$Delayed30)
```

```
  No  Yes
3432  597
```

The data set contains 4,029 flights; 3,432 were punctual, and 587 were delayed. According to the standard, the number of delayed flights should be no more than 10% of the total, or 402. Due to random variation in sampling, we cannot conclude an issue is present simply because 587 is greater than 402. Here's where the χ^2-test comes in.

```
chisq.test(table(FlightDelays$Delayed30), p=c(0.9,0.1))
```

```
    Chi-squared test for given probabilities

data:  table(FlightDelays$Delayed30)
X-squared = 103.9, df = 1, p-value < 2.2e-16
```

The H_0 assumes the proportions agree with the expected ratio. The p-value shown above is literally zero, indicating that the proportions of Yes and No do not conform to the expected 9:1 ratio at the 95% confidence level.

One remark about setting up the proportion parameter `p=c(0.9,0.1)`. Make sure that the order of the proportions matches the sequence of the categories produced by the `table()` function. Additionally, the proportions must sum to 1.

4.3.2 Two Categorical Variables

Here we examine the independence between two categorical variables. The test is colloquially called χ^2 **Test for Independence**. For example, we want to know whether a delay of more than 30 minutes depends on the carrier, that is to say whether `Delayed30` is independent of `Carrier`. The H_0 assumes that they are independent.

```
with(FlightDelays, chisq.test(Carrier, Delayed30, correct=TRUE))
```

```
    Pearson's Chi-squared test with Yates' continuity correction

data:  Carrier and Delayed30
X-squared = 13.462, df = 1, p-value = 0.0002434
```

Note that `correct=TRUE` instructs the test function to make adjustments in calculating the χ^2 statistic. The correction is only applicable for a 2x2 contingency table with small counts. However, it does little harm if you turn it on for tables with a different dimension or large counts[4]. My suggestion is to turn it on all the time.

As the p-value of 0.0002434 is less than 0.05, reject the H_0 at the 95% confidence level, meaning that `Delay30` and `Carrier` are not independent. This conclusion suggests that the proportion of delayed flights differs between the two carriers. To be clear, their dependency says nothing about the carriers' delay issue. To figure that out exactly, we should examine the proportion of delayed flights for each carrier using a new function `prop.table()` as below:

```
with(FlightDelays, prop.table(table(Carrier, Delayed30), margin=1))
```

```
       Delayed30
Carrier        No       Yes
     AA 0.8647626 0.1352374
     UA 0.8183437 0.1816563
```

[4] https://en.wikipedia.org/wiki/Yates%27s_correction_for_continuity

We feed the contingency table of `Carrier` by `Delayed30` to the `prop.table()` function with `margin=1`, meaning that values are divided by the row totals or the sum of the proportions for each row is 1. As you can see, both carriers have an issue under the 10% rule, but the issue with UA is more severe than AA.

4.4 Regression

4.4.1 Simple Linear Regression

A simple linear regression (SLR) model involves a predictor X (independent variable) and a response Y (dependent variable), where both of them are quantitative, but they have different roles.

$$Y = \beta_0 + \beta_1 X$$

where β_0 is the intercept, and β_1 is the slope.

The relationship between them in an SLR is linear and directional. Firstly, the predictor is believed to influence the response, and not the reverse; therefore, it is directional. For example, temperature (predictor) is supposed to influence ice cream sales (response), and not the other way.

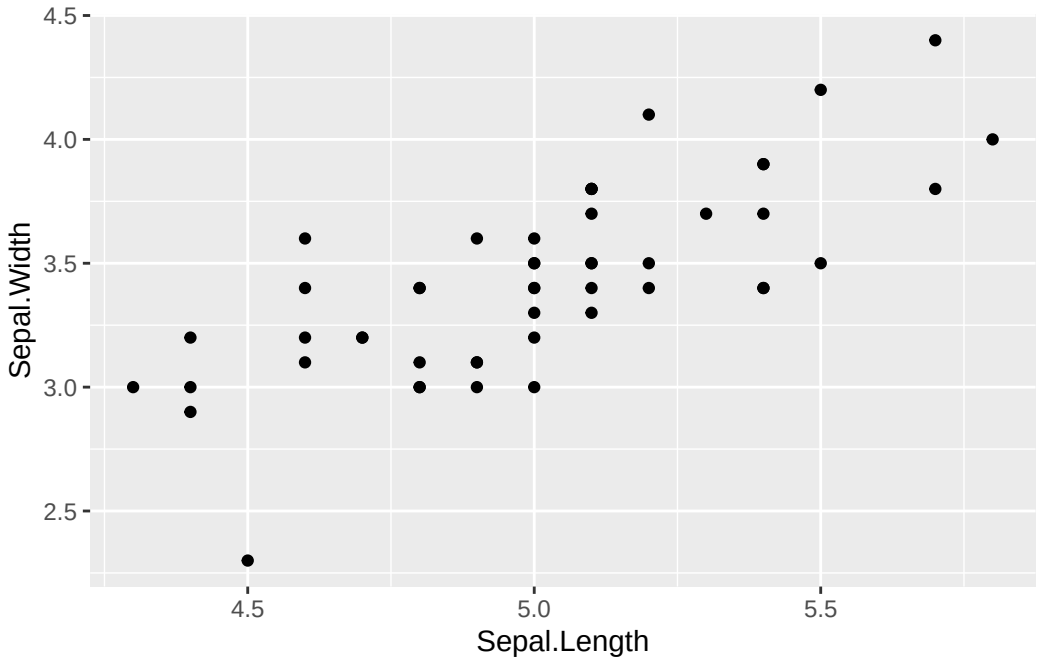
Secondly, the relationship is linear, meaning that a unit increment of X will cause Y to increase or decrease by β_1 units for every X within the range of X in the data.

Thirdly, an SLR can only establish a correlation between X and Y, but not causation.

Finally, in the context of NHST, the H_0 is $\beta_1 = 0$.

Here we will switch to the familiar built-in `iris` data set to demonstrate how to build an SLR. Suppose we are interested in whether `Sep.Length` (predictor) affects `Sep.Width` (response) for `setosa` species. The first step for building an SLR is to visualize the relationship between the two variables using a scatter plot (Section 3.2.5.1 or Section 3.4.6). Do not downplay the intuition of visual inspection. In the absence of an upward or downward trend, the attempt to build an SLR is deemed fruitless. On a positive note, the plot may still reveal an intriguing trend not as expected, requiring a different kind of model or variable transformation. Anyway, let's plot the two variables.

```
setosa_iris <- subset(iris, Species=='setosa')
ggplot(data=setosa_iris, aes(x=Sepal.Length, y=Sepal.Width)) +
  geom_point()
```



As shown above, there is an upward trend even though it is not definitive for everyone. Subjectively, we will proceed to build an SLR model.

```
slr_model <- lm(Sepal.Width ~ Sepal.Length, data=setosa_iris)
summary(slr_model)
```

```
Call:
lm(formula = Sepal.Width ~ Sepal.Length, data = setosa_iris)

Residuals:
     Min       1Q   Median       3Q      Max
-0.72394 -0.18273 -0.00306  0.15738  0.51709

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)   -0.5694     0.5217  -1.091    0.281
Sepal.Length   0.7985     0.1040   7.681 6.71e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2565 on 48 degrees of freedom
Multiple R-squared:  0.5514,    Adjusted R-squared:  0.542
F-statistic: 58.99 on 1 and 48 DF,  p-value: 6.71e-10
```

The slope and the intercept can be found under the `Estimate` column of `Coefficients` subsection. In this example, the intercept β_0 is -0.5694, and the slope β_1 (`Sepal.Length`) is 0.7985. So the SLR model is:

$$Sepal.Width = -0.5694 + 0.7985 \times Sepal.Length$$

The SLR model says that a unit increase in `Sepal.Length`, `Sepal.Width` will increase by 0.7985 units.

To judge whether they are statistically significant, examine the p-values under the column `Pr(>|t|)`. The p-value of the slope is close to zero, suggesting that it is statistically significant; unfortunately, the intercept is not. It means that the linear correlation between `Sepal.Width` and `Sepal.Length` is strong. However, it is completely accurate to predict `Sepal.Width` using `Sepal.Length`.

Since we mentioned correlation, the `Multiple R-squared` is the square of Pearson's correlation between the two variables. Let's verify this.

```
# Pearson's correlation
r <- with(setosa_iris, cor(Sepal.Length, Sepal.Width))

print(paste("Pearson's correlation =",r))
```

```
[1] "Pearson's correlation = 0.74254668566516"
```

```
# Square of Pearson's correlation is Multiple R-squared
print(paste("The square of r =",r**2))
```

```
[1] "The square of r = 0.551375580392314"
```

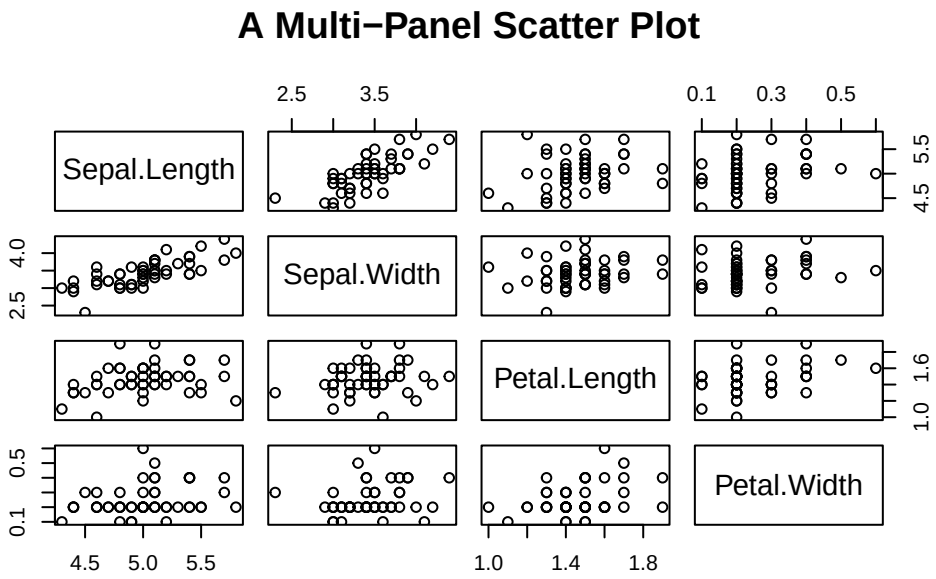
4.4.2 Multiple Linear Regression

A multiple linear regression (MLR) model involves one response and two or more predictors. The equation for MLR is

$$Y = \beta_0 + \beta_1 \times X_1 + ... + \beta_n \times X_n$$

Suppose we are keen to predict `Sepal.Width` by `Sepal.Length` and `Petal.Length` for setosa iris. Similar to SLR, we visualize the data but using a multipanel scatter plot Section 3.2.6, Section 3.4.7).

```r
pairs(setosa_iris[,1:4], main="A Multi-Panel Scatter Plot")
```



Here we visualize the trend between `Sepal.Width` and `Sepal.Length` (2nd row and 1st column), `Sepal.Width` and `Petal.Length` (2nd row and 3rd column), and `Sepal.Width` and `Petal.Width` (2nd row and 4th column). Despite the trend not being clear between `Sepal.Width` and `Petal.Length`, and `Sepal.Width` and `Petal.Width`, let's proceed to build an MLR model.

```r
mlr_model <- lm(Sepal.Width ~ Sepal.Length + Petal.Length + Petal.Width, data=setosa_iris)
summary(mlr_model)
```

```
Call:
lm(formula = Sepal.Width ~ Sepal.Length + Petal.Length + Petal.Width,
    data = setosa_iris)

Residuals:
     Min       1Q   Median       3Q      Max
-0.74373 -0.17838  0.00164  0.16876  0.53988

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  -0.48913    0.57348  -0.853    0.398
Sepal.Length  0.79669    0.11247   7.083 6.83e-09 ***
Petal.Length -0.07136    0.23244  -0.307    0.760
```

```
Petal.Width   0.13513    0.38427   0.352    0.727
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2616 on 46 degrees of freedom
Multiple R-squared:  0.553, Adjusted R-squared:  0.5239
F-statistic: 18.97 on 3 and 46 DF,  p-value: 3.759e-08
```

Based on the p-values, only `Sepal.Length` is a stable predictor of `Sepal.Width`; `Petal.Length` and `Petal.Width` do not make a statistically significant contribution to the model. This example is simple, as there are not many predictors to be considered. When the number of predictors rises, the number of choices explodes exponentially. For a four-predictor example, the number of combinations is:

$$\sum_{i=1}^{n=4} \binom{n}{i} = \binom{4}{1} + \binom{4}{2} + \binom{4}{3} + \binom{4}{4} = 1 + 6 + 6 + 1 = 14$$

R has a function to assist in finding a "not too bad" MLR model, meaning it does not guarantee to return the best model. Here is how it works. It begins with a one-predictor model; start with any predictor. Then it tries to build an intermediate model by incorporating an additional predictor from the pool that returns the least error. This step will continue until all predictors are chosen.

```
baseline_mlr <- lm(Sepal.Width ~ Sepal.Length, data=setosa_iris)
add1(baseline_mlr, scope = ~ Sepal.Length + Petal.Length + Petal.Width, test = "F")
```

```
Single term additions

Model:
Sepal.Width ~ Sepal.Length
             Df Sum of Sq    RSS     AIC F value Pr(>F)
<none>                    3.1587 -134.09
Petal.Length  1 0.0032458 3.1554 -132.15  0.0483 0.8269
Petal.Width   1 0.0052586 3.1534 -132.18  0.0784 0.7807
```

The model error is reflected in the Akaike Information Criterion (AIC) values. The intuition is that more predictors will naturally fit the data better. However, the model inevitably fits the noise as well. As a result, although the model looks great in development, it eventually fails to handle unseen data, i.e., overfitting. AIC serves as a canary. As a rule of thumb, if the difference in AIC between consecutive intermediate models is less than 2, there is no significant difference.

4.5 Summary

In this chapter, we have discussed classical statistical tests in the framework of Null Hypothesis Significance Test (NHST). A simple guideline for choosing the most appropriate test is to identify the main variable(s), how many there are, and their type(s). The table below summarizes the tests discussed in this chapter:

Name	Description	Example	Reference
One-sample t-test	Test whether the mean of a quantitative variable equals a specified value.	`t.test(FlightDelays$Delay, mu = 12)`	Section 4.2.1
Two-sample t-test	Test whether or not the means of two groups are equal.	`t.test(Delay ~ Carrier, data=FlightDelays)`	Section 4.2.3
One-way ANOVA	Test whether or not the means of three or more groups are equal.	`aov_results <- aov(Delay ~ Destination, data=FlightDelays)`	Section 4.2.4
Paired t-test	Test whether the mean difference between two measurements taken on the same subject significantly differs from zero	`with(Groceries, t.test(Target, Walmart, paired=T))`	Section 4.2.5
χ^2 goodness-of-fit	Test whether the proportions of categories match a specified expected ratio.	`chisq.test(table(FlightDelays$Delayed30), p=c(0.9,0.1))`	Section 4.3.1
χ^2 test for independence	Test the independence between two categorical variables	`with(FlightDelays, chisq.test(Carrier, Delayed30, correct=TRUE))`	Section 4.3.2
Simple Linear Regression	It examines the linear relationship between the predictor and the response.	`lm(Sepal.Width ~ Sepal.Length, data=setosa_iris)`	Section 4.4.1

Name	Description	Example	Reference
Multiple Linear Regression	It examines the linear relationship between multiple predictors and the response.	`lm(Sepal.Width ~ Sepal.Length + Petal.Length, data=setosa_iris)`	Section 4.4.2

4.6 Exercises

Use the built-in data set `mtcars` to answer these questions. Before tackling the questions below, you need to convert the categorical variables in `mtcars` to a factor. Here is the code:

```
mtcars$cyl <- factor(mtcars$cyl)
mtcars$vs <- factor(mtcars$vs, levels = c(0,1))
mtcars$am <- factor(mtcars$am, levels = c(0,1))
```

Q1. Test whether the mean `mpg` is 20 at the 95% confidence level.

Q2. `am` stands for automatic transmission. A 0 means not automatic and 1 means automatic. Test whether the mean acceleration `qsec` is the same between automatic and manual transmissions.

Q3. Test whether the mean acceleration `qsec` is the same for different numbers of cylinders `cyl`. If it is not true, identify the pair(s) that have different mean acceleration. Note that `cyl` must be converted to a factor before performing any tests.

Q4. Test whether cars with different numbers of cylinders are equally sampled in the data set.

Q5. Test whether the type of transmission `am` and the number of cylinders are independent. Note: If you encounter a warning message, it is due to small-count data. You can ignore it.

Q6. Use a linear model to test the relationship between horsepower `hp` and acceleration `qsec`.

Q7. Use a multiple linear model to study the relationship between acceleration, horsepower, weight, and MPG.

5 Dive Deeper

So far, we have discussed the basics of R that are sufficient for most situations. In this chapter, we will discuss several practical operations that require a more detailed discussion. That includes how to create summary tables from published work, create a small data set in a data frame, load a data set of any size into RStudio, define your own function, and leverage the family of `apply()` functions.

Learning Objectives:

- Recreate a summary table from published work.
- Create a small data frame with data.
- Load a data set into RStudio.
- Define your own functions.
- Harness the family of `apply()` functions.

5.1 Recreate a Summary Table in R

You see the data summary tables as those in @fig-5-1, so you want to recreate and analyze them yourself. The `table()` or `group_by()` functions mentioned in the previous chapters cannot help in this situation, as they require the raw data, like `iris`, where the information of each sample is accessible.

Here you will create the two tables in R as shown in Figure 5.1. Let us begin with the first table, which is a contingency table with two rows and five columns. As all the values in the table are numeric, each row is represented by a numeric vector. Here, we will use the `c()` function to create it. See Section 1.2.3. But the vector designated for the first row here is slightly different because we want to specify the column names and the values together (a **named vector**); as a result, the columns in the resultant table will be named.

```
row1 <- c(heart_disease=0, formerly=808, never=1802, smoke=728, unknown=1496)
row2 <- c(1, 77, 90, 61, 48)
```

A

Heart Disease	Smoking Status			
	Formerly	Never	Smoke	Unknown
0	808	1802	728	1496
1	77	90	61	48

B

Group	Daily Tobacco Consumption	Controls	Cases
A	0-9g/day	90	14
B	10-19	44	13
C	20-29	25	8
D	30+	8	11

Figure 5.1: Recreate summary tables

Note that the order of the values in the named vector `row1` determines the order of the columns in the table. Ensure that the same order is maintained for subsequent rows.

When all the rows have been defined, they will be stacked up into a table using the `rbind()` function as shown below.

```
my_tab1 <- rbind(row1, row2)
my_tab1
```

```
     heart_disease formerly never smoke unknown
row1             0      808  1802   728    1496
row2             1       77    90    61      48
```

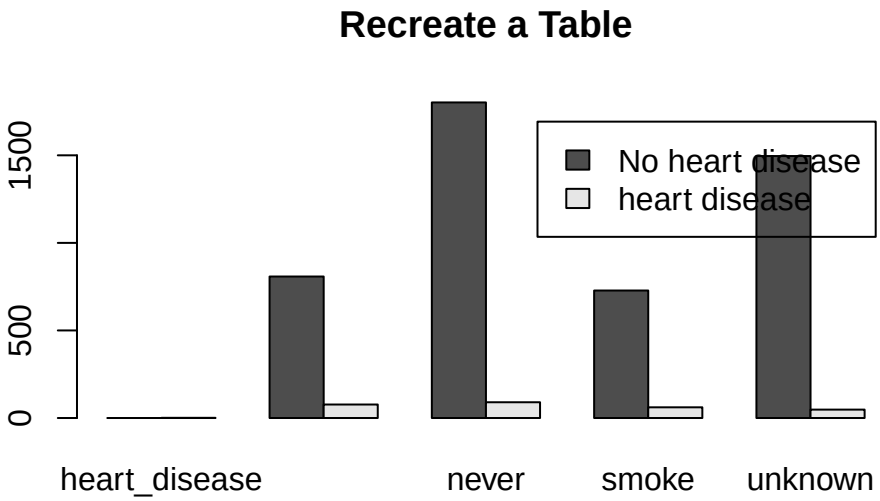
Note that names `row1` and `row2` on the left are the names of the rows (`rownames`), they do not constitute a column of the table. Hence, it is a 2x5 table as shown by the dimension function `dim()` below.

```
dim(my_tab1)
```

```
[1] 2 5
```

Now, you can plot the table Section 3.2.4.

```
barplot(my_tab1, main="Recreate a Table", beside=T,
        legend.text = c("No heart disease","heart disease"))
```



Or, perform a χ^2 test of independence.

```
chisq.test(my_tab1, correct=F)
```

```
Warning in chisq.test(my_tab1, correct = F): Chi-squared approximation may be
incorrect
```

```
    Pearson's Chi-squared test

data:  my_tab1
X-squared = 61.967, df = 4, p-value = 1.119e-12
```

5.1.1 List Objects

The second table in Figure 5.1 is different from the first one as each row contains a mixture of character (`daily tobacco onsumption`) and numeric values (`controls` and `cases`), making vectors unsuitable for this situation. R provides an object similar to vectors, ordered sequences, that is the `list` object. The main difference between a vector and a list is that the latter allows values to have different data types, i.e., **heterogeneous**. Below is an example to define rows as list objects:

```
A <- list(group="A", daily_toba="0-9", ncontrols=90, ncases=14)
B <- list("B", "10-19", 44, 13)
C <- list("C", "20-29", 25, 8)
D <- list("D", "30+", 8, 11)
```

Similarly, the row (`A`) defines the values and column names of the resultant table. Now, rows are stacked up together to form a 4x4 table.

```
my_tab2 <- rbind(A,B,C,D)
my_tab2
```

```
  group daily_toba ncontrols ncases
A "A"   "0-9"      90        14
B "B"   "10-19"    44        13
C "C"   "20-29"    25        8
D "D"   "30+"      8         11
```

```
print(dim(my_tab2))
```

```
[1] 4 4
```

5.2 Create a small data frame with data in R

Here we will discuss two approaches to store a small data set in a `data.frame` object. If the data set is large, my recommendation is to store the data in a spreadsheet, e.g., Google Sheets or Microsoft Excel, and then upload it into RStudio. See the next section for more details.

5.2.1 Use `data.frame()` Function

Here is the first approach to create a data frame that takes two steps. The first step is to define each column (variable) as a vector, and then use the `data.frame()` function to combine them into a data frame. Here is an example to create a data frame with two columns: `group`, and systolic blood pressure (`sbp`), and eight rows. Two vectors, each with eight values, are created below:

```
group_data <- c("A","A","A","A","A","B","B","B")
sbp_data <- c(120,135,133,128,130,150,160,158)
```

Next, use the `data.frame()` function to combine the vectors into a data frame.

```
small_df1 <- data.frame(group = group_data,
                        sbp = sbp_data)
small_df1
```

```
  group sbp
1     A 120
2     A 135
3     A 133
4     A 128
5     A 130
6     B 150
7     B 160
8     B 158
```

5.2.2 Use `read.table()` Function

The second approach is to type out the values in columns as in the example below, and feed it to the `read.table()` function through the `text=` parameter.

```
content = ("
group sbp
A 120
A 135
A 133
A 128
A 130
B 150
B 160
B 158
")

small_df2 <- read.table(text = content, header = TRUE)
small_df2
```

```
  group sbp
1     A 120
2     A 135
3     A 133
4     A 128
5     A 130
6     B 150
7     B 160
8     B 158
```

There is no good or bad of either approach unless you want to use some R functions to generate the column vectors. In such a case, the first approach using `data.frame()` function is more trackable.

5.3 Load Data Sets into RStudio

Data sets are usually too large to be typed using the methods discussed in the previous section. Here, you will learn several ways to load a data set stored in a file into RStudio. The two common data file formats are: comma-separated (`.csv`) or tab-separated (`.txt` or `.tsv`). In a `.csv` file, the comma (`,`) serves as a column separator, separating data in one column from the other column, as shown in panel A of Figure 5.2. The other common separator is the `tab` value, which is usually invisible but is deliberately made visible in panel B.

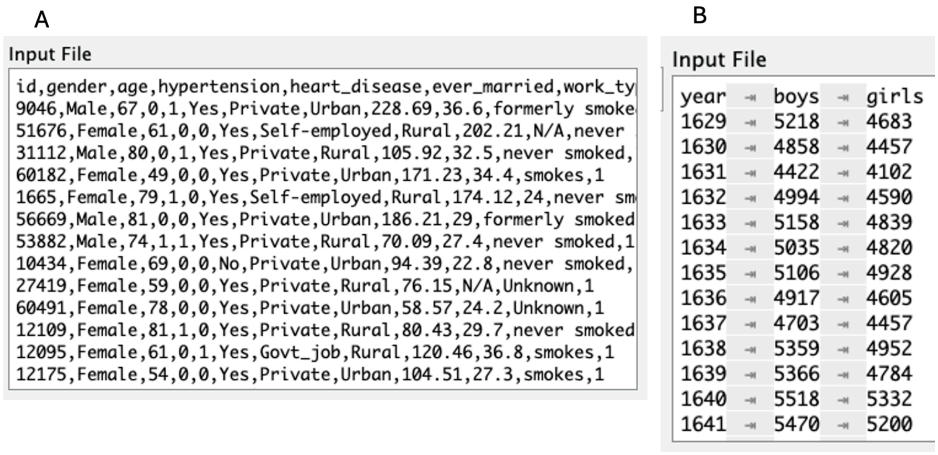


Figure 5.2: File formats

5.3.1 `read.csv()` and `read.table()` Functions

R provides two functions to read the data file directly into RStudio. `read.csv()` reads a `.csv` file from the location provided. The location can be a URL or a file path to a file on your computer. For example, load the flight delays file from here: https://www.openintro.org/data/csv/airline_delay.csv. The first parameter is the file location. Make sure that it is in quotes.

```
delays1 <- read.csv("https://www.openintro.org/data/csv/airline_delay.csv", stringsAsFactors
head(delays1)
```

```
  year month carrier      carrier_name airport
1 2020    12      9E Endeavor Air Inc.     ABE
2 2020    12      9E Endeavor Air Inc.     ABY
3 2020    12      9E Endeavor Air Inc.     AEX
4 2020    12      9E Endeavor Air Inc.     AGS
5 2020    12      9E Endeavor Air Inc.     ALB
6 2020    12      9E Endeavor Air Inc.     ATL
                                                 airport_name arr_flights
1 Allentown/Bethlehem/Easton, PA: Lehigh Valley International          44
2                     Albany, GA: Southwest Georgia Regional          90
3                   Alexandria, LA: Alexandria International          88
4                Augusta, GA: Augusta Regional at Bush Field         184
5                           Albany, NY: Albany International          76
6       Atlanta, GA: Hartsfield-Jackson Atlanta International        5985
  arr_del15 carrier_ct weather_ct nas_ct security_ct late_aircraft_ct
1         3       1.63       0.00   0.12           0             1.25
2         1       0.96       0.00   0.04           0             0.00
3         8       5.75       0.00   1.60           0             0.65
4         9       4.17       0.00   1.83           0             3.00
5        11       4.78       0.00   5.22           0             1.00
6       445     142.89      11.96 161.37           1           127.79
  arr_cancelled arr_diverted arr_delay carrier_delay weather_delay nas_delay
1             0            1        89            56             0         3
2             0            0        23            22             0         1
3             0            1       338           265             0        45
4             0            0       508           192             0        92
5             1            0       692           398             0       178
6             5            0     30756         16390          1509      5060
  security_delay late_aircraft_delay
1              0                  30
2              0                   0
3              0                  28
4              0                 224
5              0                 116
6             16                7781
```

The `stringAsFactors=T` tells the function to treat columns with text values as a categorical variable (factor), which is common in statistical analysis.

If the separator is a `tab` or something else, `read.table()` allows us to specify the actual separator. For example, another version of the same flight delays file is in tab-separated format. So, `tab` is used as the column separator of flight delays.

```
delays2 <- read.table("https://www.openintro.org/data/tab-delimited/airline_delay.txt",
                      header = T,
                      sep = "\t",
                      stringsAsFactors = T)
```

```
Warning in scan(file = file, what = what, sep = sep, quote = quote, dec = dec,
: EOF within quoted string
```

```
Warning in scan(file = file, what = what, sep = sep, quote = quote, dec = dec,
: number of items read is not a multiple of the number of columns
```

```
head(delays2)
```

```
  year month carrier      carrier_name airport
1 2020    12      9E Endeavor Air Inc.     ABE
2 2020    12      9E Endeavor Air Inc.     ABY
3 2020    12      9E Endeavor Air Inc.     AEX
4 2020    12      9E Endeavor Air Inc.     AGS
5 2020    12      9E Endeavor Air Inc.     ALB
6 2020    12      9E Endeavor Air Inc.     ATL
                                                 airport_name arr_flights
1 Allentown/Bethlehem/Easton, PA: Lehigh Valley International          44
2                     Albany, GA: Southwest Georgia Regional          90
3                   Alexandria, LA: Alexandria International          88
4                Augusta, GA: Augusta Regional at Bush Field         184
5                           Albany, NY: Albany International          76
6       Atlanta, GA: Hartsfield-Jackson Atlanta International        5985
  arr_del15 carrier_ct weather_ct nas_ct security_ct late_aircraft_ct
1         3       1.63       0.00   0.12           0             1.25
2         1       0.96       0.00   0.04           0             0.00
3         8       5.75       0.00   1.60           0             0.65
4         9       4.17       0.00   1.83           0             3.00
5        11       4.78       0.00   5.22           0             1.00
6       445     142.89      11.96 161.37           1           127.79
  arr_cancelled arr_diverted arr_delay carrier_delay weather_delay nas_delay
1             0            1        89            56             0         3
2             0            0        23            22             0         1
3             0            1       338           265             0        45
4             0            0       508           192             0        92
5             1            0       692           398             0       178
```

```
6              5             0      30756          16390          1509       5060
  security_delay late_aircraft_delay
1              0                  30
2              0                   0
3              0                  28
4              0                 224
5              0                 116
6             16                7781
```

The `tab` on the keyboard is denoted by `\t`, no space between the two characters. Additionally, not all data files contain a column heading in the first row. By default, R does not assume one exists. To override the default, like the above example, specify `header=T`.

5.3.2 Import data in RStudio

Suppose the data file has been downloaded on your computer in the `Downloads` subfolder. The first step is to upload it to the RStudio Cloud Drive via **Files->Upload** inside the Files panel, usually on the bottom right.

Next, import the data file into RStudio via **Environment -> Import Dataset -> From Text (base)** as shown below:

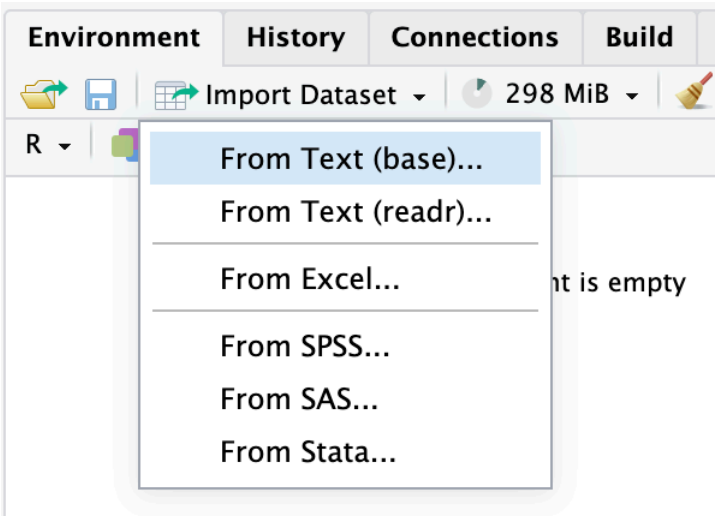


Figure 5.3: Import Data Set

Select the file to be imported as shown in Figure 5-4 below:

A window will pop up as shown in Figure 5-4, which is a preview of the data. Do the following:

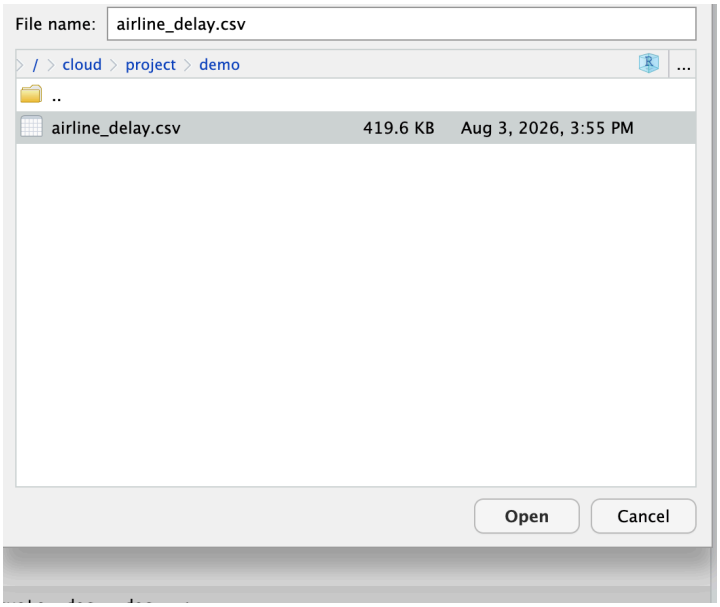


Figure 5.4: Select file

- Check `heading is Yes` box.
- Check `StringAsFactors` box.
- Click the `Import` button at the bottom.

This imports the data file into RStudio and reflects it in the **Environment** panel on the top right.

5.4 Define Functions

When no built-in functions do what you want, it is the occasion to write your own functions. A function consists of input, process, and output. A daily analogy is the vending machine. The inputs are money and choice; the process is the mechanical system to fetch the selected beverage, snack, etc., and the output is the dispensed item.

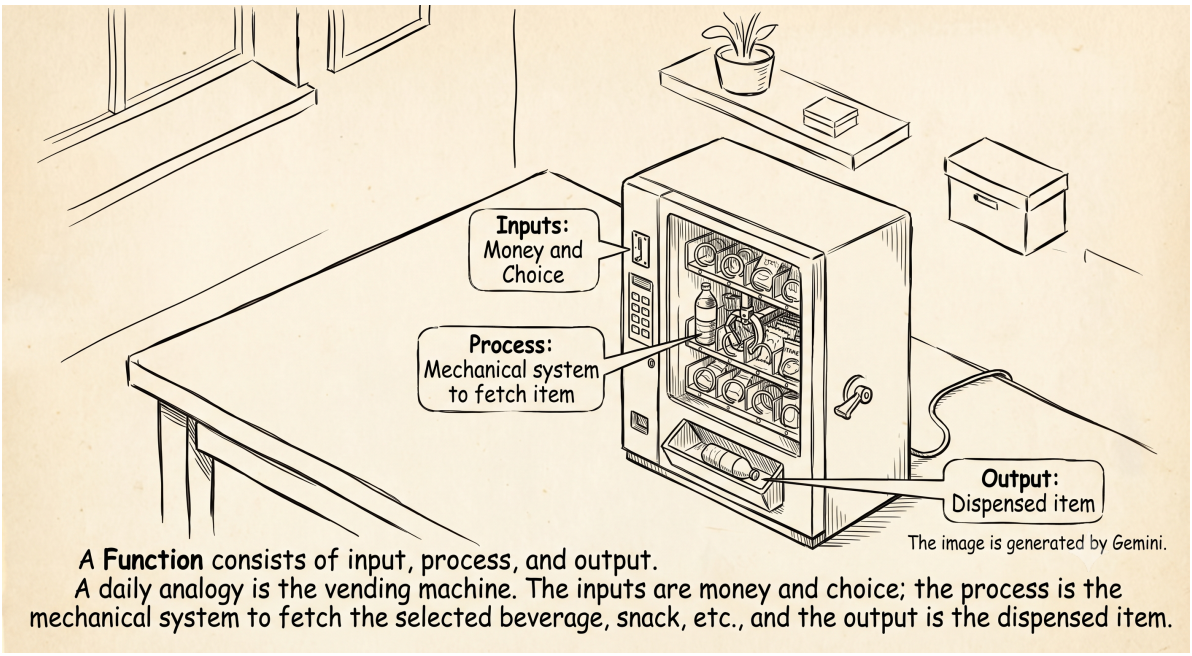


Figure 5.5: Illustration 5.1 Function.

Here are the four components required to define a function: a function name, argument(s), the body, and the output(s).

- Function name: same as the rules in defining variables.
- Argument(s): A function may have zero, 1, 2+ arguments or inputs.
- Body: R code that takes the input argument(s), if any, performs the task, and returns the output.
- Output(s): the value(s), object(s), or none returned to the caller.

Suppose we want to convert temperature from Fahrenheit to Celsius, $C = \frac{5}{9} \times (F - 32)$, with specified precision; the default is zero. Here is the function definition:

```
F2C <- function(f, decimals = 0) {
  return(round((f-32)*5/9, decimals))
}
```

The function name is `F2C`. The function takes two arguments: `f` that denotes the temperature in Fahrenheit, and the number of decimals, with the default zero. Note that arguments are not required to have default values. The body consists of one line of code:

`return(round((f-32)*5/9, decimals))`. The output is the result of the arithmetic expression.

Let's test it out.

```
temp1 <- F2C(100)
print(paste("100 F is equivalent to",temp1, "celsius"))
```

```
[1] "100 F is equivalent to 38 celsius"
```

```
temp2 <- F2C(-10, decimals = 2)
print(paste("-10 F is equivalent to",temp2, "celsius"))
```

```
[1] "-10 F is equivalent to -23.33 celsius"
```

5.5 The Apply-Like Family

The `apply()` family has seven members. Here we will focus on three members that will do 80% of the job: `apply()`, `sapply()`, and `lappy()`.

5.5.1 Mechanic of Apply-Like Functions

Generally, apply-like functions execute a function over every value of an R composite object Section 1.2.2. While `apply()` evaluates along the rows or columns of 2D objects like matrices and data frames, functions like `lapply()` or `sapply()` iterate over elements in 1D objects like vectors and lists. The function that acts on each value of the targeted object can be an R built-in function or a user-defined function. More details about the apply-like functions will be discussed below.

5.5.2 `lappy()`

It executes a function over every value of a `list` or a `vector`, not 2D objects, and returns a `list`. For example, change the 50 state names specified in the built-in list `state.name` to uppercase.

```
state.name.upper <- lapply(state.name, FUN=toupper)
print(state.name.upper[1:5])
```

```
[[1]]
[1] "ALABAMA"

[[2]]
[1] "ALASKA"

[[3]]
[1] "ARIZONA"

[[4]]
[1] "ARKANSAS"

[[5]]
[1] "CALIFORNIA"
```

You may notice the output looks strange, despite the conversion being correct. Please hold on to the question until the next section.

A note about the syntax. `toupper()` is a built-in function that changes the input text to uppercase. When we provide the function to any apply-like functions, **specify the function's name; do not include the parenthesis `( )`.**

Let's look at another example of how to pass an argument to the function. The built-in data `precip` recorded the average amount of annual precipitation (in inches) for 70 US cities. Suppose we want to convert inches to centimeters (cm) with precision provided by the caller; the default is zero.

Let's define a customized function for the conversion.

```
inch2cm <- function(inches, decimals = 0) {
  cm <- round(inches * 2.54, decimals)
  return(cm)
}
```

Next, convert all values in `precip` to cm using `lapply()`. In this call, the conversion is rounded up to the nearest whole number.

```
precip_cm <- lapply(precip, FUN=inch2cm)
print(precip_cm[1:3])
```

```
$Mobile
[1] 170
```

```
$Juneau
[1] 139

$Phoenix
[1] 18
```

Pass an additional argument, decimals=2, to `inch2cm` in the `lapply()` function.

```
precip_cm <- lapply(precip, FUN=inch2cm, 2)
print(precip_cm[1:3])
```

```
$Mobile
[1] 170.18

$Juneau
[1] 138.94

$Phoenix
[1] 17.78
```

5.5.3 `sapply()`

The "s" of `sapply()` stands for simplified. It functions almost exactly like `lapply()` except that it returns a vector or matrix. Recall that `lappy()` always returns a `list` regardless of the input.

Here's the same process to convert precipitation from inches to cm but using `sapply()`.

```
precip_cm_vector <- sapply(precip, FUN = inch2cm, 2)
print(precip_cm_vector[1:3])
```

```
 Mobile  Juneau Phoenix
 170.18  138.94   17.78
```

5.5.4 `apply()`

Here we will expand the apply operation on 2D objects like matrices or data frames using `apply()`. You can direct the function to all values in a row, a column, or both. The `apply()` function takes three arguments: a 1D or 2D object, margin, and function. Margin 1 means to

apply the function row by row; 2 means column by column. If both are given, the function will process all values of the 2D object.

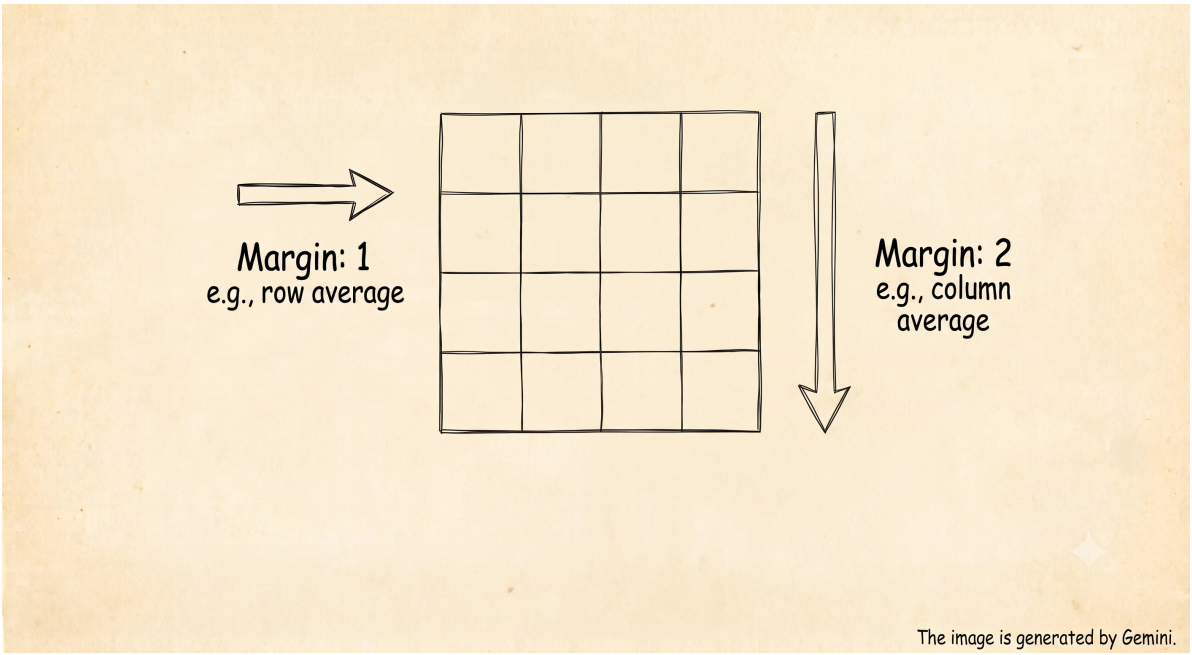


Figure 5.6: Illustration 5.1 Margins of apply().

For example, calculate the column mean of the four numeric variables in `iris`.

```
apply(iris[,1:4], MARGIN = 2, FUN=mean)
```

```
Sepal.Length  Sepal.Width Petal.Length  Petal.Width
    5.843333     3.057333     3.758000     1.199333
```

Convert all values from inches to cm in `iris`

```
iris_cm <- apply(iris[,1:4], MARGIN = c(1,2), FUN = inch2cm)
head(iris_cm)
```

```
     Sepal.Length Sepal.Width Petal.Length Petal.Width
[1,]           13           9            4           1
[2,]           12           8            4           1
[3,]           12           8            3           1
[4,]           12           8            4           1
```

```
[5,]           13            9            4            1
[6,]           14           10            4            1
```

6 R and RStudio

This chapter primarily focuses on using RStudio for data and statistical analysis. If you are new to RStudio, this chapter provides a quick overview of this indispensable interactive development environment (IDE). A note about names. You will come across the name Posit here. In 2022, the management team of RStudio wanted to expand the tool to support other programming languages, including Python and Julia. Therefore, they renamed RStudio to **Posit**. Since our focus is on R scripting, we will continue to refer to the IDE as RStudio, which is still very popular in the R community. Without further delay, let's get started.

Learning Objectives:

- Understand the differences between R and RStudio.
- Understand different options for accessing RStudio.
- Understand the basic components of the RStudio user interface.

6.1 R vs. RStudio

RStudio depends on R. Using a car analogy, nobody drives a car by pulling the engine's throttle directly. R is like the engine, and RStudio is like the dashboard. RStudio provides a user-friendly Interactive Development Environment (IDE) that lets users instruct R to perform the actual work.

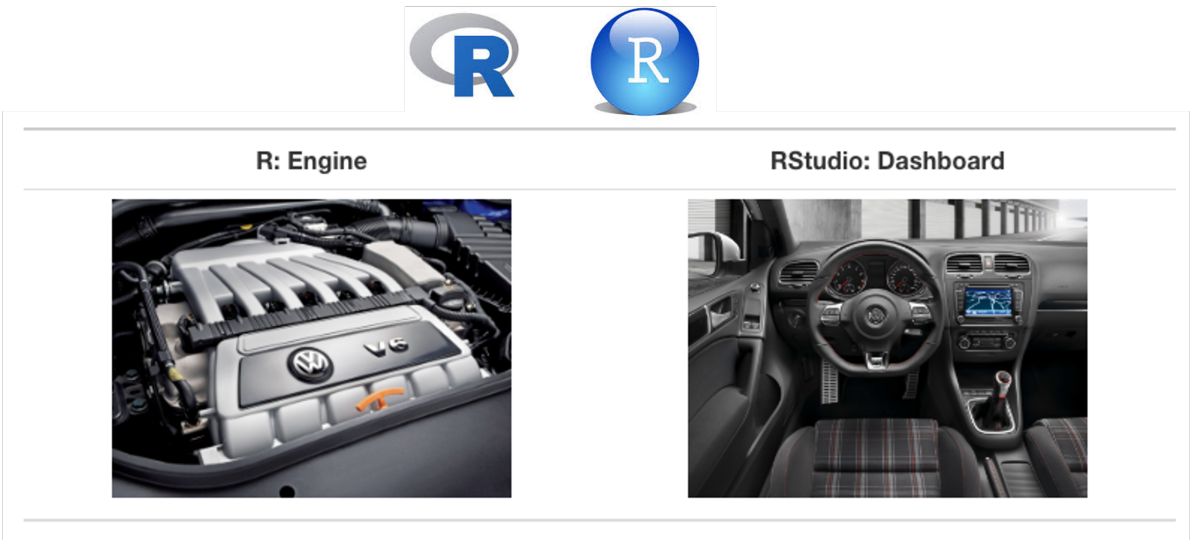


Figure 6.1: Figure 6.1: R vs. RStudio

6.2 Accessing RStudio

There are two options for accessing RStudio: using a cloud service (Posit Cloud) or installing RStudio on your own computer. For the first option, no software installation is required. All you need is a web browser and an internet connection. For the first time, sign up for a Posit account from here: Posit Cloud and choose a plan. A basic free plan is sufficient for everything discussed in this book. Your work will be saved in the cloud under your account. Regardless of what computer you use and where you work, everything you have saved will be available in your account.

Alternatively, if you prefer better control of RStudio, you can install two software packages, R and RStudio, in that sequence, on your own computers. Then it becomes your responsibility to maintain and update them. A favorable feature of R is that it can be installed on a Windows or a Mac computer. If your computer is a Chromebook, Posit Cloud is preferred, as installing R and RStudio on a Chromebook is less straightforward. For Windows and Mac, the installation is simple; it only takes a few double-clicks. Note that you must install R before RStudio. Here are the links to download R and RStudio.

6.3 Basic Components of RStudio

Below is a typical, default appearance of the RStudio interface.

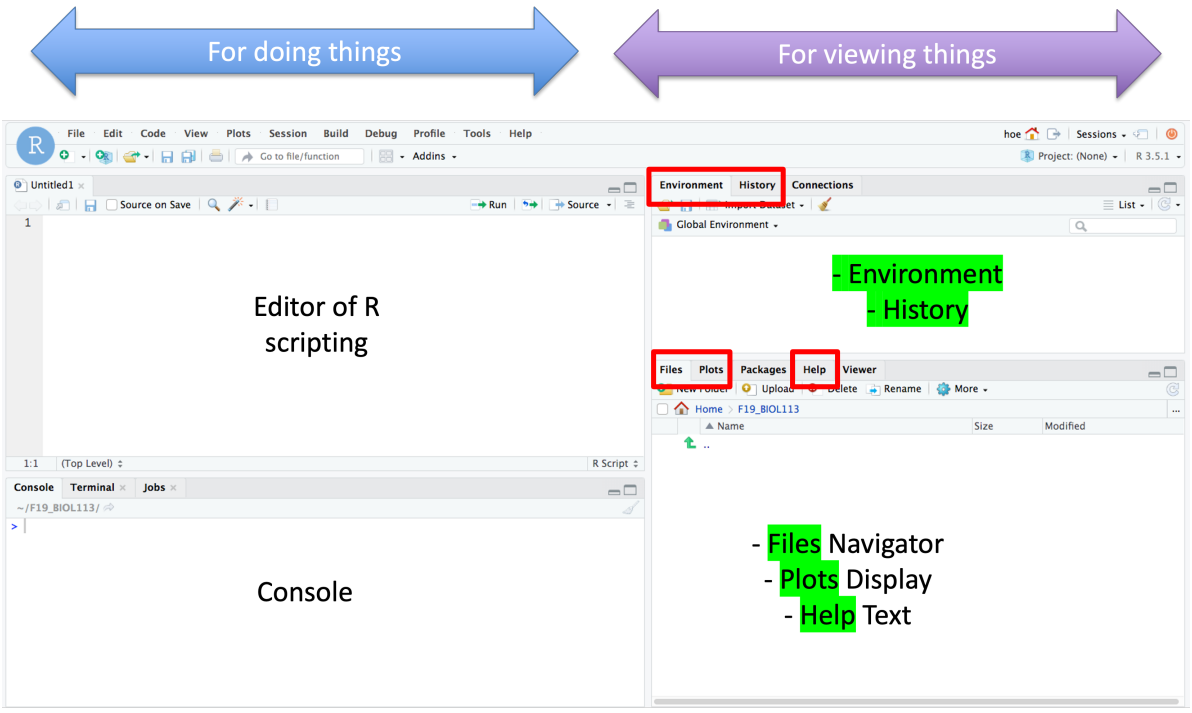


Figure 6.2: Figure 6.2: RStudio user interface

The interface is divided into **four panes**, each with a different function. The two panes on the left are designated for doing things. The two right panes are for viewing.

6.3.1 Files/Plots/Help

Let's begin with the lower right pane. Treat it like Finder on Mac or Explorer on Windows. The **Files** tab helps you navigate the cloud storage or local drive. It also provides basic file operations, such as renaming (**Rename** button) and deleting (**Delete** button) files. In the cloud version, you can use the **Upload** function to upload files from the local drive to the cloud. Likewise, files can be downloaded from the cloud to the local drive by checking the box next to the file(s) you want to download, then clicking the gear icon and selecting the **Export** option in the pull-down menu.

The **Plots** tab stores the graphs created in the current session. They will be removed once you sign off from the cloud or quit RStudio.

The **Help** tab shows the help documentation. E.g., if you invoke help for the following function, the help text will be shown in the Help window.

```
?hist
```

Often, I have found the examples at the bottom of help text the most helpful.

6.3.2 Console

The lower left pane is called the **console**. When you move the mouse anywhere within the console and click it, attention is on the console as shown below:

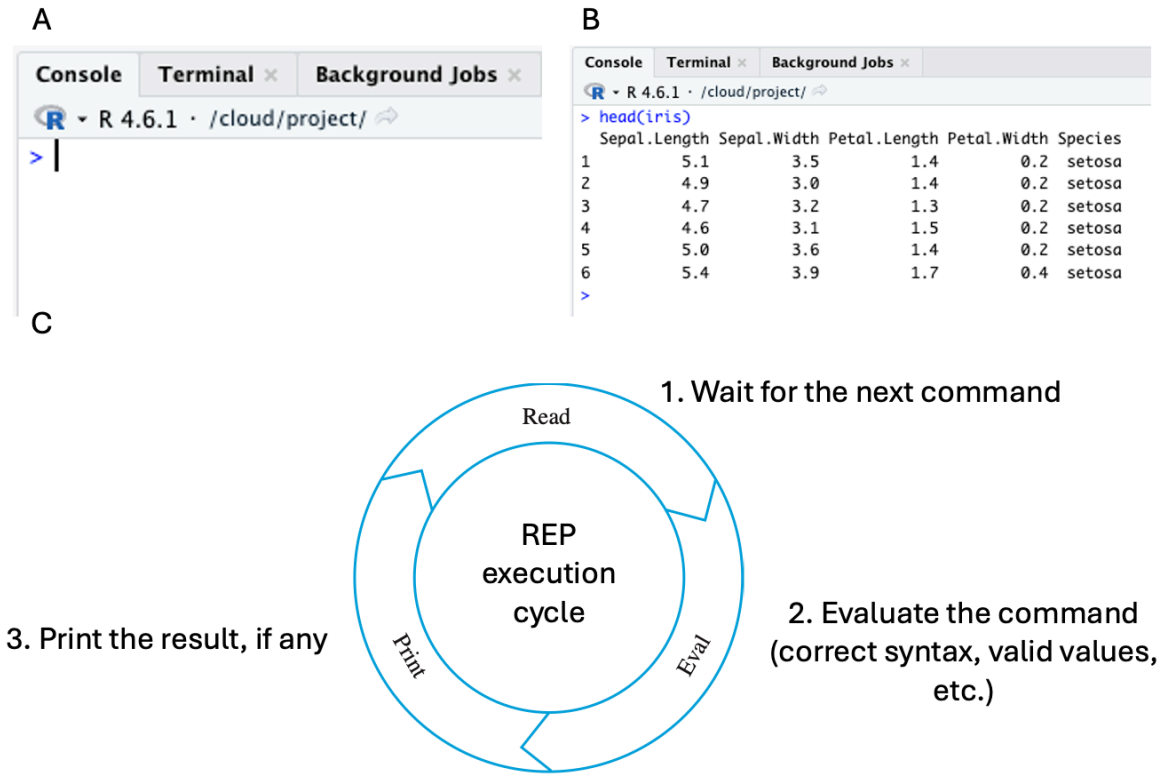


Figure 6.3: Figure 6.3: Console and REP cycle

The console is usually the place for rapid try-and-error, or **prototyping**. Additionally, all commands entered at the console are saved to a history log (discussed below).

The `>` symbol in Figure 6-3A is the prompt. Next to it is a blinking cursor, usually a vertical bar `|`. At this stage, the R engine is waiting for an instruction. For example, you enter `head(iris)` and hit the `return` button (Figure 6-3B), the R engine will go through the REP (**R**ead-**E**val-**P**rint) execution cycle (Figure 6-3C). At the Read stage, the R engine listens to

incoming commands, e.g., `head(iris)`. It will then evaluate whether the command(s) are valid (in terms of syntax and required input values). If the command is correct, the R engine will execute it and print the results, if any, at the console. If the command is invalid, an error message will be printed at the console, usually in red.

6.3.3 Environment/History

The **Environment** tab in the top right pane is like a CT scan of one's memory. It shows all the variables created in the current session. This feature is extremely useful for troubleshooting scripting problems.

RStudio splits the variables into three categories: **Data, Values,** and **Functions**. The Data subsection keeps data frames Section 1.2.5. There is a little play button next to each data frame. Clicking it will expand to a list of variables belonging to the data frame. The Values subsection lists other types of variables and a glimpse of their values, such as atomic variables Section 1.2.2, vectors Section 1.2.3, and list objects Section 5.1.1. Lastly, the Functions subsection shows the user-defined functions and the arguments Section 5.4.

The Environment tab also provides a function to import data into RStudio. See Section 5.3.2.

Another useful feature is the **History** tab. All commands executed, correct or incorrect, in the current session will be recorded in the History tab.

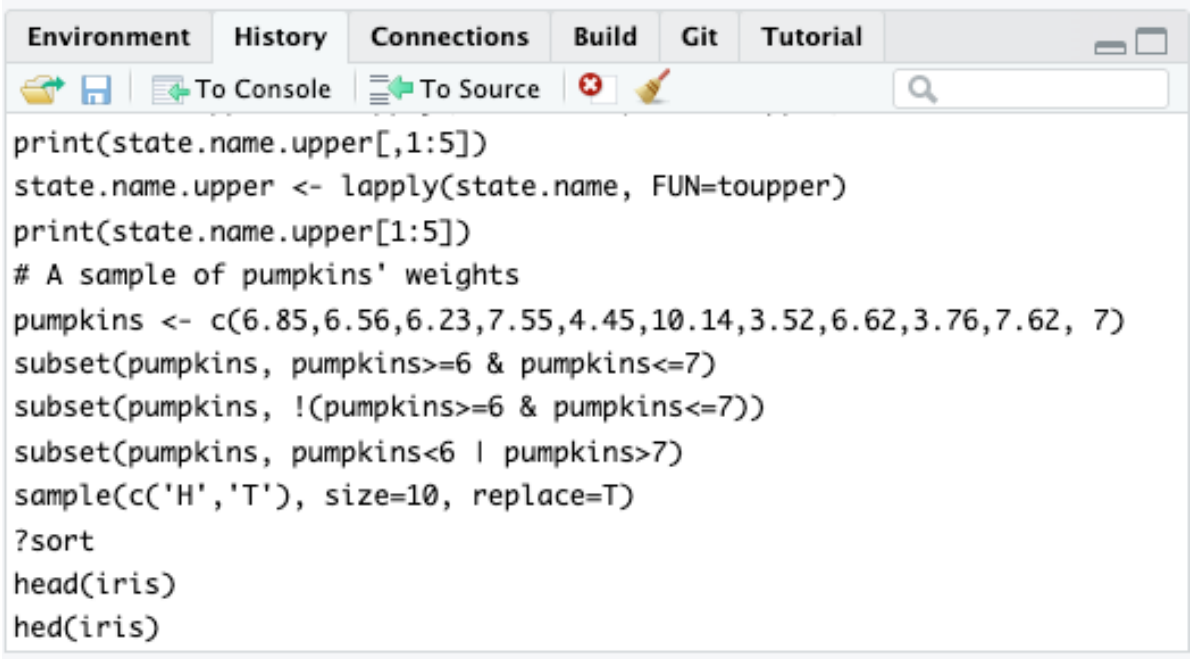


Figure 6.4: Figure 6.4: History tab

One way to make script development more convenient is to copy previously executed, tested commands from the History log to the console or a file without retyping them. Let's say you want to re-execute `head(iris)` again. You can highlight that command in the History tab and click the **"To Console"** button. The command will be copied to the console. Likewise, you can direct the copying to a script file opened by the editor (top left pane in Figure 6.2) by clicking the **"To Source"** button.

6.3.4 Editor

The last pane is the editor function on the top left. It is the place to write the R script that contains several R commands. To create a new R script file, click the menu option (top left) **File -> New File -> R Script**. A blank file is shown, and you can start writing R commands in it. Once you are done, you can save it by clicking the floppy disk icon near the top left or by selecting the File menu option **Save**. You can also use the File menu option to open an existing R script file.

There are two ways to execute the command(s) in the current script. To run the entire script file from the first line to the last line, click the **"Source"** button near the top left of the editor window. To run specific command(s), use the mouse to select the line(s) of interest and click the **"Run"** button, which is located next to the **"Source"** button.

6.4 Summary

This is a short overview of essential RStudio features. They should be adequate for 80% of the tasks. The two panes on the right side of the IDE are mainly for viewing, and the panes on the left side are mostly for doing.

You can use the **Files** tab to navigate through the files in the cloud drive or local drive. It also supports basic file operations such as deleting, renaming, uploading, and downloading files. The **Environment** tab is used to track variables (data frames, atomic variables, vectors, lists, etc.) created in the current session. The **History** tab contains a log of all the commands executed. Those commands can be copied to the console or an editing R script file conveniently using the "**To Console**" or "**To Source**" button.

The **Console** (usually in the lower-left) is the place to rapidly prototype command syntax and outcomes. To keep the commands you tested in a file, you can use the "**To Source**" function under the History tab.

The **Editor** pane is for creating or modifying R scripts. It provides functions to execute selected command(s) (**Run**) or the entire script file (**Source**).

Scripting is a practical skill. It is best learned through hands-on practice; it is like learning to swim. You don't just listen to instructions. You've got to jump into the water and get

wet! Therefore, I highly encourage you to try out the examples and exercises in the RStudio environment.

Bibliography